PrepNPlaced
prepnplaced.com

FROM ZERO + DSA + DATA ENGINEERING

PYTHON

FOR DATA ENGINEERING

END TO END NOTES, BASIC TO ADVANCED

From installing Python to DSA, Big-O and the coding patterns DE interviews keep reusing, with a 10-week plan to get there. Every topic comes with the approach, code that runs, and the real output.

80	7	18	47
PAGES	PARTS	CODING PATTERNS	SOLVED QUESTIONS

VARIABLES · LOOPS · DICTS · FUNCTIONS · GENERATORS · BIG-O · GRAPHS · PATTERNS · PANDAS · PYSPARK

Every snippet runs on Python 3.12. Libraries covered

Python · pandas · NumPy · PySpark · Airflow · pytest

- **Run, not typed**
  Every output panel is what Python printed when this PDF was built.
- **One dataset, 80 pages**
  The same store files from page 3 to page 80, shipped with the PDF.
- **Approach first**
  Each pattern shows how the question sounds and how to attack it.

stream_orders.py

```python
def read_orders(path):
    with open(path) as f:
        next(f)                  # skip header
        for line in f:           # 1 line in RAM
            *_, s, a = line.rstrip().split(",")
            yield s, float(a)

delivered = (a for s, a in read_orders("orders.csv")
             if s == "delivered")
print(f"revenue: {sum(delivered):,.2f}")
```

REAL OUTPUT, PYTHON 3.12

```
$ revenue: 18,972.00
```

A generator reads one line at a time, so this same code works on a 15-row file and a 15 GB one. Page 35 shows why.

THE ROUTE

01 **Python from zero** — install, variables, loops, lists, dicts, functions
02 **Python foundations** — objects, dicts, files, errors
03 **Advanced Python** — generators, OOP, async
04 **DSA and complexity** — Big-O, hashing, trees, graphs, DP
05 **Coding round patterns** — 18 patterns, solved
06 **Python for data engineering** — pandas, SQL, ETL, PySpark
07 **Practice and prep flow** — 10-week plan, cheat sheets

THE 10-WEEK FLOW, ZERO TO INTERVIEW (PAGE 79)

W1	Python basics	W6	Trees, graphs
W2	Data structures	W7	Patterns P1-P9
W3	Foundations	W8	Patterns P10-18
W4	Advanced + Big-O	W9	pandas, ETL
W5	Arrays, hashing	W10	Mocks

START HERE

WHAT IS INSIDE

One continuous path. Each page builds on the one before it, and every snippet runs on the same small store dataset.

START HERE

Start here: the running dataset 03
The reality: what DE coding rounds ask 04

PART 1 · PYTHON FROM ZERO

What Python is, and running your first program 05
Variables and names 06
Numbers and maths 07
Strings: working with text 08
True, False and comparisons 09
Input and output 10
Making decisions: if, elif, else 11
Loops part 1: for and range 12
Loops part 2: while, break, continue 13
Lists: ordered collections 14
Tuples and sets 15
Dictionaries: key and value 16
Which data structure? A side-by-side guide 17
Functions: reusable steps 18
Functions part 2: lambdas, sorting and building a pipeline 19
Modules, packages and pip 20
Reading errors without panic 21
Files: read and write 22
Mini project: a monthly sales report 23
Practice set 0: first steps 24

PART 2 · PYTHON FOUNDATIONS

The 6-step approach to any coding question 25
Names, objects and mutability 26
Numbers, strings and dates that bite 27
Lists, tuples, sets and Big-O 28
Dicts, defaultdict and Counter 29
Loops, comprehensions and unpacking 30
Functions, scope and closures 31
Errors, exceptions and context managers 32
Files: CSV, JSON, JSON Lines, paths 33
Practice set 1: foundations 34

PART 3 · ADVANCED PYTHON

Iterators and generators: lazy pipelines 35
itertools, functools and sorting 36
Classes and dataclasses for pipelines 37
Decorators: timing, retry, context managers 38
Concurrency: threads, processes, asyncio 39
Practice set 2: advanced 40

PART 4 · DSA AND COMPLEXITY

Big-O in plain words: how work grows 41
Space complexity: memory, recursion, streaming 42
Time vs space trade-offs 43
What Python operations really cost 44
Arrays and strings: two pointers, prefix sums 45
Hashing: two-sum, anagrams, subarray sums 46
Sliding windows of variable size 47
Stacks and queues: brackets, monotonic stacks 48
Linked lists: the three classics 49
Binary search, including search on the answer 50
Sorting: what Python does, and when to sort yourself 51
Recursion and backtracking 52
Trees: DFS, BFS, and the org chart 53
Graphs: BFS, cycles and topological sort 54
Heaps and greedy choices 55
Dynamic programming: four classics 56
Practice set 3: DSA with complexity 57

PART 5 · CODING ROUND PATTERNS

The pattern map: 18 patterns at a glance 58
P1 Frequency count · P2 Top-K 59
P3 Group by key · P4 Dedupe, keep latest 60
P5 Sliding window · P6 Merge sorted streams 61
P7 Flatten nested JSON 62
P8 Sessionize events · P9 Consecutive streaks 63
P10 Parse logs with regex 64
P11 Merge intervals · P12 Hash join 65
P13 Chunked processing · P14 Retry with backoff 66
P15 Paginated extraction · P16 Memoize, LRU cache 67
P17 Validate and quarantine · P18 Diff two snapshots 68

PART 6 · PYTHON FOR DATA ENGINEERING

pandas essentials, mapped to SQL 69
pandas pitfalls and performance 70
Python and databases: parameters, transactions, upserts 71
Build an ETL pipeline: idempotent, logged, configurable 72
Testing with pytest and data quality checks 73
From pandas to PySpark, scheduled with Airflow 74

PART 7 · PRACTICE, PREP FLOW AND REVISION

Interview set: Python coding for DE 75
Interview set: pandas and data wrangling 76
Concept questions with short answers 77
Complexity cheat sheet 78
The preparation flow: 10 weeks, zero to offer 79
Revision sheet and the last 48 hours 80

HOW TO USE THIS BOOK

Never written Python? Start at page 5 and follow the 10-week flow on page 79. **Already code in Python?** Skim Part 1, start at page 25, and skip the first two weeks of the flow. Short on time before an interview? Read page 4, then Part 4 (DSA) for algorithm rounds or Part 5 (data patterns) for data-processing rounds.

Code blocks marked **Python 3.12** were run while this PDF was built, and the output panel under or beside each one is what Python printed. Blocks marked with something else (Airflow, requests) show code that needs a live service, so they are not run.

START HERE

ONE DATASET, START TO FINISH

The same online store from the SQL End to End Notes, now as the files a data engineer actually receives: CSV, JSON, JSON Lines and a log file.

These notes take you from Python basics to the patterns that decide DE coding rounds, then into pandas, databases, testing and PySpark. The pattern section comes from the questions I keep seeing across the 20+ analytics and data engineering interviews I have sat, and the ones my mentees report back from their own loops. Every snippet reads the files below, and every output panel is what Python 3.12 printed when this PDF was built. The files ship with this PDF as `python_de_dataset.zip`.

The files

File	Format	Records	What's inside	Used for
`orders.csv`	CSV	15	order_id, customer_id, order_date, status, amount	Most of the book
`order_items.csv`	CSV	24	order_id, product_id, qty, unit_price	Group by, joins
`products.csv`	CSV	8	product_id, product_name, category, price	Lookups, pandas
`customers.json`	JSON	8	Nested address {city, pincode}, tags list	Flattening (P7), joins
`events.jsonl`	JSON Lines	12	user_id, ts, event, props {page, device}	Sessions, funnels
`logins.csv`	CSV	12	user_id, login_date	Streaks (P9)
`employees.csv`	CSV	9	emp_id, name, dept, manager_id, salary	Top-K, ties
`app.log`	Text	14	Pipeline log lines, one malformed	Regex parsing (P10)
`orders_dirty.csv`	CSV	7	Bad rows on purpose: negative, blank, bad date	Validation (P17)
`snapshot_day1/2.csv`	CSV	8 / 8	Customers on two days	Change capture (P18)

A first look

A FIRST LOOK — PYTHON 3.12

```python
import csv, json

with open("orders.csv", newline="") as f:
    first = next(csv.DictReader(f))
print(first)

customers = json.load(open("customers.json"))
print(customers[5])
```

OUTPUT

```
{'order_id': '101', 'customer_id': '1', 'order_date': '2025-01-10', 'status': 'delivered', 'amount': '1198.00'}
{'customer_id': 6, 'name': 'Sana', 'signup_date': '2025-03-15', 'referred_by': None, 'address': {'city': None, 'pincode': None
  }, 'tags': []}
```

PLANTED TRAPS

Two rows are planted on purpose. Sana (customer 6) has no city: her address holds None. Zoya (customer 8) never ordered. Many of the traps in this book trace back to those two records.

REMEMBER

CSV gives you every value as a string, even 1198.00. Converting types is your job, and it is where most parsing bugs live. Look at the output above: amount is the text "1198.00".

DA data wrangling round DE data engineering coding round EASY MEDIUM HARD difficulty

START HERE

THE REALITY OF THE CODING ROUND

Before you open LeetCode, find out which kind of coding round you are walking into. Data engineering loops use two different kinds, and they need different preparation.

APPROACH

Kind 1: DSA-style round

Looks like a software engineering round with the difficulty turned down. One or two problems in 40 to 45 minutes, usually LeetCode easy to medium: arrays, strings, hash maps, two pointers, heaps, stacks, intervals. Trees and graphs show up sometimes, more often in senior loops.

Prepared by: Part 4 of this book (pages 41 to 57).

DATA ENGINEERING NOTE

Kind 2: data-manipulation round

You get messy records: a log file, a JSON payload, a CSV with bad rows, an event stream. You parse, clean, group, dedupe and aggregate them, often without pandas, and then get asked how it scales to 50 GB.

Prepared by: Part 5 (pages 58 to 68) and the interview sets on pages 75 and 76.

Many companies use one of these, some use both, and almost all add SQL. Ask the recruiter one direct question: **"Will the coding round be algorithm problems, or data-processing tasks on records?"** Then read the most recent interview reports for that company and role, because the format changes between teams and years.

What comes up in DE coding rounds, and how deep to go

Topic	In DE loops	How deep	Pages
Hash maps and sets: counting, grouping, two-sum	**Very often**	Must be automatic	46, 59-60
Arrays and strings: two pointers, prefix sums, parsing	**Very often**	Must be automatic	45, 64
Sorting with custom keys, stability	Often	Know when and why	36, 51
Sliding windows (fixed and variable)	Often	Easy and medium	47, 61
Heaps: top-k, merge k sorted, running median	Often	Easy and medium	55, 59
Stacks, queues, intervals	Often	Easy and medium	48, 65
Binary search (including "on the answer")	Sometimes	Know the template	50
Trees and graphs: BFS, DFS, topological sort	Sometimes	Classics only	53, 54
Recursion and backtracking	Sometimes	Small inputs only	52
Dynamic programming	Rarely	Four classics	56
Linked lists	Rarely	Three classics	49

These labels are a starting map built from candidate reports and the loops my mentees bring back, not a promise. Software engineering roles go deeper on graphs and dynamic programming than DE roles do.

What the interviewer actually scores

- **Correctness** on the edge cases you were not told about: empty input, duplicates, ties, None.
- **Communication**: you said the approach before typing and talked while coding.
- **Complexity**: you stated time and space without being asked, and knew a better option exists.
- **Code quality**: names that read, small functions, no copy-paste.
- **Follow-ups**: "what if it is 50 GB?" This is where DE rounds differ from SDE rounds.

What changes with experience

Level	Same problems, but they expect
0 to 2 years	A working solution, clear talk, and the Big-O stated
2 to 5 years	The optimal approach first time, edge cases unprompted, production-style code
5+ years	Trade-offs, scaling (memory, partitions, parallelism) and design judgement on every answer

PART 1 · PYTHON FROM ZERO · LESSON 1

YOUR FIRST PYTHON PROGRAM

L1

Python is a language for giving a computer step-by-step instructions in words that read almost like English. This page gets one running.

A Python program is a text file ending in **.py**. Python reads it **from the top line to the bottom**, one instruction at a time. Data engineers use it to read files, clean data, call APIs and load warehouses, because the code stays short and the libraries already exist for almost every job.

Step 1: set up once

YOUR TERMINAL — TERMINAL · NOT RUN

```
# 1. Check Python is installed (3.10 or newer)
python3 --version

# 2. Make a folder and a virtual environment for it
mkdir python_de && cd python_de
python3 -m venv .venv
source .venv/bin/activate
# on Windows: .venv\Scripts\activate

# 3. Save the code on the right as hello.py, then run it
python3 hello.py
```

No Python yet? Install it from python.org (tick "Add Python to PATH" on Windows). Use VS Code as your editor, or Jupyter if you like running code in small pieces.

Step 2: your first program

HELLO.PY — PYTHON 3.12

```
# Lines starting with # are comments.
# Python ignores them; they are notes for humans.

print("Hello, data engineer!")
print("Orders today:", 15)
print(2 + 3)
```

OUTPUT

```
Hello, data engineer!
Orders today: 15
5
```

Three rules Python is strict about

RULE 1: INDENTATION MEANS SOMETHING — PYTHON 3.12

```
amount = 1198
if amount > 1000:
    print("big order")  # 4 spaces: inside the if
print("done")          # back left: after the if
```

OUTPUT

```
big order
done
```

RULE 2: CAPITALS MATTER — PYTHON 3.12

```
print("Amount")
Print("Amount")        # capital P: another name
```

OUTPUT

```
Amount
NameError: name 'Print' is not defined. Did you mean: 'p
  rint'?
```

REMEMBER

Rule 3: one instruction per line. Python runs each line in order. If line 5 has an error, lines 1 to 4 have already run and line 6 never does.

INTERVIEW TIP

Read error messages from the bottom up. The last line says what went wrong (here: **NameError**, Python has never heard of "Print"). Lesson 17 reads them in detail.

Word	Meaning in plain English
Program / script	A .py file with instructions
Terminal	The text window where you type commands like python3 hello.py
Interpreter	The python3 program that reads your file and runs it line by line
Output	Whatever your program prints. Every grey OUTPUT box in this book is real output
Virtual environment (.venv)	A private folder of libraries for one project, so projects do not clash

TRY IT YOURSELF BEFORE MOVING ON

1. Print your name and your city on two separate lines. **2.** Print the result of 1198 + 796 with a label in front of it. **3.** Remove the 4 spaces before print("big order") and read the error Python gives you.

PART 1 · PYTHON FROM ZERO · LESSON 2

VARIABLES: GIVING DATA A NAME

L2

A variable is a label you stick on a value so you can use it later. Most programs are just values, labels and steps.

Writing **orders = 15** means "remember 15, and call it orders". The = sign is not "equals" from maths; read it as **"becomes"**. You can change what a label points at any time, and Python always uses the latest value.

CREATE, CHANGE, CHECK THE TYPE — PYTHON 3.12

```
orders = 15                  # orders becomes 15
city = "Bengaluru"           # text goes in quotes
price = 799.0                # numbers with a dot are decimals
is_delivered = True          # True / False, capital first letter

print(orders, city, price, is_delivered)

orders = orders + 1          # use the old value to make a new one
print("now:", orders)

print(type(orders), type(city), type(price), type(is_delivered))
```

OUTPUT

```
15 Bengaluru 799.0 True
now: 16
<class 'int'> <class 'str'> <class 'float'> <c
  lass 'bool'>
```

REMEMBER

`type()` tells you what kind of value a variable holds: int (whole number), str (text), float (decimal), bool (True/False). Most beginner bugs are a value of the wrong type.

HANDY SHORTCUTS — PYTHON 3.12

```
a, b = 10, 20                # two at once
a, b = b, a                  # swap without a third variable
print(a, b)

x = y = 0                    # same value, two names
GST_RATE = 0.18              # CAPITALS: a value you never change
total = 1000 * (1 + GST_RATE)
print(x, y, total)
```

OUTPUT

```
20 10
0 0 1180.0
```

Python does not lock constants. Writing them in capitals is a promise to the reader that the value never changes.

Naming rules

Allowed	Not allowed	Why
`order_total`	`order total`	No spaces. Use _ between words (this style is called snake_case)
`order2`	`2order`	Cannot start with a digit
`total_amount`	`class, for, if`	Words Python already uses (keywords) are reserved
`customer_city`	`cc, x1, data2`	Allowed, but meaningless names make code hard to read in interviews

NAMES ARE CASE-SENSITIVE — PYTHON 3.12

```
Amount = 100
amount = 200
print(Amount, amount)        # two different variables
```

OUTPUT

```
100 200
```

USE BEFORE CREATE — PYTHON 3.12

```
print(total_orders)          # used before it was created
```

OUTPUT

```
NameError: name 'total_orders' is not defined
```

Create a variable on an earlier line than the one that uses it. Python reads top to bottom.

TRY IT YOURSELF BEFORE MOVING ON

1. Store a product name, its price and a quantity, then print the total cost. **2.** Swap the values of two variables without a third one. **3.** Print type(10), type(10.0) and type("10"). Why are they different?

PART 1 · PYTHON FROM ZERO · LESSON 3

NUMBERS AND MATHS

L3

Python is a calculator that never gets tired. Learn the operators once and you can total, average and split any amount of data.

THE OPERATORS — PYTHON 3.12

```
print(7 + 2, 7 - 2, 7 * 2)
print(7 / 2)        # true division: always a decimal
print(7 // 2)       # floor division: drop the decimals
print(7 % 2)        # remainder (modulo)
print(2 ** 10)      # power: 2 to the 10
print(2 + 3 * 4, (2 + 3) * 4)   # * before +, brackets first
```

OUTPUT

```
9 5 14
3.5
3
1
1024
14 20
```

Symbol	Does
+ - *	Add, subtract, multiply
/	Divide, gives a float
//	Divide, round down
%	Remainder. x % 2 == 0 means x is even
**	Power

EVERYDAY SUMS — PYTHON 3.12

```
amount = 1198
gst = amount * 0.18
print("GST:", gst, "rounded:", round(gst, 2))

total = 0
total += 1198          # same as total = total + 1198
total += 796
total -= 50            # a discount
print("total:", total)

orders, days = 15, 4
print("average per day:", orders / days, round(orders / days, 1))
print(abs(-50), max(3, 9, 4), min(3, 9, 4))
```

OUTPUT

```
GST: 215.64 rounded: 215.64
total: 1944
average per day: 3.75 3.8
50 9 3
```

REMEMBER

+= and -= are short for "add to what is already there". You will use them in nearly every loop to build totals.

CONVERTING BETWEEN TEXT AND NUMBERS — PYTHON 3.12

```
text_amount = "1198.50"         # from a file: it is text!
print(text_amount + "1")       # joins text, does not add

amount = float(text_amount)    # convert text to a number
print(amount + 1)
print(int("15") + 5, int(9.99), str(42) + " orders")
```

OUTPUT

```
1198.501
1199.5
20 9 42 orders
```

TRAP

Everything you read from a file or the keyboard arrives as text. "1198" + "1" is "11981", not 1199. Convert with int() or float() first. Note that int(9.99) cuts the decimals off; it does not round.

INTERVIEW TIP

Floats can show tiny surprises: 0.1 + 0.2 prints 0.30000000000000004. That is normal for computers, not a Python bug. For money you will use Decimal (page 27).

TRY IT YOURSELF BEFORE MOVING ON

1. Is 2025 an even number? Use %. **2.** Split 2,847 rupees equally between 4 people, rounded to 2 decimals. **3.** Turn the text "1,499" into the number 1499 (hint: replace, then int).

PART 1 · PYTHON FROM ZERO · LESSON 4

STRINGS: WORKING WITH TEXT

L4

Names, cities, statuses, file paths and log lines are all text. In Python text is called a string (str).

BUILD, MEASURE, SLICE — PYTHON 3.12

```
name = "Aarav"
city = 'Bengaluru'                # single or double quotes
print(name + " from " + city)     # + joins strings
print("-" * 20)                   # * repeats
print(len(name))                  # number of characters

print(name[0], name[-1])          # first and last character
print(city[0:4], city[4:])        # slices: [start:stop]
```

OUTPUT

```
Aarav from Bengaluru
--------------------
5
A v
Beng aluru
```

REMEMBER

Positions start at 0, not 1. name[0] is the first letter. Negative positions count from the end: name[-1] is the last. A slice [0:4] takes positions 0, 1, 2 and 3 and stops before 4.

THE METHODS YOU WILL USE EVERY DAY — PYTHON 3.12

```
status = "  Delivered  "
print("[" + status.strip() + "]")        # remove outer spaces
print(status.strip().lower())            # small letters
print("delivered".upper(), "pune".title())
print("2025-04-01".replace("-", "/"))
print("101,1,delivered".split(","))       # text -> list
print(" | ".join(["a", "b", "c"]))       # list -> text
print("deliver" in "delivered", "orders.csv".endswith(".csv"))
```

OUTPUT

```
[Delivered]
delivered
DELIVERED Pune
2025/04/01
['101', '1', 'delivered']
a | b | c
True True
```

TRAP

String methods return a **new** string; they never change the original. `status.strip()` on its own line does nothing useful. Write `status = status.strip()`.

F-STRINGS: THE CLEAN WAY TO BUILD TEXT — PYTHON 3.12

```
name, orders, revenue = "Diya", 3, 4143.0
print(f"{name} placed {orders} orders worth {revenue}")
print(f"{revenue:,.2f}")          # thousands separator, 2 decimals
print(f"{orders:03d}")            # pad with zeros: 003
print(f"{name:<8}|{name:>8}|")    # align left / right in 8 spaces
print(f"{0.256:.1%}")             # as a percentage
```

OUTPUT

```
Diya placed 3 orders worth 4143.0
4,143.00
003
Diya    |    Diya|
25.6%
```

Put an `f` before the quotes and any variable inside `{ }` is filled in. After a colon you can format it: decimals, commas, padding, percent.

INTERVIEW TIP

Special characters: "\n" starts a new line, "\t" is a tab. Triple quotes """...""" let a string span several lines, which is handy for SQL queries inside Python.

TRY IT YOURSELF BEFORE MOVING ON

1. Print the first 3 letters of "Hyderabad". **2.** Turn "order_date" into "Order Date" (replace and title). **3.** Count the letter "a" in "Bengaluru" with .count("a").

PART 1 · PYTHON FROM ZERO · LESSON 5

TRUE, FALSE AND COMPARISONS

L5

Every decision a program makes comes down to one question with a True or False answer.

COMPARISONS GIVE TRUE OR FALSE — PYTHON 3.12

```
amount = 1198
print(amount > 1000, amount < 500)
print(amount == 1198, amount != 1198)     # == compares, = assigns
print(amount >= 1198, amount <= 1000)
print(500 < amount < 2000)                # chained: between

status = "delivered"
print(status == "Delivered")              # capitals matter
print(status.lower() == "delivered")
```

OUTPUT

```
True False
True False
True False
True
False
True
```

TRAP

= puts a value in a variable. == asks "are these equal?". Mixing them up is the most common beginner error.

AND, OR, NOT — PYTHON 3.12

```
amount, status = 1198, "delivered"

print(amount > 1000 and status == "delivered")   # both must be True
print(amount > 5000 or status == "delivered")    # at least one
print(not status == "cancelled")                 # flips it
```

OUTPUT

```
True
True
True
```

A	B	A and B	A or B
True	True	True	True
True	False	False	True
False	False	False	False

"TRUTHY" AND "FALSY" VALUES — PYTHON 3.12

```
for value in [0, 15, "", "Pune", [], [1, 2], None]:
    print(repr(value), "->", bool(value))
```

OUTPUT

```
0 -> False
15 -> True
'' -> False
'Pune' -> True
[] -> False
[1, 2] -> True
None -> False
```

REMEMBER

Empty things count as False: 0, an empty string "", an empty list [], an empty dict {} and None. Everything else counts as True. That is why `if name:` is a common way to say "if name is not empty".

Write	Means
`x == y` `x != y`	Equal / not equal
`x > y` `x < y` `x >= y` `x <= y`	Greater, less, or equal to
`a and b`	True only if both are True
`a or b`	True if at least one is True
`not a`	The opposite
`x in [1, 2, 3]`	True if x is one of the items
`x is None`	The correct way to test for "no value"

TRY IT YOURSELF BEFORE MOVING ON

1. Write one line that is True when 1499 is between 1000 and 2000. 2. Is the empty string "" truthy or falsy? Check with bool(). 3. Write a condition that is True only for delivered orders above 1,000.

PART 1 · PYTHON FROM ZERO · LESSON 6

INPUT AND OUTPUT

L6

print() sends text out. input() takes text in. Between the two you can already build small interactive tools.

PRINT IN DETAIL — PYTHON 3.12

```
print("a", "b", "c")                 # spaces between items
print("a", "b", "c", sep=" | ")      # choose the separator
print("loading", end="... ")         # stay on the same line
print("done")
print()                              # an empty line
print("Line 1\nLine 2")             # \n is a new line
```

OUTPUT

```
a b c
a | b | c
loading... done

Line 1
Line 2
```

ASKING THE USER — PYTHON 3.12

```
name = input("Your name: ")
qty = int(input("How many items? "))    # input is always text
price = float(input("Price per item: "))

total = qty * price
print()
print(f"Hi {name}, your total is {total:,.2f}")
```

OUTPUT

```
Your name: How many items? Price per item:
Hi Meera, your total is 747.00
```

REMEMBER

When this ran for the book, we typed **Meera**, **3** and **249**. The typed text does not appear in the output box, only the prompts and the result.

input() always returns text, so wrap it in `int()` or `float()` before doing maths.

PROTECTING AGAINST BAD INPUT — PYTHON 3.12

```
raw = "abc"                  # what if the user types letters?
try:
    qty = int(raw)
except ValueError:
    print("Please type a whole number, not", repr(raw))
```

OUTPUT

```
Please type a whole number, not 'abc'
```

int("abc") raises a ValueError and would stop the program. try/except catches it so you can ask again or skip the row. Lesson 17 and page 32 go further.

PRINTING A NEAT RECEIPT — PYTHON 3.12

```
items = [("Wireless Mouse", 1, 799), ("Gel Pens", 3, 149),
         ("Water Bottle", 1, 399)]

print(f"{'item':<16}{'qty':>4}{'amount':>10}")
print("-" * 30)
total = 0
for name, qty, price in items:
    line = qty * price
    total += line
    print(f"{name:<16}{qty:>4}{line:>10,.2f}")
print("-" * 30)
print(f"{'total':<16}{'':>4}{total:>10,.2f}")
```

OUTPUT

```
item             qty    amount
------------------------------
Wireless Mouse     1    799.00
Gel Pens           3    447.00
Water Bottle       1    399.00
------------------------------
total                 1,645.00
```

Widths like :<16 (left, 16 wide) and :>10 (right, 10 wide) line up columns. This is how small reports and log lines are made readable.

DATA ENGINEERING NOTE

Data pipelines rarely ask a person for input. They get it from files, databases and APIs (Lesson 18), or from command-line options like --date (page 72). The habit is the same: treat everything that comes in as text, convert it, and handle the bad cases.

TRY IT YOURSELF BEFORE MOVING ON

1. Ask for two numbers and print their sum. **2.** Print a receipt for three items with the prices lined up on the right. **3.** What happens if the user types 3.5 when your code calls int()? Try it.

PART 1 · PYTHON FROM ZERO · LESSON 7

MAKING DECISIONS: IF, ELIF, ELSE

L7

if runs a block of code only when a condition is True. elif and else handle the other cases.

ONE DECISION, THREE OUTCOMES — PYTHON 3.12

```
amount = 1198

if amount >= 2000:
    bucket = "large"
elif amount >= 1000:            # checked only if the first was False
    bucket = "medium"
else:                           # none of the above
    bucket = "small"

print(amount, "->", bucket)
```

OUTPUT

```
1198 -> medium
```

REMEMBER

Python checks the conditions from top to bottom and runs only the **first** block that is True. That is why the order matters: put the strictest test first.

A REAL BUSINESS RULE — PYTHON 3.12

```
def delivery_fee(amount, city, is_member):
    if is_member:
        return 0
    if city in ["Bengaluru", "Mumbai", "Delhi"]:
        if amount >= 500:                   # an if inside an if
            return 0
        return 40
    return 80

print(delivery_fee(300, "Pune", True))
print(delivery_fee(799, "Mumbai", False))
print(delivery_fee(300, "Mumbai", False))
print(delivery_fee(300, "Pune", False))
```

OUTPUT

```
0
0
40
80
```

Each return leaves the function immediately, so once a rule matches, the rest is skipped. This style, called early return, keeps nesting shallow.

ONE-LINE IF, AND MATCH — PYTHON 3.12

```
amount = 796
label = "free delivery" if amount >= 500 else "pay delivery"
print(label)

status = "returned"
match status:                       # Python 3.10+
    case "delivered":
        print("count the revenue")
    case "cancelled" | "returned":
        print("exclude from revenue")
    case _:
        print("unknown status")
```

OUTPUT

```
free delivery
exclude from revenue
```

TRAP

A colon : ends every if, elif and else line, and the code under it must be indented. Forgetting either gives a SyntaxError or IndentationError before anything runs.

TRY IT YOURSELF BEFORE MOVING ON

1. Grade an order: 2,000 or more is "A", 1,000 or more is "B", otherwise "C". **2.** Check if a year is a leap year: divisible by 4, but not by 100 unless also by 400. **3.** Rewrite delivery_fee using match instead of if.

PART 1 · PYTHON FROM ZERO · LESSON 8

LOOPS PART 1: FOR AND RANGE

L8

A loop repeats code for every item in a collection. This is where programs start saving you real time.

LOOP OVER A LIST AND A STRING — PYTHON 3.12

```
amounts = [1198, 796, 1499, 1999]

for amount in amounts:          # amount takes each value in turn
    print("order of", amount)

for letter in "Pune":           # strings are collections too
    print(letter, end=" ")
print()
```

OUTPUT

```
order of 1198
order of 796
order of 1499
order of 1999
P u n e
```

RANGE: LOOPS THAT COUNT — PYTHON 3.12

```
print(list(range(5)))           # 0 up to 4 (5 is not included)
print(list(range(1, 6)))        # 1 up to 5
print(list(range(0, 20, 5)))    # steps of 5
print(list(range(5, 0, -1)))    # counting down

for day in range(1, 4):
    print(f"processing day {day}")
```

OUTPUT

```
[0, 1, 2, 3, 4]
[1, 2, 3, 4, 5]
[0, 5, 10, 15]
[5, 4, 3, 2, 1]
processing day 1
processing day 2
processing day 3
```

REMEMBER

`range(stop)` starts at 0 and stops **before** stop. `range(1, 6)` gives 1 to 5. This "up to but not including" rule is the same as slicing.

THE THREE LOOP PATTERNS YOU WILL USE FOREVER — PYTHON 3.12

```
amounts = [1198, 796, 1499, 1999, 1348]

total, count, biggest = 0, 0, amounts[0]
for a in amounts:
    total += a                  # accumulate
    count += 1                  # count
    if a > biggest:             # track the maximum
        biggest = a
print(total, count, biggest, round(total / count, 1))

for i, a in enumerate(amounts, start=1):   # position + value
    if a > 1400:
        print(f"order #{i} is big: {a}")
```

OUTPUT

```
6840 5 1999 1368.0
order #3 is big: 1499
order #4 is big: 1999
```

APPROACH

Accumulate: start a total at 0 and add to it. **Count:** start at 0 and add 1. **Track best:** keep the biggest so far and replace it when you find a bigger one. Every report you ever build is some mix of these three.

INTERVIEW TIP

Nested loops: a loop inside a loop runs the inner loop completely for each outer item. 10 outer x 10 inner = 100 runs. Keep that multiplication in mind; page 41 explains why it matters.

TRY IT YOURSELF BEFORE MOVING ON

1. Print the 7 times table from 7 x 1 to 7 x 10. **2.** Add up all the odd numbers from 1 to 99 with range and a step. **3.** Print each city in a list with its position, starting at 1.

PART 1 · PYTHON FROM ZERO · LESSON 9

LOOPS PART 2: WHILE, BREAK, CONTINUE

L9

A while loop repeats as long as a condition stays True. break and continue control any loop from the inside.

WHILE: REPEAT UNTIL SOMETHING CHANGES — PYTHON 3.12

```
balance = 1000
day = 0
while balance > 0:              # keep going while this is True
    day += 1
    balance -= 300
    print(f"day {day}: balance {balance}")
print("ran out on day", day)
```

OUTPUT

```
day 1: balance 700
day 2: balance 400
day 3: balance 100
day 4: balance -200
ran out on day 4
```

TRAP

If nothing inside the loop ever makes the condition False, it runs forever. Press Ctrl + C in the terminal to stop it. Always check: what changes on every pass?

CONTINUE SKIPS ONE ITEM, BREAK STOPS THE LOOP — PYTHON 3.12

```
statuses = ["delivered", "", "cancelled",
            "delivered", "STOP", "delivered"]

delivered = 0
for s in statuses:
    if s == "":
        continue                # skip this one, go to the next
    if s == "STOP":
        break                   # leave the loop completely
    if s == "delivered":
        delivered += 1
print("delivered before STOP:", delivered)
```

OUTPUT

```
delivered before STOP: 2
```

A REAL USE: RETRY UP TO THREE TIMES — PYTHON 3.12

```
attempt = 0
connected = False
while not connected and attempt < 3:
    attempt += 1
    connected = attempt == 3            # pretend the 3rd try works
    result = "connected" if connected else "failed"
    print(f"try {attempt}: {result}")

if not connected:
    print("giving up")
```

OUTPUT

```
try 1: failed
try 2: failed
try 3: connected
```

This is the skeleton of every retry in data engineering: a counter, a limit, and a flag that ends the loop early. Page 37 turns it into a reusable decorator.

Use for when	Use while when
You know what you are looping over: a list, a file, range(10)	You loop until something happens: money runs out, a retry succeeds
Most of the time in data work	Waiting, polling, retrying, reading an API page by page

FIND THE FIRST MATCH, THEN STOP — PYTHON 3.12

```
amounts = [796, 1198, 2397, 1499, 3498]

found_at = None
for i, a in enumerate(amounts):
    if a > 2000:
        found_at = i
        break                           # stop at the first match
print("first above 2,000:", amounts[found_at], "at", found_at)
```

OUTPUT

```
first above 2,000: 2397 at 2
```

Set a "not found yet" value (None) before the loop, then break when you find what you want. Without break the loop keeps going and wastes time on a big list.

TRY IT YOURSELF BEFORE MOVING ON

1. Start at 1 and keep doubling until you pass 1,000. How many doublings? **2.** Sum a list but stop at the first negative number. **3.** Sum a list of text amounts, skipping the empty ones with continue.

PART 1 · PYTHON FROM ZERO · LESSON 10

LISTS: ORDERED COLLECTIONS

L10

A list holds many values in order, in one variable. It is the first data structure you need and the one you will use most.

CREATE, ADD, REMOVE, CHANGE — PYTHON 3.12

```
cities = ["Bengaluru", "Mumbai", "Delhi"]
print(cities[0], cities[-1], len(cities))

cities.append("Pune")            # add to the end
cities.insert(1, "Chennai")      # add at position 1
print(cities)

cities.remove("Delhi")           # remove by value
last = cities.pop()              # remove and return the last item
print(cities, "| popped:", last)

cities[0] = "Bangalore"          # change an item
print(cities, "Mumbai" in cities)
```

OUTPUT

```
Bengaluru Delhi 3
['Bengaluru', 'Chennai', 'Mumbai', 'Delhi', 'Pu
  ne']
['Bengaluru', 'Chennai', 'Mumbai'] | popped: Pu
  ne
['Bangalore', 'Chennai', 'Mumbai'] True
```

REMEMBER

Lists are **mutable**: you can change them after creating them. Positions work exactly like strings: start at 0, negative counts from the end.

SUM, SORT, SLICE — PYTHON 3.12

```
amounts = [1198, 796, 1499, 1999, 1348]
print(sum(amounts), min(amounts), max(amounts))
print(sorted(amounts))                  # a new, sorted list
print(sorted(amounts, reverse=True)[:3])  # top 3
print(amounts[1:3], amounts[::-1])      # slice, reversed

amounts.sort()                          # sorts this list in place
print(amounts)

big = [a for a in amounts if a > 1300]  # see page 30

print(big)
```

OUTPUT

```
6840 796 1999
[796, 1198, 1348, 1499, 1999]
[1999, 1499, 1348]
[796, 1499] [1348, 1999, 1499, 796, 1198]
[796, 1198, 1348, 1499, 1999]
[1348, 1499, 1999]
```

TRAP

`sorted(x)` gives you a new list and leaves x alone. `x.sort()` changes x and gives back None. So `x = x.sort()` destroys your list.

A TABLE AS A LIST OF LISTS — PYTHON 3.12

```
orders = [
    [101, "Aarav", 1198],          # a list of lists:
    [102, "Diya", 796],            # each inner list is one row
    [103, "Aarav", 1499],
]
for order_id, name, amount in orders:     # unpack each row
    print(order_id, name, amount)
print(orders[2][1])                        # row 2, column 1
```

OUTPUT

```
101 Aarav 1198
102 Diya 796
103 Aarav 1499
Aarav
```

A list of rows is how a small table looks in plain Python. Later you will use a list of dicts (Lesson 12), which names each column instead of relying on positions.

Method	Does	Example
`append(x)`	Add one item to the end	`xs.append(5)`
`extend(other)`	Add every item of another list	`xs.extend([6, 7])`
`insert(i, x)`	Add at a position	`xs.insert(0, 1)`
`remove(x) / pop(i)`	Delete by value / by position	`xs.remove(5); xs.pop(0)`
`index(x) / count(x)`	Where is x? How many times?	`xs.index(7); xs.count(7)`
`sort() / reverse()`	Reorder in place	`xs.sort(reverse=True)`
`copy() / clear()`	New copy / empty it	`ys = xs.copy()`

TRY IT YOURSELF BEFORE MOVING ON

1. Add three cities to an empty list and print them sorted. **2.** Print the second-largest amount in a list. **3.** Remove every "cancelled" from a list of statuses by building a new list.

PART 1 · PYTHON FROM ZERO · LESSON 11

TUPLES AND SETS

L11

A tuple is a list that cannot change. A set is a bag of unique items with no order. Each solves a problem a list handles badly.

TUPLES: FIXED RECORDS — PYTHON 3.12

```
point = (12.97, 77.59)            # round brackets: a tuple
print(point[0], len(point))

lat, lon = point                  # unpacking
print(lat, lon)

def min_max(values):
    return min(values), max(values)   # returns a tuple
low, high = min_max([747, 3498, 1048])
print(low, high)
```

OUTPUT

```
12.97 2
12.97 77.59
747 3498
```

REMEMBER

Use a tuple for a fixed group of values that belong together: a coordinate, a (date, amount) pair, a database row. Trying point[0] = 5 raises a TypeError, which protects the record from accidental changes.

SETS: UNIQUE ITEMS AND FAST MEMBERSHIP — PYTHON 3.12

```
statuses = ["delivered", "cancelled", "delivered",
            "returned", "delivered"]
unique = set(statuses)
print(unique, len(unique))

ids = {101, 102, 103}
ids.add(104)
ids.add(101)                       # already there: nothing happens
print(ids, 102 in ids)

yesterday = {101, 102, 103, 104}
today = {103, 104, 105}
print("new today:", today - yesterday)
print("in both:  ", today & yesterday)
print("all:      ", yesterday | today)
```

OUTPUT

```
{'delivered', 'returned', 'cancelled'} 3
{104, 101, 102, 103} True
new today: {105}
in both:   {104, 103}
all:       {101, 102, 103, 104, 105}
```

APPROACH

Turning a list into a set is the quickest way to remove duplicates. Set maths (- & |) answers "what is new", "what is common" and "what is everything" in one line each. A set prints in any order, because it has none.

	List	Tuple	Set
Brackets	`[1, 2]`	`(1, 2)`	`{1, 2}`
Ordered, has positions	Yes	Yes	No
Can change	Yes	No	Yes (add/remove)
Duplicates	Allowed	Allowed	Removed automatically
Best for	A sequence you edit	A fixed record	Uniqueness and "is x in here?"

TRAP

An empty set is `set()`, not `{}`. The curly brackets on their own make an empty dictionary (next lesson).

TRY IT YOURSELF BEFORE MOVING ON

1. Unpack ("2025-04-20", 2397) into two variables and print them. **2.** Customers who ordered in January but not February: use set difference. **3.** How many unique cities are in ["Pune", "Delhi", "Pune", "Mumbai"]?

PART 1 · PYTHON FROM ZERO · LESSON 12

DICTIONARIES: KEY AND VALUE

L12

A dictionary stores pairs: a key you look up, and the value it points to. It is the most useful structure in data engineering.

CREATE, READ, CHANGE, ADD, REMOVE — PYTHON 3.12

```
customer = {
    "id": 4,
    "name": "Meera",
    "city": "Bengaluru",
}
print(customer["name"])                 # look up by key
customer["city"] = "Pune"               # change a value
customer["tier"] = "gold"               # add a new key
del customer["id"]                      # remove a key
print(customer)
print(customer.get("phone"), customer.get("phone", "not given"))
print("tier" in customer, len(customer))
```

OUTPUT

```
Meera
{'name': 'Meera', 'city': 'Pune', 'tier': 'gol
  d'}
None not given
True 3
```

TRAP

`customer['phone']` crashes with a KeyError when the key is missing. `customer.get('phone')` returns None instead, or the default you give it. Use get for anything that might be absent.

LOOPING OVER A DICTIONARY — PYTHON 3.12

```
prices = {"Mouse": 799, "Hub": 1499, "Pens": 149}

for product in prices:                  # loops over keys
    print(product, end=" ")
print()
for product, price in prices.items():   # key and value together
    print(f"{product:<6} {price:>5}")
print(list(prices.keys()), list(prices.values()))
```

OUTPUT

```
Mouse Hub Pens
Mouse    799
Hub     1499
Pens     149
['Mouse', 'Hub', 'Pens'] [799, 1499, 149]
```

THE TWO JOBS DICTS DO ALL DAY: COUNT AND GROUP — PYTHON 3.12

```
statuses = ["delivered", "cancelled", "delivered",
            "returned", "delivered"]

counts = {}
for s in statuses:
    if s in counts:
        counts[s] += 1
    else:
        counts[s] = 1
print(counts)

orders = [                          # a list of dicts: the
    {"id": 101, "city": "Pune", "amount": 1198},   # shape of
    {"id": 102, "city": "Delhi", "amount": 796},   # most real
    {"id": 103, "city": "Pune", "amount": 1499},   # data
]
by_city = {}
for o in orders:
    by_city[o["city"]] = by_city.get(o["city"], 0) + o["amount"]
print(by_city)
```

OUTPUT

```
{'delivered': 3, 'cancelled': 1, 'returned': 1
  }
{'Pune': 2697, 'Delhi': 796}
```

APPROACH

Counting and grouping are the same idea: the key is "what am I grouping by", the value is "what am I keeping". A list of dicts is exactly what you get from JSON files and APIs, and from csv.DictReader (page 33).

TRY IT YOURSELF BEFORE MOVING ON

1. Make a dict of product to price and print the cheapest product. **2.** Count each letter in "mississippi" with a dict. **3.** Group the words ["apple", "avocado", "banana", "blueberry"] by first letter.

PART 1 · PYTHON FROM ZERO · LESSON 13

WHICH DATA STRUCTURE WHEN?

L13

Choosing the right container is half of every coding question. Here are all four side by side, with the question to ask yourself.

	list	tuple	set	dict
Looks like	`[1, 2, 2]`	`(1, 2)`	`{1, 2}`	`{"a": 1}`
Ordered	Yes	Yes	No	Yes (insertion order)
Changeable	Yes	No	Yes	Yes
Duplicates	Yes	Yes	No	Keys unique, values can repeat
Find an item	Slow: checks one by one	Slow	Fast	Fast, by key
Ask yourself	"Do I need order or positions?"	"Is it a fixed record?"	"Do I only care if it exists?"	"Do I look things up by a name or id?"
Real example	Rows of a file, daily totals in date order	A (date, amount) pair	IDs already loaded	customer_id to customer record

WHY THE CHOICE MATTERS: FINDING ONE ID IN A MILLION — PYTHON 3.12

```python
import time

ids_list = list(range(1_000_000))
ids_set = set(ids_list)

start = time.perf_counter()
print(999_999 in ids_list)              # checks a million items
list_ms = (time.perf_counter() - start) * 1000

start = time.perf_counter()
print(999_999 in ids_set)               # jumps straight to it
set_ms = (time.perf_counter() - start) * 1000
print(f"list {list_ms:.2f} ms vs set {set_ms:.4f} ms")
```

OUTPUT

```
True
True
list 10.66 ms vs set 0.0049 ms
```

REMEMBER

Both give the same answer. The list checks items one by one; the set jumps straight to the answer. Do this check 1 lakh times in a loop and the difference becomes minutes against milliseconds. Page 41 explains why with Big-O.

CONVERTING BETWEEN THEM — PYTHON 3.12

```python
names = ["Aarav", "Diya", "Aarav", "Kabir"]
print(list(set(names)))                 # dedupe (order lost)
print(list(dict.fromkeys(names)))       # dedupe, order kept

ids = [101, 102, 103]
amounts = [1198, 796, 1499]
print(dict(zip(ids, amounts)))          # two lists -> one dict
print(sorted({"b": 2, "a": 1}.items())) # dict -> sorted pairs
print(tuple(ids), set("banana"))
```

OUTPUT

```
['Diya', 'Kabir', 'Aarav']
['Aarav', 'Diya', 'Kabir']
{101: 1198, 102: 796, 103: 1499}
[('a', 1), ('b', 2)]
(101, 102, 103) {'a', 'n', 'b'}
```

`zip` pairs items from two lists by position. `dict.fromkeys` keeps the first time each name appears, so it removes duplicates without shuffling the order.

Quick check: which would you use?

Task	Pick	Because
Store the 15 orders in the order they happened	list	Order matters
Remember which customer ids you have already loaded	set	Only "seen or not", fast
Look up a product's price by its name	dict	Lookup by key
Return a (min, max) pair from a function	tuple	Fixed record
Count orders per status	dict	status -> count

PART 1 · PYTHON FROM ZERO · LESSON 14

FUNCTIONS: REUSABLE STEPS

L14

A function is a named recipe. Write the steps once, then run them on any input by calling the name.

DEFINE ONCE, CALL MANY TIMES — PYTHON 3.12

```
def add_gst(amount, rate=0.18):
    """Return the amount with GST added."""
    total = amount * (1 + rate)
    return round(total, 2)

print(add_gst(1000))              # uses the default rate
print(add_gst(1000, 0.05))        # overrides it
print(add_gst(rate=0.12, amount=500))   # by name, any order
```

OUTPUT

```
1180.0
1050.0
560.0
```

REMEMBER

`def` starts the definition. Inside the brackets are **parameters**: the inputs. `return` sends the answer back to whoever called it. The text in triple quotes is a docstring: a one-line description others can read.

PRINT VS RETURN — PYTHON 3.12

```
def total_print(a, b):
    print(a + b)      # shows it, returns nothing

def total_return(a, b):
    return a + b            # gives the answer back

x = total_print(2, 3)
y = total_return(2, 3)
print("x is", x, "| y is", y)
```

OUTPUT

```
5
x is None | y is 5
```

total_print showed 5 but returned nothing, so x is None. If you need to use the answer later, return it.

VARIABLES INSIDE STAY INSIDE — PYTHON 3.12

```
def double(n):
    result = n * 2    # only exists in here
    return result

print(double(21))
print(result)               # not visible outside
```

OUTPUT

```
42
NameError: name 'result' is not defined
```

Each call gets its own fresh set of local variables, which disappear when the function returns.

SMALL FUNCTIONS THAT BUILD ON EACH OTHER — PYTHON 3.12

```
def is_delivered(status):
    return status.strip().lower() == "delivered"

def delivered_total(rows):
    total = 0
    for status, amount in rows:
        if is_delivered(status):    # a function in a function
            total += amount
    return total

rows = [("Delivered", 1198), ("cancelled", 1999), (" delivered ", 796)]
print(delivered_total(rows))
```

OUTPUT

```
1994
```

APPROACH

Good functions do one job and have a name that says what it is. Then bigger functions read like sentences. Interviewers notice this more than clever one-liners.

Word	Meaning
Parameter	The name in the definition: amount in def add_gst(amount)
Argument	The value you pass when calling: 1000 in add_gst(1000)
Return value	What the call gives back. No return statement means None
Default value	Used when the caller leaves that argument out: rate=0.18
Docstring	The text in triple quotes right under def; shown by help(add_gst)

TRY IT YOURSELF BEFORE MOVING ON

1. Write is_even(n) that returns True or False. **2.** Write to_rupees(1198) that returns "Rs 1,198.00". **3.** Give a function a default currency of "INR" and call it with and without it.

PART 1 · PYTHON FROM ZERO · LESSON 15

FUNCTIONS PART 2: A MINI PIPELINE

L15

Functions are values: you can put them in a list, pass them to other functions and chain them. That is how data pipelines are built.

KEY FUNCTIONS AND LAMBDA — PYTHON 3.12

```
products = [("Mouse", 799), ("Hub", 1499),
            ("Pens", 149), ("Lamp", 1199)]

def by_price(item):
    return item[1]

print(sorted(products, key=by_price))
print(sorted(products, key=lambda item: item[1], reverse=True))
print(max(products, key=lambda item: item[1]))
```

OUTPUT

```
[('Pens', 149), ('Mouse', 799), ('Lamp', 1199
  ), ('Hub', 1499)]
[('Hub', 1499), ('Lamp', 1199), ('Mouse', 799
  ), ('Pens', 149)]
('Hub', 1499)
```

REMEMBER

`key=` tells sorted, max and min what to compare. A `lambda` is a tiny function written inline: `lambda item: item[1]` means "given an item, return its price". Use it only for one short expression.

PARSE, FILTER, SUMMARISE: YOUR FIRST PIPELINE — PYTHON 3.12

```
raw_rows = ["101, Delivered ,1198", "102,cancelled,1999",
            "bad row", "103,  delivered,1499"]

def parse(line):
    parts = line.split(",")
    if len(parts) != 3:
        return None                          # cannot use this row
    order_id, status, amount = parts
    return {"id": int(order_id), "status": status.strip().lower(),
            "amount": float(amount)}

def keep_delivered(rows):
    return [r for r in rows if r and r["status"] == "delivered"]

def summarise(rows):
    revenue = sum(r["amount"] for r in rows)
    return {"orders": len(rows), "revenue": revenue}

rows = [parse(line) for line in raw_rows]          # step 1
clean = keep_delivered(rows)                         # step 2
print(summarise(clean))                              # step 3
```

OUTPUT

```
{'orders': 2, 'revenue': 2697.0}
```

DATA ENGINEERING NOTE

Every data pipeline has this shape: **read and parse**, **clean and filter**, **aggregate and output**. Real ones read files and write to a warehouse, but the functions look like these. The bad row became None and was filtered out instead of crashing the job.

ANY NUMBER OF ARGUMENTS — PYTHON 3.12

```
def report(*amounts, currency="Rs"):
    total = sum(amounts)
    print(f"{len(amounts)} orders, total {currency} {total:,}")

report(1198, 796, 1499)
report(1999, currency="INR")
```

OUTPUT

```
3 orders, total Rs 3,493
1 orders, total INR 1,999
```

`*amounts` collects however many values you pass into a tuple. Page 31 covers `*args` and `**kwargs` properly.

TRY IT YOURSELF BEFORE MOVING ON

1. Sort the employees in employees.csv by salary using a lambda. **2.** Add a step to the mini pipeline that drops rows with a duplicate id. **3.** Write average(*numbers) that works for any count of numbers.

PART 1 · PYTHON FROM ZERO · LESSON 16

MODULES, PACKAGES AND PIP

L16

Nobody writes everything from scratch. A module is a file of ready-made code you import; pip installs other people's modules.

THE STANDARD LIBRARY: ALREADY INSTALLED — PYTHON 3.12

```
import math
import random
from datetime import date, timedelta
import statistics as st              # a shorter nickname

print(math.sqrt(144), math.ceil(4.2), math.floor(4.8))
random.seed(1)                       # same numbers every run
print(random.randint(1, 6), random.choice(["Pune", "Delhi"]))
print(date(2025, 4, 1) + timedelta(days=30))
print(st.mean([1198, 796, 1499]), st.median([1198, 796, 1499]))
```

OUTPUT

```
12.0 5 4
2 Pune
2025-05-01
1164.3333333333333 1198
```

REMEMBER

`import math` gives you the whole module, used as `math.sqrt`. `from datetime import date` pulls in just one name. `as st` gives a nickname.

YOUR OWN MODULE — PYTHON 3.12

```
import pathlib

pathlib.Path("helpers.py").write_text(
    "def clean(s):\n"
    "    return s.strip().lower()\n\n"
    "STATUSES = {'delivered', 'cancelled', 'returned'}\n")

import helpers                       # your file is a module
print(helpers.clean("  DELIVERED "), helpers.STATUSES)
```

OUTPUT

```
delivered {'delivered', 'returned', 'cancelle
  d'}
```

Any .py file in the same folder can be imported by its name. This snippet writes helpers.py first, then imports it. Real projects split code into modules like extract.py, transform.py and load.py.

PIP IN THE TERMINAL — TERMINAL · NOT RUN

```
# inside your activated .venv (Lesson 1)
pip install pandas requests      # get packages
pip list                         # what is installed
pip freeze > requirements.txt    # save versions
pip install -r requirements.txt  # on another machine
```

Module	Use it for	Pages
`csv, json, pathlib`	Reading and writing files	33
`datetime, zoneinfo`	Dates and time zones	27
`collections`	Counter, defaultdict, deque	29, 36
`itertools, functools`	Iteration and function tools	36
`re`	Patterns in text, log parsing	64
`pandas`	Tables (pip install)	69, 70

IF __NAME__ == "__MAIN__": RUN ONLY WHEN EXECUTED — PYTHON 3.12

```
import pathlib, subprocess, sys

pathlib.Path("tool.py").write_text(
    "def main():\n"
    "    print('running the tool')\n\n"
    "if __name__ == '__main__':\n"
    "    main()\n")

import tool                          # importing: main() does not run
print("imported; its __name__ is", tool.__name__)
run = subprocess.run([sys.executable, "tool.py"],
                     capture_output=True, text=True)
print("run directly:", run.stdout.strip())
```

OUTPUT

```
imported; its __name__ is tool
run directly: running the tool
```

A file can be both a module you import and a program you run. Code under the __main__ check runs only when you execute the file, not when another file imports it. Every pipeline script uses this.

TRY IT YOURSELF BEFORE MOVING ON

1. Print math.pi rounded to 3 decimals. **2.** Find the weekday name of 2025-04-01 with datetime (strftime("%A")). **3.** Create utils.py with one function, then import and call it.

PART 1 · PYTHON FROM ZERO · LESSON 17

READING ERRORS WITHOUT PANIC

L17

Errors are Python telling you exactly what went wrong and where. Learning to read them is faster than any amount of guessing.

A REAL TRACEBACK — PYTHON 3.12

```
def average(values):
    return sum(values) / len(values)

orders = []
print(average(orders))
```

OUTPUT

```
ZeroDivisionError: division by zero
```

APPROACH

Read from the bottom. The last line is the error type and message: ZeroDivisionError, division by zero. The full message also lists the file and line number that failed. Here: the list was empty, so len() was 0.

Error	What it means	Typical cause	Fix
`SyntaxError`	Python cannot read the line	Missing colon, bracket or quote	Look at the line and the one above
`IndentationError`	Spacing is wrong	Mixed tabs and spaces, missing indent after :	Use 4 spaces everywhere
`NameError`	Unknown name	Typo, wrong capital, used before created	Check spelling and order
`TypeError`	Wrong kind of value	"5" + 5, calling a non-function	Convert types: int(), str()
`ValueError`	Right type, bad value	int("abc"), float("")	Validate or try/except
`IndexError`	Position does not exist	xs[10] on a 5-item list	Check len() first
`KeyError`	Key not in the dict	d["phone"] when there is none	Use d.get("phone")
`ZeroDivisionError`	Dividing by zero	Average of an empty list	Check for empty first
`FileNotFoundError`	No such file	Wrong name or folder	Check the path with pathlib

TRY / EXCEPT: HANDLE THE ERROR, KEEP GOING — PYTHON 3.12

```
rows = ["1198", "796", "abc", "", "1499"]

total, bad = 0, []
for r in rows:
    try:
        total += float(r)          # might fail
    except ValueError:
        bad.append(r)              # note it and keep going
print("total:", total, "| could not read:", bad)
```

OUTPUT

```
total: 3493.0 | could not read: ['abc', '']
```

TRAP

Catch the specific error you expect (here ValueError). A bare `except:` hides real bugs like typos. Page 32 goes deeper: else, finally and your own error types.

DEBUGGING: PRINT WHAT YOU HAVE, THEN ASSERT WHAT YOU EXPECT — PYTHON 3.12

```
orders = [{"id": 101, "amount": "1198"}, {"id": 102, "amount": 796}]

total = 0
for o in orders:
    kind = type(o["amount"]).__name__
    print("DEBUG", o["id"], repr(o["amount"]), kind)
    total += float(o["amount"])
print("total:", total)

assert total == 1994.0, f"expected 1994, got {total}"
print("check passed")
```

OUTPUT

```
DEBUG 101 '1198' str
DEBUG 102 796 int
total: 1994.0
check passed
```

APPROACH

When the answer is wrong, print each value and its type right before the line that misbehaves. Here one amount is text and one is a number. assert stops the program with your message if a condition is False: a free, quick test.

TRY IT YOURSELF BEFORE MOVING ON

1. Cause a KeyError on purpose and read the last line of the message. **2.** Return 0 instead of crashing when averaging an empty list. **3.** Fix "Total: " + 5 in two different ways.

PART 1 · PYTHON FROM ZERO · LESSON 18

FILES: READ AND WRITE

L18

Data engineering starts with files. Reading, writing and appending text files needs only open() and a with block.

READING A FILE — PYTHON 3.12

```
with open("orders.csv") as f:          # open for reading
    header = f.readline()               # first line
    first_row = f.readline()
print(repr(header))
print(first_row.strip().split(","))

with open("orders.csv") as f:
    lines = f.read().splitlines()       # the whole file
print(len(lines), "lines; last:", lines[-1])
```

OUTPUT

```
'order_id,customer_id,order_date,status,amount\
  n'
['101', '1', '2025-01-10', 'delivered', '1198.0
  0']
16 lines; last: 115,5,2025-04-28,delivered,1598
  .00
```

REMEMBER

`with open(...) as f:` opens the file and closes it automatically when the block ends, even if an error happens. Lines come in with a "\n" at the end; strip() removes it.

LOOP OVER A FILE LINE BY LINE — PYTHON 3.12

```
with open("orders.csv") as f:
    next(f)                             # skip the header
    delivered = 0
    for line in f:                      # one line at a time
        cols = line.strip().split(",")
        status, amount = cols[3], cols[4]
        if status == "delivered":
            delivered += float(amount)
print(f"delivered revenue: {delivered:,.2f}")
```

OUTPUT

```
delivered revenue: 18,972.00
```

Looping over f reads one line at a time, so this works on a file of any size. split(",") is fine for simple files; real CSVs with quoted commas need the csv module (page 33).

WRITING AND APPENDING — PYTHON 3.12

```
with open("report.txt", "w") as f:   # write (replace)
    f.write("date,revenue\n")
    f.write("2025-04-20,2397\n")

with open("report.txt", "a") as f:   # append
    f.write("2025-04-28,1598\n")

print(open("report.txt").read())
```

OUTPUT

```
date,revenue
2025-04-20,2397
2025-04-28,1598
```

TRAP

"w" empties the file first, every time. Opening yesterday's report with "w" by mistake deletes it. Use "a" to add to the end, and "x" to create only if the file does not exist yet.

Mode	Does	If the file exists	If it does not
"r"	Read (the default)	Reads it	FileNotFoundError
"w"	Write	Empties it first	Creates it
"a"	Append	Adds to the end	Creates it
"x"	Create	FileExistsError	Creates it

PATHS AND FOLDERS WITH PATHLIB — PYTHON 3.12

```
from pathlib import Path

out_dir = Path("reports") / "2025-04"          # / joins path parts
out_dir.mkdir(parents=True, exist_ok=True)      # make folders
(out_dir / "summary.txt").write_text("orders,15\n")

print(Path("orders.csv").exists(), Path("missing.csv").exists())
print(sorted(p.name for p in Path(".").glob("*.json*")))
print([str(p) for p in Path("reports").rglob("*.txt")])
```

OUTPUT

```
True False
['customers.json', 'events.jsonl']
['reports/2025-04/summary.txt']
```

pathlib builds paths that work on Windows, Mac and Linux, so you never glue folder names with "/" or "\\" by hand. glob finds files by pattern; rglob searches sub-folders too.

TRY IT YOURSELF BEFORE MOVING ON

1. Count the lines in orders.csv without the header. 2. Write every cancelled order into cancelled.txt. 3. Append a line with today's total to report.txt without deleting what is there.

PART 1 · PYTHON FROM ZERO · LESSON 19

MINI PROJECT: MONTHLY SALES REPORT

L19

Everything from Lessons 1 to 18 in one small program: read a file, clean, decide, group, calculate and print a report.

DAILY_SALES_REPORT.PY — PYTHON 3.12

```
# daily_sales_report.py: only the basics from Lessons 1 to 18

def read_orders(path):
    """Read orders.csv into a list of dicts."""
    rows = []
    with open(path) as f:
        header = f.readline().strip().split(",")
        for line in f:
            values = line.strip().split(",")
            rows.append(dict(zip(header, values)))
    return rows

def month_of(day):
    return day[:7]                                  # "2025-04-20" -> "2025-04"

def build_report(rows):
    report = {}
    for r in rows:
        if r["status"] != "delivered":
            continue
        month = month_of(r["order_date"])
        if month not in report:
            report[month] = {"orders": 0, "revenue": 0.0, "best": 0.0}
        stats = report[month]
        amount = float(r["amount"])
        stats["orders"] += 1
        stats["revenue"] += amount
        if amount > stats["best"]:
            stats["best"] = amount
    return report

def print_report(report):
    print(f"{'month':<9}{'orders':>7}{'revenue':>11}{'average':>10}{'best':>9}")
    print("-" * 46)
    grand = 0.0
    for month in sorted(report):
        s = report[month]
        avg = s["revenue"] / s["orders"]
        grand += s["revenue"]
        print(f"{month:<9}{s['orders']:>7}{s['revenue']:>11,.2f}{avg:>10,.2f}{s['best']:>9,.0f}")
    print("-" * 46)
    print(f"{'total':<9}{'':>7}{grand:>11,.2f}")

rows = read_orders("orders.csv")
print(len(rows), "orders read")
print_report(build_report(rows))
```

OUTPUT

```
15 orders read
month     orders    revenue   average     best
----------------------------------------------
2025-01        2   1,994.00    997.00    1,198
2025-02        2   2,847.00  1,423.50    1,499
2025-03        4   7,291.00  1,822.75    3,498
2025-04        4   6,840.00  1,710.00    2,397
----------------------------------------------
total             18,972.00
```

Line of thinking	Where it is in the code	Lesson
Read the file into rows, with column names	read_orders: open, readline, split, zip, dict	12, 13, 18
Skip what does not count	if r["status"] != "delivered": continue	7, 9
Group by month	a dict keyed by "2025-04", created the first time a month appears	12
Count, total, track the best	the three loop patterns	8
Print it, keep it readable	f-strings with widths; four small, well-named functions	4, 14

Your turn: report per city instead of per month (load customers.json into a lookup dict), then add a cancellation rate. Do both without looking back and you are ready for Part 2.

PART 1 · PYTHON FROM ZERO · PRACTICE

PRACTICE SET 0: FIRST STEPS

P0

Beginner questions that still come up as warm-ups in interviews. Try each one in your editor before reading the answer.

Q1. Print the numbers 1 to 15, but "Fizz" for multiples of 3, "Buzz" for multiples of 5 and "FizzBuzz" for both. DA DE EASY

Approach: Check the "both" case first, because 15 is also a multiple of 3. Use % for "is a multiple of".

```
out = []
for n in range(1, 16):
    if n % 15 == 0:
        out.append("FizzBuzz")
    elif n % 3 == 0:
        out.append("Fizz")
    elif n % 5 == 0:
        out.append("Buzz")
    else:
        out.append(str(n))
print(" ".join(out))
```

OUTPUT

```
1 2 Fizz 4 Buzz Fizz 7 8 Fizz Buzz 11 Fizz 13 14
  FizzBuzz
```

Q2. Find the largest and smallest amount without using max() or min(). DA DE EASY

Approach: Track the best so far (Lesson 8). Start from the first item, not from 0, or negative numbers break it.

```
amounts = [1198, 796, 1499, 1999, 747, 3498]
low = high = amounts[0]
for a in amounts[1:]:
    if a > high:
        high = a
    if a < low:
        low = a
print(low, high)
```

OUTPUT

```
747 3498
```

Q3. Count how many times each word appears in a sentence, ignoring case. DA DE EASY

Approach: Lowercase, split into words, then count with a dict (Lesson 12).

```
text = "Data pipelines move data and data pipelines fail"
counts = {}
for word in text.lower().split():
    counts[word] = counts.get(word, 0) + 1
print(counts)
```

OUTPUT

```
{'data': 3, 'pipelines': 2, 'move': 1, 'and': 1,
  'fail': 1}
```

Q4. Check whether a string reads the same backwards, ignoring spaces and case. DA DE EASY

Approach: Keep only letters, lowercase them, compare with the reversed slice [::-1].

```
def is_palindrome(text):
    letters = "".join(ch.lower() for ch in text if ch.isalpha())
    return letters == letters[::-1]

print(is_palindrome("Never odd or even"), is_palindrome("Python"))
```

OUTPUT

```
True False
```

Q5. Given a list of order statuses, return each status with the share of orders it makes up, as a percentage. DA EASY

Approach: Count first, then divide each count by the total. Format with f-strings (Lesson 4).

```
statuses = []
with open("orders.csv") as f:
    next(f)
    for line in f:
        statuses.append(line.strip().split(",")[3])
counts = {}
for s in statuses:
    counts[s] = counts.get(s, 0) + 1
for s, n in counts.items():
    print(f"{s:<10} {n:>2}  {n / len(statuses):.1%}")
```

OUTPUT

```
delivered  12  80.0%
cancelled   2  13.3%
returned    1  6.7%
```

PART 2 · PYTHON FOUNDATIONS · BEFORE YOU CODE

THE 6-STEP APPROACH

Use this on every coding question in the book. DE interviewers grade how you get there almost as much as the code.

01 Pin the I/O
Say the exact input and output. "A list of dicts in, a dict of totals out."

02 Edge cases
Empty input, None, duplicates, bad types, ties. Ask which ones matter.

03 Brute force
Say the simple version and its cost first. Then improve it.

04 Pick the structure
dict for lookups, set for membership, deque for windows, heap for top-K.

05 Small functions
One function per idea. Run each on the sample before moving on.

06 Test and cost
Run your edge cases, then state time and memory in Big-O.

Watch it work on one question

Q6. From orders.csv, return each customer's total delivered spend, highest first. Ties broken by customer_id. DE EASY

Step	What you say in the interview
01 I/O	In: rows from orders.csv. Out: a list of `(customer_id, total)` tuples, highest total first.
02 Edges	Amounts arrive as strings. Some orders are cancelled or returned. A customer may have no delivered orders and should not appear.
03 Brute	For each customer, scan all orders: O(customers x orders). Fine for 15 rows, not for 15 million.
04 Structure	One pass with a `defaultdict(Decimal)` keyed by customer: O(n). Decimal because this is money (page 27).
05 Functions	`load_rows` reads and converts, `delivered_totals` aggregates, `sorted` ranks.
06 Test	Run on the sample, check one total by hand (Aarav: 1198 + 1499 + 2397 = 5094), then say: O(n) to sum, O(k log k) to sort k customers.

SOLUTION — PYTHON 3.12

```python
import csv
from collections import defaultdict
from decimal import Decimal

def load_rows(path):
    with open(path, newline="") as f:
        for row in csv.DictReader(f):
            row["customer_id"] = int(row["customer_id"])
            row["amount"] = Decimal(row["amount"])
            yield row

def delivered_totals(rows):
    totals = defaultdict(Decimal)
    for r in rows:
        if r["status"] == "delivered":
            totals[r["customer_id"]] += r["amount"]
    return totals

totals = delivered_totals(load_rows("orders.csv"))
ranked = sorted(totals.items(), key=lambda kv: (-kv[1], kv[0]))
for cid, total in ranked:
    print(cid, total)
```

OUTPUT

```
5 5096.00
1 5094.00
2 4143.00
6 1998.00
4 1048.00
7 846.00
3 747.00
```

INTERVIEW TIP
Customer 8 (Zoya) is missing from the output, and that is correct: she has no orders. Say so before the interviewer asks. It shows you checked the output against the input, not just that the code ran.

APPROACH
When you get stuck: go back to step 1 and write one line of expected output by hand. Most wrong answers come from a fuzzy idea of the output, not from syntax.

PART 2 · PYTHON FOUNDATIONS · CHAPTER 1

NAMES, OBJECTS AND MUTABILITY

01

A Python variable is a name tag on an object, not a box that holds a value. Two names can point at the same object, and that causes most beginner bugs.

TWO NAMES, ONE OBJECT — PYTHON 3.12

```
a = [1, 2, 3]
b = a            # a second name, same list
b.append(4)
print(a, a is b)

c = a.copy()     # a new list
c.append(5)
print(a, c, a is c)

x, y = [1, 2], [1, 2]
print(x == y, x is y)   # equal value, different objects
```

OUTPUT

```
[1, 2, 3, 4] True
[1, 2, 3, 4] [1, 2, 3, 4, 5] False
True False
```

`==` compares values. `is` compares identity: same object in memory. Use `is` only for `None`, `True` and `False`.

Immutable (cannot change in place)	Mutable (can change in place)
`int float bool str tuple frozenset bytes None`	`list dict set bytearray`, most objects you create
"Changing" one makes a new object: s = s + "x"	Changes are seen by every name pointing at it

The mutable default argument trap

THE BUG AND THE FIX — PYTHON 3.12

```
def add_row(row, batch=[]):      # default made ONCE
    batch.append(row)
    return batch

print(add_row("r1"))
print(add_row("r2"))             # r1 is still there

def add_row_fixed(row, batch=None):
    batch = [] if batch is None else batch
    batch.append(row)
    return batch

print(add_row_fixed("r1"), add_row_fixed("r2"))
```

OUTPUT

```
['r1']
['r1', 'r2']
['r1'] ['r2']
```

TRAP

Default values are evaluated once, when def runs. Every call that omits batch shares that one list. In a pipeline this means rows from yesterday's run leaking into today's batch.

Shallow vs deep copy

COPIES OF NESTED DATA — PYTHON 3.12

```
import copy

rows = [{"id": 1, "tags": ["new"]}]
shallow = rows.copy()          # new outer list only
deep = copy.deepcopy(rows)     # new everything

shallow[0]["tags"].append("vip")
print(rows[0]["tags"])         # changed through shallow
print(deep[0]["tags"])         # untouched
```

OUTPUT

```
['new', 'vip']
['new']
```

DATA ENGINEERING NOTE

Config dicts and row templates are nested. If one pipeline step edits a shallow copy, the change leaks into every other step that shares the inner objects. Deep copy, or better, build new dicts: `{**row, 'tags': [*row['tags'], 'vip']}`.

INTERVIEW TIP

"Is Python pass-by-value or pass-by-reference?" Neither. It passes references to objects by value: a function gets a new name for the same object. Mutate the object and the caller sees it. Rebind the name inside the function and the caller does not.

PART 2 · PYTHON FOUNDATIONS · CHAPTER 2

NUMBERS, STRINGS AND DATES

02

Money, text encodings and time zones break more pipelines than any algorithm. Each one has a single rule to remember.

FLOATS, DECIMAL, ROUNDING — PYTHON 3.12

```
from decimal import Decimal, ROUND_HALF_UP

print(0.1 + 0.2, 0.1 + 0.2 == 0.3)
print(Decimal("0.1") + Decimal("0.2"))
print(Decimal(0.1))             # from a float: too late

print(round(2.5), round(3.5))   # banker's rounding
price = Decimal("2.675")
print(price.quantize(Decimal("0.01"), ROUND_HALF_UP))

print(7 // 2, -7 // 2, int(-3.5))  # floor vs trunc
```

OUTPUT

```
0.30000000000000004 False
0.3
0.1000000000000000055511151231257827021181583404541015625
2 4
2.68
3 -4 -3
```

TRAP

Money in float drifts. Build Decimal from a **string**, never from a float, and round with an explicit mode. `round()` rounds halves to the nearest even number, so 2.5 becomes 2. `//` floors toward minus infinity: -7 // 2 is -4, not -3.

STRINGS — PYTHON 3.12

```
status = "  DELIVERED \n"
print(repr(status.strip().lower()))

amount = 1198.5
print(f"{amount:,.2f} | {amount:>10.1f} | {0.256:.1%}")

parts = "101,1,2025-01-10,delivered,1198.00".split(",")
print(parts[3], "|".join(parts[:2]))

word = "café"
print(word, len(word), len(word.encode("utf-8")))
```

OUTPUT

```
'delivered'
1,198.50 |     1198.5 | 25.6%
delivered 101|1
café 4 5
```

REMEMBER

`str` is text, `bytes` is what sits on disk. The é in café is one character but two bytes in UTF-8; the rupee sign is three. Always pass `encoding='utf-8'` to `open()`: the default depends on the machine.

Building a long string with `+=` in a loop copies it every time. Collect parts in a list and `''.join()` once.

DATES AND TIME ZONES — PYTHON 3.12

```
from datetime import date, datetime, timedelta, timezone
from zoneinfo import ZoneInfo

d = datetime.strptime("2025-04-01 10:00", "%Y-%m-%d %H:%M")
print(d.isoformat(), d.tzinfo)                  # naive

ist = d.replace(tzinfo=ZoneInfo("Asia/Kolkata"))
print(ist.astimezone(timezone.utc).isoformat())

print(date(2025, 3, 31) + timedelta(days=1))
print((date(2025, 4, 28) - date(2025, 3, 12)).days)
print(d.strftime("%Y-%m"), d.weekday())         # Monday = 0
```

OUTPUT

```
2025-04-01T10:00:00 None
2025-04-01T04:30:00+00:00
2025-04-01
47
2025-04 1
```

DATA ENGINEERING NOTE

A datetime with no tzinfo is "naive": Python has no idea which zone it is in. Store and compare timestamps in UTC, convert to IST only for display. Mixing naive and aware datetimes raises a TypeError, which is better than a silent 5.5 hour shift.

PART 2 · PYTHON FOUNDATIONS · CHAPTER 3

LISTS, TUPLES, SETS AND BIG-O

03

Picking the right container is most of the answer in a coding round. The cost table below is worth memorising.

SLICING AND SORTING — PYTHON 3.12

```
amounts = [1198, 796, 1499, 1999, 1348]
print(amounts[0], amounts[-1], amounts[1:3], amounts[::-1][:2])

print(sorted(amounts, reverse=True)[:3])  # new list
amounts.sort()                  # in place, returns None
print(amounts)

orders = [("Delhi", 747), ("Pune", 3498), ("Delhi", 846),
          ("Pune", 1598), ("Mumbai", 796)]
by_city_then_amt = sorted(orders, key=lambda o: (o[0], -o[1]))
print(by_city_then_amt)
```

OUTPUT

```
1198 1348 [796, 1499] [1348, 1999]
[1999, 1499, 1348]
[796, 1198, 1348, 1499, 1999]
[('Delhi', 846), ('Delhi', 747), ('Mumbai', 796),
  ('Pune', 3498), ('Pune', 1598)]
```

TRAP

`list.sort()` returns None. `x = x.sort()` wipes your list. Use `sorted(x)` when you need the result.

Python sorts are stable: rows with equal keys keep their input order, which is what makes multi-pass sorting work.

SETS DO RECONCILIATION IN ONE LINE — PYTHON 3.12

```
source = {101, 102, 103, 104, 105}
target = {101, 102, 104, 106}

print("missing in target:", source - target)
print("extra in target:  ", target - source)
print("in both:          ", source & target)
print("all ids:          ", sorted(source | target))

point = (12.97, 77.59)          # tuple: fixed record
lat, lon = point
print(lat, lon, {point: "Bengaluru"}[point])
```

OUTPUT

```
missing in target: {105, 103}
extra in target:   {106}
in both:           {104, 101, 102}
all ids:           [101, 102, 103, 104, 105, 106]
12.97 77.59 Bengaluru
```

DATA ENGINEERING NOTE

Set difference is the quickest load check there is: ids in the source but not the target were lost, ids in the target but not the source should not be there.

Tuples are immutable, so they can be dict keys and set members. Lists cannot.

The cost table

Operation	list	set / dict	Why it matters
`x in c`	O(n) scan	O(1) average	Checking 1 lakh ids against a list of 1 lakh is 10 billion comparisons. Make it a set.
Append / add	O(1) amortised	O(1) average	Both fine
Insert or delete at front	O(n)	n/a	Use `collections.deque`: O(1) at both ends
Get by index / key	O(1)	O(1) average	dict lookups are why one-pass solutions work
Sort	O(n log n)	n/a	Top 3 of a million? `heapq.nlargest` is O(n log k)
Memory	Lowest	Higher (hash table)	For huge data, stream it instead (page 35)

INTERVIEW TIP

Tuples make good composite keys: `totals[(customer_id, month)] += amount` groups by two columns with one dict, the same way GROUP BY customer_id, month does in SQL. Interviewers like hearing you connect the Python structure to the SQL you would write.

PART 2 · PYTHON FOUNDATIONS · CHAPTER 4

DICTS AND COLLECTIONS

04

The dict is the most important data structure in data engineering Python. Grouping, counting, joining and deduping all start with one.

LOOKUPS THAT DO NOT CRASH — PYTHON 3.12

```
import json
customers = json.load(open("customers.json"))
by_id = {c["customer_id"]: c for c in customers}

print(by_id[2]["name"])
print(by_id.get(99))            # None, no KeyError
print(by_id.get(99, {}).get("name", "unknown"))

sana = by_id[6]
print(sana["address"]["city"])  # key exists, is None
city = (sana.get("address") or {}).get("city") or "Unknown"
print(city)
```

OUTPUT

```
Diya
None
unknown
None
Unknown
```

TRAP

`.get(key, default)` only uses the default when the key is **missing**. Sana's city key exists with value None, so `.get('city', 'Unknown')` still returns None. Use `or` to cover both, or check explicitly.

GROUP, COUNT, MERGE — PYTHON 3.12

```
import csv
from collections import Counter, defaultdict

rows = list(csv.DictReader(open("order_items.csv")))

items_per_order = defaultdict(list)       # group
for r in rows:
    items_per_order[r["order_id"]].append(r["product_id"])
print(dict(list(items_per_order.items())[:3]))

counts = Counter(r["product_id"] for r in rows)   # count
print(counts.most_common(3))

merged = {"retries": 3, "env": "dev"} | {"env": "prod"}
print(merged)                             # right side wins
```

OUTPUT

```
{'101': ['1', '5'], '102': ['3', '6'], '103': ['2']
  }
[('1', 5), ('5', 4), ('3', 3)]
{'retries': 3, 'env': 'prod'}
```

REMEMBER

`defaultdict(list)` creates the empty list the first time a key appears, so grouping needs no if/else. `Counter` is a dict of counts with `most_common` built in. Dicts keep insertion order (guaranteed since Python 3.7).

DELETING WHILE ITERATING — PYTHON 3.12

```
prices = {"mouse": 799, "hub": 1499, "pens": 149}
for name, p in prices.items():
    if p < 500:
        del prices[name]
```

OUTPUT

```
RuntimeError: dictionary changed size during iteration
```

BUILD A NEW DICT INSTEAD — PYTHON 3.12

```
prices = {"mouse": 799, "hub": 1499, "pens": 149}
prices = {k: v for k, v in prices.items()
          if v >= 500}
print(prices)
```

OUTPUT

```
{'mouse': 799, 'hub': 1499}
```

INTERVIEW TIP

"Why are dict lookups O(1)?" Keys are hashed to a slot in an array, so finding a key does not scan the others. That is also why keys must be hashable (immutable): if a key could change, its hash and slot would go stale.

PART 2 · PYTHON FOUNDATIONS · CHAPTER 5

LOOPS AND COMPREHENSIONS

05

A comprehension is a loop that builds a collection in one expression. Use it when it reads like a sentence, and a plain loop when it does not.

LIST, SET AND DICT COMPREHENSIONS — PYTHON 3.12

```
import csv
rows = list(csv.DictReader(open("orders.csv")))

amounts = [float(r["amount"]) for r in rows
           if r["status"] == "delivered"]
print(len(amounts), max(amounts))

statuses = {r["status"] for r in rows}            # set
by_id = {int(r["order_id"]): r["status"] for r in rows}
print(sorted(statuses), by_id[104])

matrix = [[1, 2], [3, 4], [5]]
print([x for row in matrix for x in row])        # flatten
```

OUTPUT

```
12 3498.0
['cancelled', 'delivered', 'returned'] cancelled
[1, 2, 3, 4, 5]
```

REMEMBER

Read a nested comprehension left to right like the loops it replaces: for row in matrix, then for x in row.

If it needs two conditions and a nested loop, a regular for loop is clearer. Interviewers notice readability.

ZIP, ENUMERATE, UNPACKING, ANY, ALL — PYTHON 3.12

```
ids = [101, 102, 103]
names = ["Aarav", "Diya"]
print(list(zip(ids, names)))            # stops at shortest

for i, name in enumerate(names, start=1):
    print(i, name)

first, *rest = [101, 102, 103, 104]
print(first, rest)

print(any(n.startswith("D") for n in names),
      all(i > 100 for i in ids))

if (n := len(ids)) > 2:                 # walrus: assign + test
    print("big batch of", n)
```

OUTPUT

```
[(101, 'Aarav'), (102, 'Diya')]
1 Aarav
2 Diya
101 [102, 103, 104]
True True
big batch of 3
```

TRAP

`zip` silently drops the extra items when lengths differ: 103 vanished above. In Python 3.10+ write `zip(ids, names, strict=True)` to get a ValueError instead of quiet data loss.

FOR ... ELSE — PYTHON 3.12

```
targets = [1499, 9999]
amounts = [1198, 796, 1499]
for t in targets:
    for a in amounts:
        if a == t:
            print(t, "found")
            break
    else:                           # runs only if no break
        print(t, "not found")
```

OUTPUT

```
1499 found
9999 not found
```

The `else` of a loop runs when the loop finishes without `break`. Handy for "search and report if missing", and a common "what does this print?" question.

MAP/FILTER VS COMPREHENSIONS VS GENERATORS — PYTHON 3.12

```
amounts = ["1198.00", "796.00", "", "1499.00"]

total_a = sum(map(float, filter(None, amounts)))
total_b = sum(float(a) for a in amounts if a)   # same, clearer
print(total_a, total_b)

squares = [x * x for x in range(1_000)]          # builds a list
total_c = sum(x * x for x in range(1_000))       # no list
print(len(squares), total_c)
```

OUTPUT

```
3493.0 3493.0
1000 332833500
```

REMEMBER

`filter(None, xs)` drops falsy items such as empty strings. Most teams prefer the comprehension form because it reads in one direction.

Inside sum(), min() or any(), drop the square brackets: a generator expression feeds values one at a time and never builds the list. Page 35 goes deeper.

PART 2 · PYTHON FOUNDATIONS · CHAPTER 6

FUNCTIONS, SCOPE AND CLOSURES

06

Pipelines are built out of small functions. Know how arguments bind, where names are looked up, and what a closure keeps alive.

ARGUMENTS — PYTHON 3.12

```
def load(path, *columns, sep=",", **options):
    print(path, columns, sep, options)

load("orders.csv")
load("orders.csv", "order_id", "amount", sep="|", skip=1)

def area(*, width, height):        # keyword-only
    return width * height
print(area(width=3, height=4))

def summary(values):
    return min(values), max(values), sum(values) / len(values)
lo, hi, avg = summary([747, 3498, 1048])
print(lo, hi, round(avg, 1))
```

OUTPUT

```
orders.csv () , {}
orders.csv ('order_id', 'amount') | {'skip': 1}
12
747 3498 1764.3
```

`*columns` collects extra positional arguments into a tuple, `**options` collects extra keywords into a dict. A bare `*` forces the rest to be passed by name, which stops mixups like `area(4, 3)`.

SCOPE: UNBOUNDLOCALERROR — PYTHON 3.12

```
count = 0
def bump():
    count += 1      # assigning makes it local
bump()
```

OUTPUT

```
UnboundLocalError: cannot access local variable 'count'
  where it is not associated with a value
```

Assigning to a name anywhere inside a function makes it local for the whole function. Python looks names up in order: Local, Enclosing, Global, Built-in (LEGB).

CLOSURE WITH NONLOCAL — PYTHON 3.12

```
def make_counter():
    count = 0
    def bump():
        nonlocal count  # closure keeps it alive
        count += 1
        return count
    return bump

c = make_counter()
print(c(), c(), c())
```

OUTPUT

```
1 2 3
```

LATE BINDING, AND FUNCTIONS AS VALUES — PYTHON 3.12

```
fns = [lambda: i for i in range(3)]
print([f() for f in fns])              # late binding

fns = [lambda i=i: i for i in range(3)]
print([f() for f in fns])              # bind now

steps = [str.strip, str.lower, lambda s: s.replace(" ", "_")]
value = "  Order Date "
for step in steps:                     # functions are values
    value = step(value)
print(value)
```

OUTPUT

```
[2, 2, 2]
[0, 1, 2]
order_date
```

TRAP

Closures capture the variable, not its value at that moment. All three lambdas read i after the loop ended, so all return 2. A default argument freezes the value when the lambda is created.

A PURE, TYPED TRANSFORM FUNCTION — PYTHON 3.12

```
STATUS_MAP = {"delivered": "delivered", "dlvd": "delivered",
              "cancelled": "cancelled", "canceled": "cancelled"}

def normalize_status(raw: str | None) -> str:
    "Map messy source statuses to one vocabulary."
    if not raw:
        return "unknown"
    return STATUS_MAP.get(raw.strip().lower(), "unknown")

assert normalize_status(" DLVD ") == "delivered"
assert normalize_status(None) == "unknown"
print([normalize_status(s) for s in ["Canceled", "shipped", ""]])
print(normalize_status.__doc__)
```

OUTPUT

```
['cancelled', 'unknown', 'unknown']
Map messy source statuses to one vocabulary.
```

DATA ENGINEERING NOTE

Pure functions take input and return output with no hidden state and no I/O. They are easy to test (page 73) and safe to run in parallel. Keep reading and writing at the edges of the pipeline and make everything in between pure.

PART 2 · PYTHON FOUNDATIONS · CHAPTER 7

ERRORS AND CONTEXT MANAGERS

07

A pipeline that crashes on one bad row loses the other million. Catch narrowly, count what you skip, and always clean up.

TRY, EXCEPT, ELSE, FINALLY — PYTHON 3.12

```python
def parse_amount(text):
    try:
        value = float(text)
    except ValueError:
        print("  except: bad value", repr(text))
        return None
    else:
        print("  else: parsed", value)
        return value
    finally:
        print("  finally: always runs")

for t in ["1198.00", "abc"]:
    print(t, "->", parse_amount(t))
```

OUTPUT

```
  else: parsed 1198.0
  finally: always runs
1198.00 -> 1198.0
  except: bad value 'abc'
  finally: always runs
abc -> None
```

REMEMBER

`else` runs only when no exception happened. `finally` runs every time, even after `return`. Keep the try block small, so you only catch the error you expect.

CUSTOM EXCEPTIONS AND CHAINING — PYTHON 3.12

```python
class DataQualityError(Exception):
    """raised when a batch fails a hard check"""

def check_batch(rows):
    bad = [r for r in rows if r["amount"] <= 0]
    if bad:
        raise DataQualityError(f"{len(bad)} rows <= 0")

try:
    check_batch([{"amount": 10}, {"amount": -50}])
except DataQualityError as e:
    print("blocked load:", e)

try:
    int("12a")
except ValueError as e:
    raise RuntimeError("could not parse order_id") from e
```

OUTPUT

```
blocked load: 1 rows <= 0
RuntimeError: could not parse order_id
```

TRAP

Never write a bare `except:` or `except Exception: pass`. It also swallows typos (NameError) and Ctrl+C, and the job "succeeds" with missing data. `raise ... from e` keeps the original error in the traceback.

A CONTEXT MANAGER ALWAYS CLEANS UP — PYTHON 3.12

```python
class Connection:
    def __enter__(self):
        print("open connection")
        return self
    def __exit__(self, exc_type, exc, tb):
        name = exc_type.__name__ if exc_type else None
        print("close connection, error:", name)
        return False                # do not swallow errors

try:
    with Connection() as conn:
        print("loading rows")
        raise TimeoutError("warehouse timeout")
except TimeoutError:
    print("caught after cleanup")
```

OUTPUT

```
open connection
loading rows
close connection, error: TimeoutError
caught after cleanup
```

`with` calls `__exit__` whether the block succeeds or fails. That is why `with open(...)` never leaks file handles. Page 38 shows the shorter `@contextmanager` version.

PART 2 · PYTHON FOUNDATIONS · CHAPTER 8

FILES: CSV, JSON, JSON LINES

08

Most DE Python starts with reading a file someone else wrote. Know the standard library before reaching for pandas.

PATHLIB + CSV — PYTHON 3.12

```python
from pathlib import Path
import csv

data = Path(".")
print(sorted(p.name for p in data.glob("*.csv"))[:4])
p = Path("orders.csv")
print(p.suffix, p.stat().st_size, "bytes")

with open("orders.csv", newline="", encoding="utf-8") as f:
    reader = csv.DictReader(f)
    print(reader.fieldnames)
    rows = [{**r, "amount": float(r["amount"])} for r in reader]

out = Path("delivered.csv")
with out.open("w", newline="", encoding="utf-8") as f:
    w = csv.DictWriter(f, fieldnames=["order_id", "amount"],
                       extrasaction="ignore")
    w.writeheader()
    w.writerows(r for r in rows if r["status"] == "delivered")
print(out.read_text().splitlines()[:3])
```

OUTPUT

```
['employees.csv', 'logins.csv', 'order_items.csv
  ', 'orders.csv']
.csv 566 bytes
['order_id', 'customer_id', 'order_date', 'statu
  s', 'amount']
['order_id,amount', '101,1198.0', '102,796.0']
```

TRAP

Open CSV files with `newline=''`. Without it, a quoted field containing a newline is split in two, and on Windows you get blank lines between rows.

JSON VS JSON LINES — PYTHON 3.12

```python
import json

with open("events.jsonl", encoding="utf-8") as f:
    for i, line in enumerate(f):          # one record at a time
        event = json.loads(line)
        if i < 2:
            print(event["event"], event["props"]["device"])

customers = json.load(open("customers.json"))   # whole file
print(len(customers), customers[1]["address"])
print(json.dumps({"id": 1, "city": None}))
```

OUTPUT

```
view mobile
cart mobile
8 {'city': 'Mumbai', 'pincode': '400001'}
{"id": 1, "city": null}
```

DATA ENGINEERING NOTE

A JSON file is one big document: you must load all of it. JSON Lines is one document per line, so you can stream it, split it across workers and append to it. Most event pipelines use JSON Lines for exactly this reason.

Read it with	When
`for line in f`	Big text files. Reads one line at a time, constant memory.
`f.read() / f.readlines()`	Small files only. Loads everything into memory.
`csv.DictReader`	CSV with a header. Handles quoting and embedded commas that split(",") gets wrong.
`json.load / json.loads`	A file object / a string. Watch the s.
`pd.read_csv(..., chunksize=n)`	Large CSV into pandas in batches (page 70).

PART 2 · PYTHON FOUNDATIONS · PRACTICE

PRACTICE SET 1: FOUNDATIONS

P1

Cover the solution, write your own, then compare outputs. Every answer reads the store files.

Q7. Total amount and order count per status from orders.csv, without pandas. DE EASY

Approach: One pass, a dict of [count, total] per status.

```
import csv
from collections import defaultdict

stats = defaultdict(lambda: [0, 0.0])
for r in csv.DictReader(open("orders.csv")):
    s = stats[r["status"]]
    s[0] += 1
    s[1] += float(r["amount"])
for status, (n, total) in sorted(stats.items()):
    print(f"{status:<10} {n:>2} {total:>9.2f}")
```

OUTPUT

```
cancelled   2   3498.00
delivered  12  18972.00
returned    1    798.00
```

Q8. Which customers in customers.json have never placed an order? DE EASY

Approach: Two sets of ids and a difference. O(n + m).

```
import csv, json

customers = json.load(open("customers.json"))
all_ids = {c["customer_id"] for c in customers}
ordered = {int(r["customer_id"])
           for r in csv.DictReader(open("orders.csv"))}
names = {c["customer_id"]: c["name"] for c in customers}
print([names[i] for i in sorted(all_ids - ordered)])
```

OUTPUT

```
['Zoya']
```

Q9. Count events per device from events.jsonl. The device sits inside a nested props object. DE EASY

Approach: Stream the file, read the nested key safely, count with Counter.

```
import json
from collections import Counter

with open("events.jsonl") as f:
    devices = Counter(json.loads(line).get("props", {})
                      .get("device", "unknown") for line in f)
print(devices.most_common())
```

OUTPUT

```
[('mobile', 7), ('desktop', 4), ('tablet', 1)]
```

Q10. Clean these amount strings into Decimal, skipping the ones that cannot be parsed: "₹1,198.00", " 796 ", "N/A", "1,499.5". DE MEDIUM

Approach: Strip symbols and separators, try Decimal, catch only InvalidOperation.

```
from decimal import Decimal, InvalidOperation

raw = ["\u20b91,198.00", " 796 ", "N/A", "1,499.5"]
clean, rejected = [], []
for text in raw:
    t = text.replace("\u20b9", "").replace(",", "").strip()
    try:
        clean.append(Decimal(t))
    except InvalidOperation:
        rejected.append(text)
print(clean, rejected)
```

OUTPUT

```
[Decimal('1198.00'), Decimal('796'), Decimal('14
  99.5')] ['N/A']
```

Q11. From employees.csv, find the second-highest distinct salary. Return None if it does not exist. DE MEDIUM

Approach: Set of salaries removes ties, sort descending, guard the index.

```
import csv

rows = csv.DictReader(open("employees.csv"))
salaries = {int(r["salary"]) for r in rows}
ordered = sorted(salaries, reverse=True)
second = ordered[1] if len(ordered) > 1 else None
print(second, ordered[:3])
```

OUTPUT

```
180000 [250000, 180000, 160000]
```

PART 3 · ADVANCED PYTHON · CHAPTER 9

ITERATORS AND GENERATORS

09

A generator produces values one at a time, only when asked. That single idea lets a laptop process files bigger than its memory.

A THREE-STAGE LAZY PIPELINE — PYTHON 3.12

```
import csv

def read_rows(path):
    with open(path, newline="") as f:
        yield from csv.DictReader(f)       # one row at a time

def delivered(rows):
    for r in rows:
        if r["status"] == "delivered":
            yield r

def amounts(rows):
    for r in rows:
        yield float(r["amount"])

pipeline = amounts(delivered(read_rows("orders.csv")))
print(type(pipeline).__name__)    # nothing has run yet
print(sum(pipeline))
print(sum(pipeline))              # already used up
```

OUTPUT

```
generator
18972.0
0
```

TRAP

A generator can be consumed once. The second sum() gets an empty stream and returns 0 with no error. If you need the values twice, store them in a list or call the pipeline function again.

Each stage pulls one row from the stage before it. At no point does the whole file sit in memory.

WHO RUNS WHEN — PYTHON 3.12

```
def numbers():
    for i in range(3):
        print("  produce", i)
        yield i

for n in numbers():
    print("consume", n)
```

OUTPUT

```
  produce 0
consume 0
  produce 1
consume 1
  produce 2
consume 2
```

MEMORY: LIST VS GENERATOR — PYTHON 3.12

```
import sys

as_list = [x for x in range(1_000_000)]
as_gen = (x for x in range(1_000_000))
print(sys.getsizeof(as_list), "bytes")
print(sys.getsizeof(as_gen), "bytes")
print(sum(as_gen) == sum(as_list))
```

OUTPUT

```
8448728 bytes
192 bytes
True
```

The list holds a million references. The generator object stays the same size whatever range you give it.

NEXT(), STOPITERATION AND YIELD FROM — PYTHON 3.12

```
nums = iter([10, 20])
print(next(nums), next(nums))
print(next(nums, "done"))      # default, no StopIteration

def flatten(items):
    for x in items:
        if isinstance(x, list):
            yield from flatten(x)  # delegate to a sub-generator
        else:
            yield x
print(list(flatten([1, [2, [3, 4]], 5])))
```

OUTPUT

```
10 20
done
[1, 2, 3, 4, 5]
```

REMEMBER

A for loop is just repeated `next()` calls that stop when `StopIteration` is raised. `yield from sub` passes every value of sub through, which makes recursive flattening short.

	List / list comprehension	Generator / generator expression
Memory	All items at once	One item at a time
Reusable	Yes, iterate many times	No, single pass
len(), indexing	Yes	No
Use for	Small results you reuse, sorting, random access	Large files, streams, pipelines, anything you feed to sum/min/any

PART 3 · ADVANCED PYTHON · CHAPTER 10

ITERTOOLS, FUNCTOOLS, SORTING

10

The standard library already has the building blocks most coding rounds ask you to write by hand. Know them, then be ready to write them yourself.

GROUPBY ONLY GROUPS NEIGHBOURS — PYTHON 3.12

```
from itertools import groupby

statuses = ["delivered", "cancelled", "delivered", "delivered"]

print([(k, len(list(g))) for k, g in groupby(statuses)])
print([(k, len(list(g))) for k, g in groupby(sorted(statuses))])
```

OUTPUT

```
[('delivered', 1), ('cancelled', 1), ('delivere
  d', 2)]
[('cancelled', 1), ('delivered', 3)]
```

TRAP

itertools.groupby is not SQL GROUP BY. It starts a new group every time the key changes, so unsorted input gives repeated keys. Sort by the same key first, or use a dict.

THE FIVE YOU WILL USE MOST — PYTHON 3.12

```
from itertools import (batched, islice, chain,
                       accumulate, pairwise)

ids = range(101, 108)
print(list(batched(ids, 3)))            # Python 3.12+
print(list(islice(ids, 2, 5)))          # slice any iterable
print(list(chain([1, 2], (3,), "ab")))  # glue iterables
print(list(accumulate([1198, 796, 1499])))   # running total
print(list(pairwise(["01-10", "02-02", "03-05"])))
```

OUTPUT

```
[(101, 102, 103), (104, 105, 106), (107,)]
[103, 104, 105]
[1, 2, 3, 'a', 'b']
[1198, 1994, 3493]
[('01-10', '02-02'), ('02-02', '03-05')]
```

REMEMBER

`batched` is new in 3.12. On older versions, practice Q on page 40 builds it from `islice`. `pairwise` gives consecutive pairs, which is how you compute gaps between dates.

MIXED SORT DIRECTIONS, PARTIAL, REDUCE — PYTHON 3.12

```
from functools import partial, reduce
from operator import itemgetter

rows = [("Delhi", 747), ("Pune", 3498),
        ("Delhi", 846), ("Pune", 1598)]

# city descending, amount ascending:
# two stable sorts, least important key first
rows.sort(key=itemgetter(1))
rows.sort(key=itemgetter(0), reverse=True)
print(rows)

parse_bin = partial(int, base=2)
print(parse_bin("1010"), reduce(lambda a, b: a * b, [1, 2, 3]))
```

OUTPUT

```
[('Pune', 1598), ('Pune', 3498), ('Delhi', 747)
  , ('Delhi', 846)]
10 6
```

INTERVIEW TIP

"Sort by city descending, then amount ascending." Negating a key works only for numbers. For strings, sort twice, least important key first. It works because Python sorts are stable.

Tool	Does	Hand-written version asked in rounds
`collections.Counter`	Counts items	dict with get(k, 0) + 1
`heapq.nlargest(k, xs)`	Top k without a full sort	A size-k min-heap (P2)
`collections.deque(maxlen=k)`	Fixed-size sliding window	List with pop(0), which is O(n)
`functools.lru_cache`	Memoize a function	dict cache, OrderedDict for LRU (P16)
`bisect.insort`	Keep a list sorted on insert	Binary search by hand

PART 3 · ADVANCED PYTHON · CHAPTER 11

CLASSES AND DATACLASSES

11

Data engineering code uses classes for two jobs: typed records, and interchangeable steps that share one interface.

A TYPED, VALIDATED RECORD — PYTHON 3.12

```python
from dataclasses import dataclass
from datetime import date
from decimal import Decimal

@dataclass(frozen=True, slots=True)
class Order:
    order_id: int
    customer_id: int
    order_date: date
    status: str
    amount: Decimal

    def __post_init__(self):
        if self.amount <= 0:
            raise ValueError(f"order {self.order_id}: amount <= 0")

    @classmethod
    def from_row(cls, r: dict) -> "Order":
        return cls(int(r["order_id"]), int(r["customer_id"]),
                   date.fromisoformat(r["order_date"]),
                   r["status"].strip().lower(), Decimal(r["amount"]))

o = Order.from_row({"order_id": "101", "customer_id": "1",
                    "order_date": "2025-01-10",
                    "status": " Delivered", "amount": "1198.00"})
print(o)
print(o == Order(101, 1, date(2025, 1, 10), "delivered",
                 Decimal("1198.00")))
try:
    Order(116, 8, date(2025, 5, 2), "delivered", Decimal("-50"))
except ValueError as e:
    print("rejected:", e)
```

OUTPUT

```
Order(order_id=101, customer_id=1, order_date=da
  tetime.date(2025, 1, 10), status='delivered',
  amount=Decimal('1198.00'))
True
rejected: order 116: amount <= 0
```

REMEMBER

`@dataclass` writes `__init__`, `__repr__` and `__eq__` for you. `frozen=True` makes instances read-only, so a record cannot be changed halfway through a pipeline. `slots=True` cuts memory per object, which matters when you hold a million of them.

ONE INTERFACE, MANY SOURCES — PYTHON 3.12

```python
import csv, json
from abc import ABC, abstractmethod

class Extractor(ABC):
    @abstractmethod
    def rows(self): ...

class CsvExtractor(Extractor):
    def __init__(self, path): self.path = path
    def rows(self):
        with open(self.path, newline="") as f:
            yield from csv.DictReader(f)

class JsonlExtractor(Extractor):
    def __init__(self, path): self.path = path
    def rows(self):
        with open(self.path) as f:
            for line in f:
                yield json.loads(line)

sources = [CsvExtractor("orders.csv"),
           JsonlExtractor("events.jsonl")]
for source in sources:
    n = sum(1 for _ in source.rows())   # same call, any source
    print(type(source).__name__, n)

try:
    Extractor()
except TypeError as e:
    print(e)
```

OUTPUT

```
CsvExtractor 15
JsonlExtractor 12
Can't instantiate abstract class Extractor witho
  ut an implementation for abstract method 'rows
  '
```

DATA ENGINEERING NOTE

This is how real frameworks are shaped: Airflow operators, Spark data sources, dbt adapters. The pipeline only knows "an Extractor has rows()". Adding a new source means a new class, not edits all over the codebase.

INTERVIEW TIP

Prefer composition (a pipeline that holds steps) over deep inheritance chains. Past two levels of inheritance, code gets hard to follow.

PART 3 · ADVANCED PYTHON · CHAPTER 12

DECORATORS AND CONTEXT MANAGERS

12

A decorator wraps a function to add behaviour around it: timing, retries, logging, caching. Pipelines use them everywhere.

A TIMING DECORATOR — PYTHON 3.12

```python
import functools, time

def timed(fn):
    @functools.wraps(fn)            # keep name and docstring
    def wrapper(*args, **kwargs):
        start = time.perf_counter()
        result = fn(*args, **kwargs)
        took = time.perf_counter() - start
        print(f"{fn.__name__} took {took:.1f}s")
        return result
    return wrapper

@timed
def load_batch(rows):
    "Pretend to write rows to the warehouse."
    time.sleep(0.2)
    return len(rows)

print(load_batch([1, 2, 3]), load_batch.__name__)
print(load_batch.__doc__)
```

OUTPUT

```
load_batch took 0.2s
3 load_batch
Pretend to write rows to the warehouse.
```

REMEMBER

`@timed` above a def means `load_batch = timed(load_batch)`. Without `functools.wraps`, the wrapped function would report its name as "wrapper", which breaks logs and Airflow task names.

A DECORATOR THAT TAKES ARGUMENTS — PYTHON 3.12

```python
import functools, time

def retry(times=3, delay=0.01, on=(ConnectionError,)):
    def decorator(fn):
        @functools.wraps(fn)
        def wrapper(*args, **kwargs):
            for attempt in range(1, times + 1):
                try:
                    return fn(*args, **kwargs)
                except on as e:
                    print(f"attempt {attempt} failed: {e}")
                    if attempt == times:
                        raise
                    time.sleep(delay * 2 ** (attempt - 1))
        return wrapper
    return decorator

calls = {"n": 0}
@retry(times=3)
def fetch_page():
    calls["n"] += 1
    if calls["n"] < 3:
        raise ConnectionError("warehouse timeout")
    return "page ok"
print(fetch_page())
```

OUTPUT

```
attempt 1 failed: warehouse timeout
attempt 2 failed: warehouse timeout
page ok
```

APPROACH

A decorator with arguments is three functions deep: retry(times) returns decorator(fn), which returns wrapper(*args). Interviewers often ask for exactly this one. P14 on page 66 adds jitter and a cap.

TRAP

Retry only errors that can succeed on a second try: timeouts and dropped connections. Retrying a ValueError from bad data just fails three times, slower.

@CONTEXTMANAGER: SETUP, YIELD, TEARDOWN — PYTHON 3.12

```python
from contextlib import contextmanager

@contextmanager
def step(name):
    print(f"start {name}")
    try:
        yield
        print(f"done  {name}")
    except Exception as e:
        print(f"fail  {name}: {e}")
        raise
    finally:
        print(f"cleanup {name}")
with step("extract"):
    rows = [1, 2, 3]
try:
    with step("load"):
        raise TimeoutError("warehouse timeout")
except TimeoutError:
    pass
```

OUTPUT

```
start extract
done  extract
cleanup extract
start load
fail  load: warehouse timeout
cleanup load
```

Everything before yield is setup, everything after it is teardown. The finally block runs whether the step passed or failed, which is where you release locks, close connections or delete temp files.

PART 3 · ADVANCED PYTHON · CHAPTER 13

CONCURRENCY: THREADS, PROCESSES, ASYNC

13

Waiting on the network and crunching numbers need different tools. Pick by what your code spends its time doing.

THREADS FOR I/O-BOUND WORK — PYTHON 3.12

```
import time
from concurrent.futures import ThreadPoolExecutor

def fetch(page):                  # stands in for an API call
    time.sleep(0.3)               # waiting on the network
    return f"page {page}: 100 rows"

start = time.perf_counter()
results = [fetch(p) for p in range(1, 6)]
print(f"one by one: {time.perf_counter() - start:.1f}s")

start = time.perf_counter()
with ThreadPoolExecutor(max_workers=5) as pool:
    results = list(pool.map(fetch, range(1, 6)))
print(f"5 threads:  {time.perf_counter() - start:.1f}s")
print(results[0], "...", results[-1])
```

OUTPUT

```
one by one: 1.5s
5 threads:  0.3s
page 1: 100 rows ... page 5: 100 rows
```

REMEMBER

While one thread waits on the network, the others run. Five 0.3 second waits overlap into roughly one. `pool.map` returns results in input order even when they finish out of order.

ASYNCIO: ONE THREAD, MANY WAITS — PYTHON 3.12

```
import asyncio, time

async def fetch(page):
    await asyncio.sleep(0.3)      # non-blocking wait
    return page * 100

async def main():
    start = time.perf_counter()
    jobs = (fetch(p) for p in range(1, 6))
    rows = await asyncio.gather(*jobs)
    took = time.perf_counter() - start
    print(rows, f"{took:.1f}s")

asyncio.run(main())
```

OUTPUT

```
[100, 200, 300, 400, 500] 0.3s
```

PROCESSES FOR CPU-BOUND WORK — PYTHON 3.12

```
from concurrent.futures import ProcessPoolExecutor

def checksum(chunk):              # CPU-bound work
    return sum(x * x % 7 for x in chunk)

if __name__ == "__main__":        # needed for processes
    step = 200_000
    chunks = [range(i, i + step)
              for i in range(0, 800_000, step)]
    with ProcessPoolExecutor(max_workers=2) as pool:
        print(list(pool.map(checksum, chunks)))
```

OUTPUT

```
[399999, 400002, 399996, 400002]
```

	Threads	Processes	asyncio
Best for	I/O: API calls, S3 downloads, DB queries	CPU: parsing, hashing, compression, heavy transforms	Thousands of concurrent I/O calls
GIL effect	Only one thread runs Python bytecode at a time	Each process has its own interpreter and GIL	Single thread, so no GIL contention
Cost	Low	High: separate memory, data must be pickled	Lowest, but every library call must be async
DE example	Pull 50 API pages in parallel	Parse 200 large files on 8 cores	aiohttp crawler, async DB drivers

INTERVIEW TIP

"What is the GIL?" The Global Interpreter Lock lets only one thread execute Python bytecode at a time in a process. It is released while a thread waits on I/O, which is why threads still help for network work. Python 3.13 added an optional free-threaded build without the GIL; standard builds still have it.

PART 3 · ADVANCED PYTHON · PRACTICE

PRACTICE SET 2: ADVANCED

P2

Generators, itertools, decorators and classes, the way coding rounds ask for them.

Q12. Write chunked(iterable, n) that yields lists of n items, without itertools.batched. It must work on a generator.

DE MEDIUM

Approach: islice pulls n items from one shared iterator until a slice comes back empty.

```
from itertools import islice

def chunked(iterable, n):
    it = iter(iterable)                 # one shared iterator
    while batch := list(islice(it, n)):
        yield batch

print(list(chunked(range(101, 108), 3)))
print(list(chunked((x for x in "abcde"), 2)))
```

OUTPUT

```
[[101, 102, 103], [104, 105, 106], [107]]
[['a', 'b'], ['c', 'd'], ['e']]
```

Q13. Run-length encode a status stream: ["D","D","C","D","D","D"] becomes "D2C1D3".

DE EASY

Approach: groupby on unsorted input is exactly right here: we want runs of neighbours.

```
from itertools import groupby

stream = ["D", "D", "C", "D", "D", "D"]
runs = (f"{k}{sum(1 for _ in g)}" for k, g in groupby(stream))
print("".join(runs))
```

OUTPUT

```
D2C1D3
```

Q14. Write a decorator count_calls that records how many times each decorated function ran.

DE MEDIUM

Approach: Store state on the wrapper itself, and keep the name with functools.wraps.

```
import functools

def count_calls(fn):
    @functools.wraps(fn)
    def wrapper(*args, **kwargs):
        wrapper.calls += 1
        return fn(*args, **kwargs)
    wrapper.calls = 0
    return wrapper

@count_calls
def load(batch): return len(batch)

for b in ([1, 2], [3], [4, 5, 6]):
    load(b)
print(load.__name__, load.calls)
```

OUTPUT

```
load 3
```

Q15. Sort employees.csv by department A to Z, then salary high to low, then name. Print the top two per department.

DE MEDIUM

Approach: One sort with a tuple key (salary negated), then groupby on the now-sorted department.

```
import csv
from itertools import groupby, islice

rows = list(csv.DictReader(open("employees.csv")))
rows.sort(key=lambda r: (r["dept"], -int(r["salary"]), r["name"]))
for dept, group in groupby(rows, key=lambda r: r["dept"]):
    top = [(r["name"], r["salary"]) for r in islice(group, 2)]
    print(dept, top)
```

OUTPUT

```
Analytics [('Ishita', '160000'), ('Neha', '150000')]
Engineering [('Anita', '250000'), ('Rahul', '180000')]
Sales [('Dev', '90000'), ('Tara', '90000')]
```

APPROACH

Before Part 4. DSA problems and data patterns alike are mostly a dict, a set, a heap or a deque, plus one of the tools from these six pages. If any answer here felt shaky, reread pages 35 and 36 first: generators and itertools carry most of the next ten pages.

PART 4 · DSA AND COMPLEXITY · CHAPTER 14

BIG-O IN PLAIN WORDS

14

Big-O answers one question: when the input gets 10 times bigger, how much more work does my code do? Not seconds, but how the work grows.

Think of finding one customer's name in a register. If the register is **unsorted**, you may read every line: double the register, double the work. That is **O(n)**. If it is **sorted**, you open the middle and throw away half each time: a register 1,000 times bigger needs only about 10 more looks. That is **O(log n)**. If you compare every name with every other name to find duplicates, double the register and the work goes up 4 times. That is **O(n²)**.

STEPS NEEDED AS N GROWS — PYTHON 3.12

```
import math

rows = [("O(1)", lambda n: 1),
        ("O(log n)", lambda n: math.log2(n)),
        ("O(n)", lambda n: n),
        ("O(n log n)", lambda n: n * math.log2(n)),
        ("O(n^2)", lambda n: n * n),
        ("O(2^n)", lambda n: 2 ** n)]
sizes = [10, 1_000, 100_000]
print(f"{'growth':<11}" + "".join(f"{s:>16,}" for s in sizes))
for name, f in rows:
    cells = ""
    for s in sizes:
        if name == "O(2^n)" and s > 64:
            cells += f"{'too many':>16}"
        else:
            cells += f"{f(s):>16,.0f}"
    print(f"{name:<11}" + cells)
```

OUTPUT

```
growth                10           1,000         100,000
O(1)                   1               1               1
O(log n)               3              10              17
O(n)                  10           1,000         100,000
O(n log n)            33           9,966       1,660,964
O(n^2)               100       1,000,000  10,000,000,000
O(2^n)             1,024        too many        too many
```

O(n log n) for a lakh of rows is about 17 lakh steps. O(n²) is 10 billion. That gap is why one solution finishes in a blink and the other times out.

MEASURED: DOUBLE N, SEE WHAT HAPPENS — PYTHON 3.12

```
import random, time

def has_dup_nested(xs):            # O(n^2): compare every pair
    for i in range(len(xs)):
        for j in range(i + 1, len(xs)):
            if xs[i] == xs[j]:
                return True
    return False

def has_dup_set(xs):               # O(n): one pass with a set
    seen = set()
    for x in xs:
        if x in seen:
            return True
        seen.add(x)
    return False

def ms(fn, xs):
    t = time.perf_counter(); fn(xs)
    return (time.perf_counter() - t) * 1000

for n in (1_000, 2_000, 4_000):
    xs = random.sample(range(10 * n), n)   # no duplicates: worst case
    a, b = ms(has_dup_nested, xs), ms(has_dup_set, xs)
    print(f"n={n:>5}  nested {a:7.1f} ms   set {b:5.2f} ms")
```

OUTPUT

```
n= 1000  nested    21.1 ms   set  0.09 ms
n= 2000  nested    91.5 ms   set  0.35 ms
n= 4000  nested   348.7 ms   set  0.44 ms
```

REMEMBER

Every time n doubles, the nested version takes about 4 times longer (n²). The set version only doubles (n). Your milliseconds will differ; the growth will not.

Five rules for working it out

- **Loops one after another add:** O(n) + O(n) = O(n).
- **Loops inside loops multiply:** a loop over n inside a loop over n is O(n²).
- **Drop constants:** 3n + 5 is still O(n). Big-O is about growth, not exact counts.
- **Keep the biggest term:** n² + n is O(n²).
- **Different inputs, different letters:** orders x customers is O(n x m), not O(n²).

How big can n be?

If n is about	You can afford	Typical approach
20	O(2^n), O(n!)	Try every subset or order
1,000 to 5,000	O(n²)	Compare pairs
1 lakh to 10 lakh	O(n log n), O(n)	Sort, hash, one pass
Crores	O(n), O(log n)	Stream, binary search, or Spark

Rule of thumb: plain Python manages very roughly 10 million simple steps a second. Read the input size in the question and pick a complexity that fits.

PART 4 · DSA AND COMPLEXITY · CHAPTER 15

SPACE COMPLEXITY

15

Space complexity is the same idea for memory: how much extra memory your code needs as the input grows. In data engineering this is often the limit you hit first.

Count only the **extra** memory your code creates, not the input you were given. A loop that keeps one running total uses O(1) extra space whether the file has 10 rows or 10 crore. Copying every row into a list uses O(n). A recursive function that calls itself n levels deep uses O(n) as well, because every unfinished call waits on the stack.

O(N) VS O(1) EXTRA MEMORY, MEASURED — PYTHON 3.12

```python
import tracemalloc

def peak_kb(fn):
    tracemalloc.start()
    fn()
    _, peak = tracemalloc.get_traced_memory()
    tracemalloc.stop()
    return peak // 1024

n = 1_000_000
as_list = lambda: sum([x * 2 for x in range(n)])  # a list
as_gen = lambda: sum(x * 2 for x in range(n))     # streams
print("list:", peak_kb(as_list), "KB extra")
print("gen: ", peak_kb(as_gen), "KB extra")
```

OUTPUT

```
list: 39496 KB extra
gen:  0 KB extra
```

Same answer, same O(n) time. The list version holds a million numbers at once; the generator holds one. On a file bigger than RAM, only the second one finishes.

RECURSION USES STACK SPACE — PYTHON 3.12

```python
import sys

def total(xs, i=0):        # recursion: O(n) stack
    if i == len(xs):
        return 0
    return xs[i] + total(xs, i + 1)

print(sys.getrecursionlimit())
print(total(list(range(500))))
print(total(list(range(5_000))))
```

OUTPUT

```
1000
124750
RecursionError: maximum recursion depth exceeded
```

THE SAME LOGIC AS A LOOP — PYTHON 3.12

```python
def total(xs):                 # loop: O(1) extra
    s = 0
    for x in xs:
        s += x
    return s

print(total(list(range(5_000))))
print(total(range(10_000_000)))
```

OUTPUT

```
12497500
49999995000000
```

CPython stops recursion at about 1,000 levels by default, so deep recursion on real data crashes. Python does not optimise tail calls: turn deep recursion into a loop or an explicit stack.

Code that...	Extra space	Plain-language reason
keeps a few counters	O(1)	The same handful of variables, whatever n is
builds a set, dict or list of the input	O(n)	One entry per item
slices `xs[1:]` or copies a list	O(k)	A slice is a new list of k items, not a view
recurses d levels deep	O(d)	Each waiting call sits on the stack
builds an n x n table (some DP)	O(n²)	Every pair gets a cell
streams a file with a generator	O(1)	Only the current row is in memory

INTERVIEW TIP

Interviewers often ask "can you do it in O(1) extra space?" after you solve with a set. Usually the answer is sort first (if you may change the input) or two pointers. Say the trade out loud: "O(1) space, but O(n log n) time and I modify the input."

PART 4 · DSA AND COMPLEXITY · CHAPTER 16

TIME VS SPACE TRADE-OFFS

16

Most "optimise this" questions are a trade: spend memory to save time, or spend time to save memory. Say which one you are making and why.

THREE WAYS TO ANSWER "IS THIS ID PRESENT?" — PYTHON 3.12

```python
import sys, time, bisect

ids = list(range(0, 2_000_000, 2))          # 10 lakh sorted ids
targets = list(range(0, 2_000_000, 2_001))  # 1,000 lookups

def bench(fn):
    t = time.perf_counter(); fn(); return (time.perf_counter() - t) * 1000

as_set = set(ids)
lst = bench(lambda: [t in ids for t in targets])
st = bench(lambda: [t in as_set for t in targets])
bs = bench(lambda: [ids[bisect.bisect_left(ids, t)] == t for t in targets])

kb = lambda obj: sys.getsizeof(obj) // 1024
print(f"list scan {lst:8.1f} ms   memory {kb(ids):>6} KB")
print(f"set       {st:8.2f} ms   memory {kb(as_set):>6} KB")
print(f"bisect    {bs:8.2f} ms   memory      0 KB extra")
```

OUTPUT

```
list scan   8504.1 ms   memory   7812 KB
set            0.71 ms   memory  32768 KB
bisect         2.11 ms   memory      0 KB extra
```

APPROACH

The set is fastest but costs extra memory, several times the list. Binary search on the already sorted list is nearly as fast and needs no extra memory. The plain scan needs nothing extra and is thousands of times slower.

PRECOMPUTE ONCE, ANSWER FAST FOREVER: PREFIX SUMS — PYTHON 3.12

```python
from itertools import accumulate

daily = [0, 1999, 0, 0, 846, 0, 0, 2397, 0, 1598]
prefix = [0, *accumulate(daily)]          # O(n) once, O(n) memory

def range_sum(i, j):                      # O(1) per question
    return prefix[j + 1] - prefix[i]

print(prefix)
print(range_sum(1, 4), range_sum(0, 9), sum(daily[1:5]))
```

OUTPUT

```
[0, 0, 1999, 1999, 1999, 2845, 2845, 2845, 52
  42, 5242, 6840]
2845 6840 2845
```

Summing a range by hand costs O(k) every time you ask. One O(n) pass to build the prefix array makes every later question O(1). Worth it when the same data is queried many times.

Problem	Spend memory for speed	Spend time to save memory	Pick the cheaper-memory side when
Is x present?	set: O(1) lookup, O(n) memory	sort + bisect: O(log n), no extra	Data is already sorted or memory is tight
Find duplicates	set in one pass: O(n) / O(n)	sort then compare neighbours: O(n log n) / O(1)	You may modify the input
Top k of n	sort all: O(n log n) / O(n)	size-k heap: O(n log k) / O(k)	n is huge or streamed
Many range sums	prefix array: O(1) per query	loop per query: O(k)	Only a few queries
Repeat lookups	cache results (lru_cache)	recompute each time	Inputs rarely repeat
Join two files	hash join: O(n + m), small side in RAM	sort-merge join: O(n log n), streams	Neither side fits in memory
Process a file	load it all: fast, O(n) RAM	stream rows: O(1) RAM	Almost always in DE

DATA ENGINEERING NOTE

In DE interviews the follow-up is almost always "and if it does not fit in memory?" The answers are the right column: stream, chunk, sort-merge, partition by key and process partitions separately, or hand it to Spark, which does all four for you.

PART 4 · DSA AND COMPLEXITY · CHAPTER 17

WHAT PYTHON OPERATIONS COST

17

Half of all slow interview code is one innocent-looking line inside a loop. Learn the cost of the built-ins and you will spot it immediately.

Structure	Cheap: O(1) or O(log n)	Expensive: O(n) or worse	Watch out
`list`	xs[i], append, pop(), len	`x in xs`, insert(0, x), pop(0), remove(x), index(x), slicing O(k)	A list is an array, not a linked list
`dict`	d[k], d.get, k in d, set, delete	Iterating all keys, copying	Worst case O(n) with heavy hash collisions, rare in practice
`set`	add, remove, x in s	Union, intersection: O(len of inputs)	Items must be hashable
`collections.deque`	append, appendleft, pop, popleft	Indexing into the middle	Use it for queues and windows
`heapq`	push, pop: O(log n), peek heap[0]: O(1)	heapify is O(n); searching a heap is O(n)	Min-heap only; negate for max
`str`	len, s[i]	`s += x` in a loop, `sub in s`, replace, split	Strings are immutable: every change makes a copy
`sorted / list.sort`		O(n log n)	Sorting inside a loop is O(n² log n)
`bisect`	bisect_left: O(log n)	insort: O(n), because the list shifts	Only on a sorted list

ONE LINE, A HUGE DIFFERENCE — PYTHON 3.12

```python
import time
from collections import deque

n = 100_000
def ms(fn):
    t = time.perf_counter()
    fn()
    return (time.perf_counter() - t) * 1000

def drain_list():
    q = list(range(n))
    while q:
        q.pop(0)              # shifts every remaining item

def drain_deque():
    q = deque(range(n))
    while q:
        q.popleft()           # O(1)

print(f"list.pop(0)     {ms(drain_list):8.1f} ms")
print(f"deque.popleft() {ms(drain_deque):8.1f} ms")
```

OUTPUT

```
list.pop(0)        728.8 ms
deque.popleft()     10.5 ms
```

TRAP

`pop(0)` looks constant but moves every remaining item one step left: O(n) each time, O(n²) for the whole loop. Any queue (BFS, a task queue, a window) should be a deque.

THE HIDDEN LOOP INSIDE "IN" — PYTHON 3.12

```python
orders = list(range(50_000))
vip_list = list(range(0, 50_000, 7))
vip_set = set(vip_list)

import time
t = time.perf_counter()
a = [o for o in orders if o in vip_list]     # O(n x m)
t1 = time.perf_counter()
b = [o for o in orders if o in vip_set]      # O(n)
t2 = time.perf_counter()
print(len(a) == len(b))
print(f"list {1000*(t1-t):.0f} ms, set {1000*(t2-t1):.1f} ms")
```

OUTPUT

```
True
list 3573 ms, set 2.0 ms
```

`o in vip_list` is a loop hiding inside a loop. Converting the list to a set once (O(m)) turns the whole filter from O(n x m) into O(n + m). This is the single most common fix in DE coding rounds.

PART 4 · DSA AND COMPLEXITY · CHAPTER 18

ARRAYS AND STRINGS: TWO POINTERS

18

Two pointers walk through the data from two positions at once. They turn many O(n²) "check every pair" problems into a single O(n) pass.

PAIR WITH A TARGET SUM IN A SORTED LIST — PYTHON 3.12

```
def pair_with_sum(sorted_amounts, target):
    i, j = 0, len(sorted_amounts) - 1
    while i < j:
        s = sorted_amounts[i] + sorted_amounts[j]
        if s == target:
            return sorted_amounts[i], sorted_amounts[j]
        if s < target:
            i += 1            # need a bigger sum: move left up
        else:
            j -= 1            # need a smaller sum: move right down
    return None

amounts = sorted([1198, 796, 1499, 1999, 1348, 1048, 747, 3498])
print(amounts)
print(pair_with_sum(amounts, 2847), pair_with_sum(amounts, 5))
```

OUTPUT

```
[747, 796, 1048, 1198, 1348, 1499, 1999, 3498]
(1348, 1499) None
```

APPROACH

Sorted input means every move discards options safely. O(n) time and O(1) extra space. On unsorted input, the hash map version (page 46) is O(n) time but O(n) space; sorting first costs O(n log n). That is the trade to mention.

READ AND WRITE POINTERS — PYTHON 3.12

```
def dedupe_sorted(xs):            # in place, O(1) extra
    if not xs:
        return 0
    write = 1
    for read in range(1, len(xs)):
        if xs[read] != xs[write - 1]:
            xs[write] = xs[read]
            write += 1
    del xs[write:]
    return write

ids = [101, 101, 102, 103, 103, 103, 105]
print(dedupe_sorted(ids), ids)
```

OUTPUT

```
4 [101, 102, 103, 105]
```

POINTERS FROM BOTH ENDS — PYTHON 3.12

```
def is_palindrome(s):
    i, j = 0, len(s) - 1
    while i < j:
        if not s[i].isalnum():
            i += 1
        elif not s[j].isalnum():
            j -= 1
        elif s[i].lower() != s[j].lower():
            return False
        else:
            i, j = i + 1, j - 1
    return True

print(is_palindrome("Was it a car or a cat I saw?"),
      is_palindrome("prepnplaced"))
```

OUTPUT

```
True False
```

PREFIX SUMS AND PARSING WITHOUT REGEX — PYTHON 3.12

```
from itertools import accumulate

changes = [5, -2, -8, 3, 4, -1, 6, -9]     # daily change in signups
running = list(accumulate(changes))
print("running total:", running)
print("lowest point after day", running.index(min(running)) + 1)

s = "2025-04-01 02:00:05 WARN job=orders_daily"
date, time_, level, job = s.split(maxsplit=3)
print(date, level, job.partition("=")[2])
```

OUTPUT

```
running total: [5, 3, -5, -2, 2, 1, 7, -2]
lowest point after day 3
2025-04-01 WARN orders_daily
```

`split(maxsplit=3)` stops after three cuts, so the rest of the line stays whole. `partition` always returns three parts, so it never raises on a missing "=" the way unpacking a split can.

PART 4 · DSA AND COMPLEXITY · CHAPTER 19

HASHING: THE MOST ASKED TOPIC

19

A hash map answers "have I seen this before?" in O(1). Most medium problems become easy once you ask what to store in it.

In plain words: a dict turns a key into a number (the hash) and uses it to jump straight to a slot. No scanning. The cost is memory: one slot per key, plus spare room so slots stay mostly empty.

TWO-SUM AND GROUP ANAGRAMS — PYTHON 3.12

```python
def two_sum(nums, target):           # unsorted input
    seen = {}                          # value -> index
    for i, x in enumerate(nums):
        if target - x in seen:
            return seen[target - x], i
        seen[x] = i
    return None

print(two_sum([1198, 796, 1499, 1999, 1348], 2847))

from collections import defaultdict
def group_anagrams(words):
    groups = defaultdict(list)
    for w in words:
        groups["".join(sorted(w))].append(w)    # same letters, same key
    return list(groups.values())

words = ["listen", "silent", "enlist", "data", "tada", "etl"]
print(group_anagrams(words))
```

OUTPUT

```
(2, 4)
[['listen', 'silent', 'enlist'], ['data', 'tada'
  ], ['etl']]
```

APPROACH

Two-sum: for each number, ask "is its partner already in the map?" O(n) time, O(n) space. Anagrams: the trick is the key. Sorted letters make every anagram share one key: O(n x k log k) for n words of length k.

PREFIX SUM + HASH MAP, AND FIRST UNIQUE — PYTHON 3.12

```python
from collections import Counter

def subarrays_with_sum(nums, k):
    count, running = 0, 0
    seen = Counter({0: 1})              # prefix sum -> how many times
    for x in nums:
        running += x
        count += seen[running - k]      # earlier prefixes that leave k
        seen[running] += 1
    return count

print(subarrays_with_sum([1, 2, 3, -1, 1, 2], 3))

def first_unique(stream):
    counts = Counter(stream)
    return next((x for x in stream if counts[x] == 1), None)

print(first_unique(["u2", "u1", "u2", "u3", "u1"]))
```

OUTPUT

```
4
u3
```

REMEMBER

Subarray sum equals k is a classic medium. Brute force checks every start and end: O(n²). If the running total now is R and an earlier running total was R - k, the numbers in between add up to k. A Counter of earlier totals makes it O(n).

Store in the map...	To answer	Example
value -> index	"Where is the partner?"	Two-sum
sorted letters -> words	"Which items are equivalent?"	Group anagrams, normalised addresses
item -> count	"How many / first unique / most common?"	Counter, first non-repeating
prefix sum -> count	"How many ranges add to k?"	Subarray sum equals k
id -> latest record	"What is the current state?"	Dedupe, CDC (P4)

PART 4 · DSA AND COMPLEXITY · CHAPTER 20

SLIDING WINDOWS, VARIABLE SIZE

20

A variable window grows on the right until a rule breaks, then shrinks from the left until the rule holds again. Each item enters and leaves once, so it is O(n).

LONGEST SUBSTRING WITHOUT REPEATING CHARACTERS — PYTHON 3.12

```python
def longest_unique(s):
    last = {}                       # char -> last index seen
    start = best = 0
    for i, ch in enumerate(s):
        if ch in last and last[ch] >= start:
            start = last[ch] + 1     # shrink past the repeat
        last[ch] = i
        best = max(best, i - start + 1)
    return best

print([longest_unique(s) for s in ["abcabcbb", "pwwkew", ""]])
```

OUTPUT

```
[3, 3, 0]
```

APPROACH

Rule: no repeated character inside the window. When a repeat appears, jump the left edge past its last position. O(n) time, O(k) space for k distinct characters.

SHORTEST RUN OF DAYS REACHING A TARGET — PYTHON 3.12

```python
def shortest_with_sum(nums, target):
    left = total = 0
    best = float("inf")
    for right, x in enumerate(nums):
        total += x                          # grow
        while total >= target:              # rule met: try to shrink
            best = min(best, right - left + 1)
            total -= nums[left]
            left += 1
    return 0 if best == float("inf") else best

print(shortest_with_sum([2, 3, 1, 2, 4, 3], 7))
```

OUTPUT

```
2
```

Works because all values are positive: growing the window only increases the total, shrinking only decreases it. With negative numbers this breaks; use the prefix-sum map from page 46 instead.

DE FLAVOUR: BUSIEST 30-MINUTE WINDOW OF EVENTS — PYTHON 3.12

```python
from collections import deque
from datetime import datetime, timedelta

times = ["10:00", "10:02", "10:05", "10:20", "10:22", "10:40",
         "11:00", "11:10", "11:15", "12:00"]
ts = [datetime.strptime(t, "%H:%M") for t in times]

window, peak = deque(), (0, None)
for t in ts:                                    # already sorted by time
    window.append(t)
    while window[0] <= t - timedelta(minutes=30):
        window.popleft()                        # older than 30 minutes
    if len(window) > peak[0]:
        peak = (len(window), t.strftime("%H:%M"))
print("busiest 30-minute window:", peak[0], "events, ending", peak[1])
```

OUTPUT

```
busiest 30-minute window: 5 events, ending 1
  0:22
```

DATA ENGINEERING NOTE

The same shape powers rate limiting ("no more than 100 calls a minute") and alerting ("more than 5 errors in 10 minutes"). Each timestamp enters and leaves the deque once: O(n).

	Fixed-size window (P5)	Variable-size window (this page)
Size	Always k items or k days	Grows and shrinks with a rule
Move	Add one on the right, drop one on the left	Add on the right; drop from the left while the rule is broken
State	Running total, or a deque of the last k	A dict of counts or last positions, or a running total
Typical question	Rolling 7-day revenue, moving average	Longest run with no repeats, shortest run reaching a target

PART 4 · DSA AND COMPLEXITY · CHAPTER 21

STACKS AND QUEUES

21

A stack is last in, first out, like a pile of plates. A queue is first in, first out, like a ticket line. Picking the right one is usually the whole trick.

BALANCED BRACKETS — PYTHON 3.12

```python
def balanced(text):
    pairs = {")": "(", "]": "[", "}": "{"}
    stack = []
    for ch in text:
        if ch in "([{":
            stack.append(ch)
        elif ch in pairs:
            if not stack or stack.pop() != pairs[ch]:
                return False
    return not stack                    # nothing left open

print(balanced('{"a": [1, 2, {"b": (3)}]}'),
      balanced('{"a": [1, 2}'), balanced("(("))
```

OUTPUT

```
True False False
```

REMEMBER

The most recent unclosed bracket must close first, which is exactly what a stack gives you. O(n) time, O(n) space in the worst case (all openers).

MONOTONIC STACK: DAYS UNTIL A HIGHER VALUE — PYTHON 3.12

```python
def days_until_higher(revenue):
    answer = [0] * len(revenue)
    stack = []                          # indices still waiting
    for i, r in enumerate(revenue):
        while stack and revenue[stack[-1]] < r:
            j = stack.pop()
            answer[j] = i - j           # day j finally beaten
        stack.append(i)
    return answer

print(days_until_higher([1994, 2847, 7291, 6840, 7500, 3000, 3100]))
```

OUTPUT

```
[1, 1, 2, 1, 0, 1, 0]
```

APPROACH

Brute force looks ahead from every day: O(n²). The stack keeps days still waiting for a higher value, in decreasing order. Each day is pushed and popped once: O(n). This is the "daily temperatures" / "next greater element" family.

A QUEUE: LIST A FOLDER TREE LEVEL BY LEVEL — PYTHON 3.12

```python
from collections import deque

lake = {"raw": {"orders": ["01.csv", "02.csv"],
                "events": ["a.jsonl"]},
        "README.md": None}

q, files = deque([("", lake)]), []
while q:
    path, node = q.popleft()            # FIFO: shallow first
    for name, child in node.items():
        full = f"{path}/{name}"
        if isinstance(child, dict):
            q.append((full, child))     # a folder: visit later
        elif child is None:
            files.append(full)
        else:
            files += [f"{full}/{f}" for f in child]
print(files)
```

OUTPUT

```
['/README.md', '/raw/orders/01.csv', '/raw/orde
  rs/02.csv', '/raw/events/a.jsonl']
```

Queues drive BFS (pages 53 and 54), task schedulers and buffers between pipeline stages. Always a deque, never list.pop(0) (page 44).

Use a stack (list: append / pop) when	Use a queue (deque: append / popleft) when
The most recent item must be handled first: brackets, undo, nested parsing	Items must be handled in arrival order: tasks, messages, buffers
You go deep first: DFS, backtracking, recursion turned into a loop	You go level by level: BFS, shortest path when every step costs 1
You wait for "the next bigger/smaller value": monotonic stack	You keep a time window: last 30 minutes of events

PART 4 · DSA AND COMPLEXITY · CHAPTER 22

LINKED LISTS: THE THREE CLASSICS

22

Rare in DE loops, common in SDE loops. Python has no built-in linked list, so learn these three and move on.

In plain words: each node holds a value and a pointer to the next node, like a treasure hunt where each clue tells you where the next one is. You cannot jump to item 5 directly (O(n) to reach it), but you can add or remove a node anywhere in O(1) once you are standing next to it.

REVERSE, DETECT A CYCLE, MERGE TWO SORTED — PYTHON 3.12

```python
from dataclasses import dataclass

@dataclass
class Node:
    val: int
    next: "Node | None" = None

def build(vals):
    head = None
    for v in reversed(vals):
        head = Node(v, head)
    return head

def to_list(head):
    out = []
    while head:
        out.append(head.val); head = head.next
    return out

def reverse(head):                    # O(n) time, O(1) space
    prev = None
    while head:
        head.next, prev, head = prev, head, head.next
    return prev

def has_cycle(head):                  # Floyd: slow and fast pointers
    slow = fast = head
    while fast and fast.next:
        slow, fast = slow.next, fast.next.next
        if slow is fast:
            return True
    return False

def merge(a, b):                      # two sorted lists
    dummy = tail = Node(0)
    while a and b:
        if a.val <= b.val:
            tail.next, a = a, a.next
        else:
            tail.next, b = b, b.next
        tail = tail.next
    tail.next = a or b
    return dummy.next

print(to_list(reverse(build([1, 2, 3, 4]))))
loop = build([1, 2, 3]); loop.next.next.next = loop.next
print(has_cycle(loop), has_cycle(build([1, 2, 3])))
print(to_list(merge(build([1, 4, 9]), build([2, 3, 10]))))
```

OUTPUT

```
[4, 3, 2, 1]
True False
[1, 2, 3, 4, 9, 10]
```

APPROACH

Reverse: walk once, flip each pointer back. Keep three names: prev, current, next.

Cycle: a slow pointer moves 1 step, a fast one 2. If there is a loop, fast laps slow and they meet. O(1) space, unlike a set of visited nodes.

Merge: the same two-pointer merge as P6, with a dummy head so the first node is not a special case.

TRAP

The swap line in reverse looks clever but order matters: Python evaluates the right side first, then assigns left to right. If you are unsure, write it as three plain lines. Interviewers prefer clear over clever.

DATA ENGINEERING NOTE

Where linked lists hide in tools you use. collections.deque is built from linked blocks, which is why both ends are O(1). An LRU cache (P16) is a hash map plus a doubly linked list: the map finds a key in O(1), the list moves it to the front in O(1). OrderedDict gives you both, so in Python you rarely build one yourself.

Operation	Python list (array)	Linked list
Read item i	O(1)	O(n): walk from the head
Insert or delete at the front	O(n): shift everything	O(1)
Insert or delete next to a node you hold	O(n)	O(1)
Memory	Compact	Extra pointer per item

PART 4 · DSA AND COMPLEXITY · CHAPTER 23

BINARY SEARCH

23

Cut the search space in half with every look. On sorted data, 10 lakh items need about 20 looks. The harder version searches for the answer itself.

THE ONE TEMPLATE TO MEMORISE — PYTHON 3.12

```
import bisect

def lower_bound(xs, target):    # first i with xs[i] >= target
    lo, hi = 0, len(xs)
    while lo < hi:
        mid = (lo + hi) // 2
        if xs[mid] < target:
            lo = mid + 1                # answer is to the right
        else:
            hi = mid                    # mid could be the answer
    return lo

dates = ["2025-01-10", "2025-02-02", "2025-03-05", "2025-04-20"]
for d in ["2025-02-02", "2025-03-01", "2026-01-01"]:
    print(d, lower_bound(dates, d), bisect.bisect_left(dates, d))
```

OUTPUT

```
2025-02-02 1 1
2025-03-01 2 2
2026-01-01 4 4
```

TRAP

Off-by-one errors are the whole difficulty. Use one template every time: `lo, hi = 0, len(xs)`, loop `while lo < hi`, and either `lo = mid + 1` or `hi = mid`. It returns len(xs) when nothing qualifies, so check before indexing.

BINARY SEARCH ON THE ANSWER: SLOWEST RATE THAT STILL MEETS THE DEADLINE — PYTHON 3.12

```
import math

files_rows = [1_20_000, 45_000, 3_00_000, 80_000, 2_10_000]
HOURS = 8

def hours_needed(rate):                 # files run one at a time
    return sum(math.ceil(rows / rate) for rows in files_rows)

def min_rate(hours):                    # binary search on the answer
    lo, hi = 1, max(files_rows)
    while lo < hi:
        mid = (lo + hi) // 2
        if hours_needed(mid) <= hours:
            hi = mid                    # fast enough: try slower
        else:
            lo = mid + 1                # too slow: go faster
    return lo

r = min_rate(HOURS)
print(r, "rows/hour ->", hours_needed(r), "hours")
print(r - 1, "rows/hour ->", hours_needed(r - 1), "hours")
```

OUTPUT

```
120000 rows/hour -> 8 hours
119999 rows/hour -> 9 hours
```

APPROACH

You are not searching a list; you are searching the range of possible answers. It works because the check is monotonic: if a rate is fast enough, every faster rate is too. Cost: O(n log M), where M is the largest possible rate.

Same family: "Koko eating bananas", "ship packages within D days", "minimum batch size".

Use binary search when	Cost	Example
Data is sorted and you need a position	O(log n)	First order on or after a date
A yes/no check flips once as the answer grows	O(check x log range)	Minimum throughput, minimum capacity
You need many lookups on static sorted data	O(log n) each, no extra memory	Page 43 trade-off vs a set

DE FLAVOUR: WHICH DAY DID THE BAD DATA START? — PYTHON 3.12

```
from datetime import date, timedelta

days = [date(2025, 3, 1) + timedelta(d) for d in range(60)]
checks = 0
def output_is_wrong(day):               # pretend: rerun and compare
    global checks
    checks += 1
    return day >= date(2025, 4, 7)    # the bug shipped on 7 April

lo, hi = 0, len(days)
while lo < hi:
    mid = (lo + hi) // 2
    if output_is_wrong(days[mid]):
        hi = mid
    else:
        lo = mid + 1
print("first bad day:", days[lo], "after", checks, "checks, not 60")
```

OUTPUT

```
first bad day: 2025-04-07 after 6 checks, not 6
  0
```

DATA ENGINEERING NOTE

When a report has been wrong "for a while", binary search over the partitions finds the first bad day in about log2(60), roughly 6 checks. git bisect uses the same idea on commits.

PART 4 · DSA AND COMPLEXITY · CHAPTER 24

SORTING

24

You will almost never write a sort at work, but interviews ask how sorting works, what it costs, and when sorting first makes a problem easy.

MERGE SORT AND COUNTING SORT — PYTHON 3.12

```python
def merge_sort(xs):                    # O(n log n) time, O(n) space
    if len(xs) <= 1:
        return xs
    mid = len(xs) // 2
    left, right = merge_sort(xs[:mid]), merge_sort(xs[mid:])
    out, i, j = [], 0, 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:          # <= keeps it stable
            out.append(left[i]); i += 1
        else:
            out.append(right[j]); j += 1
    return out + left[i:] + right[j:]

def counting_sort(xs, max_val):        # O(n + k) for small integers
    counts = [0] * (max_val + 1)
    for x in xs:
        counts[x] += 1
    return [v for v, c in enumerate(counts) for _ in range(c)]

print(merge_sort([1999, 747, 3498, 796, 1348]))
print(counting_sort([3, 1, 2, 3, 0, 2, 1], 3))
```

OUTPUT

```
[747, 796, 1348, 1999, 3498]
[0, 1, 1, 2, 2, 3, 3]
```

REMEMBER

Merge sort halves the list (log n levels) and merges each level in O(n): O(n log n). The merge needs a new list: O(n) space.

Counting sort beats O(n log n) when values are small integers (ratings 1 to 5, hour of day 0 to 23), because it never compares items.

Algorithm	Time (typical)	Extra space	Stable	Know it for
Timsort (Python sorted, list.sort)	O(n log n), O(n) if nearly sorted	Up to O(n)	Yes	What you actually use
Merge sort	O(n log n) always	O(n)	Yes	External sort of huge files
Quick sort	O(n log n), O(n²) worst	O(log n)	No	In-place sorting, quickselect
Heap sort	O(n log n)	O(1)	No	Heaps (page 55)
Counting / bucket sort	O(n + k)	O(k)	Yes	Small integer keys, top-k by frequency

APPROACH

Sort first, then the problem gets easy. After sorting: duplicates sit next to each other, intervals can be merged left to right (P11), two pointers work (page 45), and the median is the middle item. Ask yourself early: "would this be easy if the data were sorted?"

DATA ENGINEERING NOTE

Data bigger than memory? External merge sort: read chunks that fit in RAM, sort each and write it to disk, then merge the sorted chunks with a heap (heapq.merge). Databases and Spark shuffles do this under the hood.

A CUSTOM COMPARISON: ARRANGE NUMBERS INTO THE LARGEST NUMBER — PYTHON 3.12

```python
from functools import cmp_to_key

def largest_number(nums):
    strs = [str(n) for n in nums]
    def order(a, b):                   # which pair reads bigger?
        return -1 if a + b > b + a else (1 if a + b < b + a else 0)
    strs.sort(key=cmp_to_key(order))
    out = "".join(strs)
    return "0" if out[0] == "0" else out

print(largest_number([3, 30, 34, 5, 9]), largest_number([0, 0]))
```

OUTPUT

```
9534330 0
```

Some orders are easier to write as "is a before b?" than as a key. cmp_to_key turns that comparison into a key. Still O(n log n) comparisons, each costing the length of the strings.

PART 4 · DSA AND COMPLEXITY · CHAPTER 25

RECURSION AND BACKTRACKING

25

Recursion solves a problem by solving a smaller copy of it. Backtracking is recursion that tries a choice, explores it, then undoes it and tries the next.

Every recursive function needs two things: a **base case** that stops (an empty list, a leaf node), and a **step** that makes the problem smaller. Forget the base case and you get a RecursionError; forget to shrink and you get the same thing.

RECURSION ON NESTED DATA — PYTHON 3.12

```
def depth(obj):                  # how deeply nested is it?
    if isinstance(obj, dict) and obj:
        return 1 + max(depth(v) for v in obj.values())
    if isinstance(obj, list) and obj:
        return 1 + max(depth(v) for v in obj)
    return 0                             # base case: a plain value

payload = {"order": {"id": 114,
                     "items": [{"sku": 4, "tags": ["new"]}]}}
print(depth(payload), depth(42), depth({}))
```

OUTPUT

```
5 0 0
```

Time O(n) for n values in the payload; space O(d) for nesting depth d, from the call stack. Flattening JSON (P7) is the same walk.

BACKTRACKING: EVERY SUBSET, EVERY ORDER — PYTHON 3.12

```
def subsets(items):
    out, path = [], []
    def go(i):
        if i == len(items):
            out.append(path.copy())      # one complete choice
            return
        go(i + 1)                        # choice 1: skip items[i]
        path.append(items[i])            # choice 2: take items[i]
        go(i + 1)
        path.pop()                       # undo before returning
    go(0)
    return out

def permutations(items):
    if len(items) <= 1:
        return [items[:]]
    return [[x] + rest for i, x in enumerate(items)
            for rest in permutations(items[:i] + items[i + 1:])]

print(subsets(["city", "status", "month"]))
print(len(permutations(list("abcd"))), permutations([1, 2, 3])[:3])
```

OUTPUT

```
[[], ['month'], ['status'], ['status', 'month'],
  ['city'], ['city', 'month'], ['city', 'status']
  , ['city', 'status', 'month']]
24 [[1, 2, 3], [1, 3, 2], [2, 1, 3]]
```

TRAP

Subsets of n items: 2^n. Orders of n items: n!. At n = 20 that is about 10 lakh subsets but 2.4 quintillion orders. Backtracking is only for small inputs; if n is large, look for a pattern or a greedy rule instead.

DATA ENGINEERING NOTE

Real use: all combinations of grouping columns for a cube, or trying filter combinations in a test. Fine at 5 columns (32 subsets), not at 40.

Recursion is a good fit	Prefer a loop or explicit stack
Data is naturally nested: JSON, trees, folders	Depth could exceed about 1,000 (Python's default limit)
Divide and conquer: merge sort, binary search	A simple running total or scan
Exploring choices: subsets, permutations, paths	Production code on untrusted, deeply nested input

BACKTRACKING WITH PRUNING — PYTHON 3.12

```
def valid_brackets(n):
    out = []
    def go(s, opened, closed):
        if len(s) == 2 * n:
            out.append(s); return
        if opened < n:
            go(s + "(", opened + 1, closed)
        if closed < opened:          # prune: never close early
            go(s + ")", opened, closed + 1)
    go("", 0, 0)
    return out

print(valid_brackets(3))
```

OUTPUT

```
['((()))', '(()())', '(())()', '()(())', '()()(
  )']
```

Pruning cuts branches that can never lead to a valid answer, so the search explores far fewer than the 2^(2n) raw strings. Good backtracking is mostly good pruning.

PART 4 · DSA AND COMPLEXITY · CHAPTER 26

TREES: DFS AND BFS

26

A tree is a hierarchy: one root, and every other node has exactly one parent. Org charts, folders, JSON and category menus are all trees.

BUILD THE ORG CHART FROM EMPLOYEES.CSV, THEN WALK IT TWO WAYS — PYTHON 3.12

```python
import csv
from collections import defaultdict, deque

emps = list(csv.DictReader(open("employees.csv")))
name = {e["emp_id"]: e["name"] for e in emps}
children = defaultdict(list)
root = None
for e in emps:
    if e["manager_id"]:
        children[e["manager_id"]].append(e["emp_id"])
    else:
        root = e["emp_id"]

def dfs(node, depth=0):                # depth-first: go deep, then back
    print("  " * depth + name[node])
    for c in children[node]:
        dfs(c, depth + 1)

def bfs(start):                        # breadth-first: level by level
    levels, q = [], deque([(start, 0)])
    while q:
        node, d = q.popleft()
        if d == len(levels):
            levels.append([])
        levels[d].append(name[node])
        q.extend((c, d + 1) for c in children[node])
    return levels

dfs(root)
print(bfs(root))
```

OUTPUT

```
Anita
  Rahul
    Priya
    Karan
  Neha
    Arjun
    Ishita
  Dev
    Tara
[['Anita'], ['Rahul', 'Neha', 'Dev'], ['Priya',
   'Karan', 'Arjun', 'Ishita', 'Tara']]
```

APPROACH

DFS (a stack, or recursion) follows one branch to the bottom first: good for paths, depth and "everything under this manager".

BFS (a queue) visits level by level: good for "how many levels", "nearest" and shortest paths when every step costs the same.

Both visit each node once: O(n) time. DFS uses O(height) space, BFS O(width).

BINARY SEARCH TREE: WHY BALANCE MATTERS — PYTHON 3.12

```python
from dataclasses import dataclass

@dataclass
class T:
    val: int
    left: "T | None" = None
    right: "T | None" = None

def insert(node, v):                   # binary search tree
    if node is None:
        return T(v)
    if v < node.val:
        node.left = insert(node.left, v)
    else:
        node.right = insert(node.right, v)
    return node

def inorder(n):                        # left, root, right = sorted
    return inorder(n.left) + [n.val] + inorder(n.right) if n else []

def height(n):
    return 1 + max(height(n.left), height(n.right)) if n else 0

root = None
for v in [50, 30, 70, 20, 40, 60, 80]:
    root = insert(root, v)
chain = None
for v in [10, 20, 30, 40, 50, 60, 70]:  # sorted: a line
    chain = insert(chain, v)
print(inorder(root), height(root), height(chain))
```

OUTPUT

```
[20, 30, 40, 50, 60, 70, 80] 3 7
```

TRAP

A balanced BST finds a value in O(log n): height 3 for 7 nodes. Insert sorted data and every node goes right: height 7, and search becomes O(n). That is why databases use self-balancing B-trees for indexes.

Traversal	Order it visits	Use it for
Pre-order / post-order (DFS)	Node first / children first	Print a hierarchy / totals that need children first (team headcount)
In-order / level-order (BFS)	Sorted BST values / level by level	Read a BST in order / depth of the org chart, nearest match

PART 4 · DSA AND COMPLEXITY · CHAPTER 27

GRAPHS: THE DAG BEHIND EVERY PIPELINE

27

A graph is things connected to things. An Airflow DAG is a graph, and scheduling its tasks in a valid order is a textbook algorithm.

RUN ORDER FOR A PIPELINE DAG, AND CATCHING A CYCLE — PYTHON 3.12

```python
from collections import defaultdict, deque

edges = [("extract_orders", "clean_orders"),
         ("extract_customers", "dim_customer"),
         ("clean_orders", "fact_orders"),
         ("dim_customer", "fact_orders"),
         ("fact_orders", "daily_report"),
         ("dim_customer", "daily_report")]

def topo_order(edges):                          # Kahn's algorithm
    graph, indegree = defaultdict(list), defaultdict(int)
    for a, b in edges:
        graph[a].append(b)
        indegree[b] += 1
        indegree.setdefault(a, 0)
    ready = deque(sorted(n for n, d in indegree.items() if d == 0))
    order = []
    while ready:
        n = ready.popleft()
        order.append(n)
        for nxt in graph[n]:
            indegree[nxt] -= 1
            if indegree[nxt] == 0:              # all its inputs are done
                ready.append(nxt)
    if len(order) < len(indegree):
        raise ValueError("cycle: some tasks wait on each other")
    return order

print(topo_order(edges))
try:
    topo_order(edges + [("daily_report", "extract_orders")])
except ValueError as e:
    print(e)
```

OUTPUT

```
['extract_customers', 'extract_orders', 'dim_
  customer', 'clean_orders', 'fact_orders', '
  daily_report']
cycle: some tasks wait on each other
```

APPROACH

Count each task's unfinished inputs (indegree). Start with tasks that have none, and every time a task finishes, lower its children's counts. If some tasks never reach zero, there is a cycle. O(V + E) for V tasks and E dependencies.

DATA ENGINEERING NOTE

This is what Airflow, dbt and Spark's scheduler do before running anything, and why they refuse a DAG with a cycle.

BFS ON A GRID: COUNT CONNECTED GROUPS — PYTHON 3.12

```python
from collections import deque

grid = ["11000",
        "11010",
        "00011",
        "00000"]

def islands(grid):
    rows, cols, seen, count = len(grid), len(grid[0]), set(), 0
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == "1" and (r, c) not in seen:
                count += 1                      # new island: flood it
                q = deque([(r, c)]); seen.add((r, c))
                while q:
                    y, x = q.popleft()
                    for dy, dx in ((1, 0), (-1, 0), (0, 1), (0, -1)):
                        ny, nx = y + dy, x + dx
                        inside = 0 <= ny < rows and 0 <= nx < cols
                        if (inside and grid[ny][nx] == "1"
                                and (ny, nx) not in seen):
                            seen.add((ny, nx)); q.append((ny, nx))
    return count

print(islands(grid))
```

OUTPUT

```
2
```

"Number of islands" is the most asked graph question. A grid is a graph where each cell links to its four neighbours. Every cell is visited once: O(rows x cols).

Mark a cell as seen when you add it to the queue, not when you pop it, or the same cell can be queued many times.

The question asks	Reach for	Cost
Run order of dependent tasks	Topological sort (Kahn)	O(V + E)
Is there a cycle in the dependencies?	Kahn leftover count, or DFS with "visiting" marks	O(V + E)
Fewest steps between two nodes, equal step cost	BFS	O(V + E)
Cheapest path with different costs	Dijkstra's algorithm with a heap	O(E log V)
Which items are connected (groups, islands)	BFS / DFS flood fill, or union-find	O(V + E)

PART 4 · DSA AND COMPLEXITY · CHAPTER 28

HEAPS AND GREEDY

28

A heap always hands you the smallest item in O(log n). A greedy algorithm makes the locally best choice at each step. Together they solve most scheduling questions.

MINIMUM ROOMS, AND MOST NON-OVERLAPPING JOBS — PYTHON 3.12

```python
import heapq

bookings = [("05-01", "05-04"), ("05-03", "05-06"), ("05-06", "05-08"),
            ("05-02", "05-05"), ("05-04", "05-07")]

def rooms_needed(stays):
    ends = []                                   # min-heap of check-outs
    for start, end in sorted(stays):
        if ends and ends[0] <= start:           # a room is free again
            heapq.heapreplace(ends, end)
        else:
            heapq.heappush(ends, end)           # open another room
    return len(ends)

def max_jobs(windows):              # greedy: earliest finish
    count, last_end = 0, ""
    for start, end in sorted(windows, key=lambda w: w[1]):
        if start >= last_end:
            count, last_end = count + 1, end
    return count

print(rooms_needed(bookings), max_jobs(bookings))
```

OUTPUT

```
3 2
```

APPROACH

Rooms: the heap top is the earliest check-out. If it is before the next check-in, reuse that room; else open a new one. O(n log n).

Greedy jobs: always pick the job that finishes first. It leaves the most time for everything else, and that choice is provably optimal here.

RUNNING MEDIAN OF A STREAM WITH TWO HEAPS — PYTHON 3.12

```python
import heapq

class RunningMedian:                        # two heaps
    def __init__(self):
        self.low, self.high = [], []       # max-heap (negated), min-heap
    def add(self, x):
        heapq.heappush(self.low, -x)
        heapq.heappush(self.high, -heapq.heappop(self.low))
        if len(self.high) > len(self.low):
            heapq.heappush(self.low, -heapq.heappop(self.high))
    def median(self):
        if len(self.low) > len(self.high):
            return -self.low[0]
        return (-self.low[0] + self.high[0]) / 2

rm = RunningMedian()
for latency in [120, 340, 95, 410, 180, 150]:
    rm.add(latency)
    print(latency, "->", rm.median())
```

OUTPUT

```
120 -> 120
340 -> 230.0
95 -> 120
410 -> 230.0
180 -> 180
150 -> 165.0
```

REMEMBER

The lower half lives in a max-heap, the upper half in a min-heap, kept within one item of each other. The median is always at the tops. O(log n) per new value, O(1) to read the median. Sorting the stream after every value would be O(n log n) each time.

TRAP

Greedy is not always right. Coins [1, 3, 4] for 6: greedy takes 4 + 1 + 1 = 3 coins, the best is 3 + 3 = 2 coins. Page 56 solves it properly.

Heap recipe	Keep in the heap	Cost
Top k largest	The k largest so far, smallest on top	O(n log k)
k-th largest	Same; the answer is the top	O(n log k)
Merge k sorted files	One head item per file	O(n log k)
Rooms / machines needed	End times of busy rooms	O(n log n)
Running median	Lower half (max-heap) and upper half (min-heap)	O(log n) per item
Next job to run by priority	(priority, job) pairs	O(log n) per push or pop

PART 4 · DSA AND COMPLEXITY · CHAPTER 29

DYNAMIC PROGRAMMING BASICS

29

Dynamic programming means: remember the answers to smaller problems so you never solve the same one twice. Rare in DE loops, but these four come up.

THE SAME ANSWER, THREE COSTS — PYTHON 3.12

```python
from functools import lru_cache

calls = 0
def fib_slow(n):
    global calls; calls += 1
    return n if n < 2 else fib_slow(n - 1) + fib_slow(n - 2)

@lru_cache(maxsize=None)
def fib_memo(n):                        # top-down: recursion + cache
    return n if n < 2 else fib_memo(n - 1) + fib_memo(n - 2)

def fib_table(n):                       # bottom-up: O(1) space
    a, b = 0, 1
    for _ in range(n):
        a, b = b, a + b
    return a

print(fib_slow(25), "slow calls:", calls)
print(fib_memo(25), fib_table(25), fib_table(90))
```

OUTPUT

```
75025 slow calls: 242785
75025 75025 2880067194370816120
```

APPROACH

The slow version recomputes the same values again and again: about 2^n calls. Caching makes it O(n). The table version keeps only the last two values: O(n) time, O(1) space.

COIN CHANGE, MAX SUBARRAY (KADANE), CLIMBING STAIRS — PYTHON 3.12

```python
def min_coins(coins, amount):
    INF = float("inf")
    best = [0] + [INF] * amount      # fewest coins for a
    for a in range(1, amount + 1):
        for c in coins:
            if c <= a and best[a - c] + 1 < best[a]:
                best[a] = best[a - c] + 1
    return best[amount] if best[amount] != INF else -1

def max_subarray(xs):                # Kadane
    best = cur = xs[0]
    for x in xs[1:]:
        cur = max(x, cur + x)            # extend, or start fresh
        best = max(best, cur)
    return best

def climb(n):                            # ways to climb 1 or 2 steps
    a, b = 1, 1
    for _ in range(n):
        a, b = b, a + b
    return a

print(min_coins([1, 3, 4], 6), min_coins([1, 2, 5, 10, 20, 50], 88))
print(max_subarray([5, -2, -8, 3, 4, -1, 6, -9]))
print(climb(10))
```

OUTPUT

```
2 6
12
89
```

REMEMBER

Coin change: best[a] is built from best[a - coin] for each coin: O(amount x coins) time, O(amount) space. It gets 2 coins for 6, where greedy gets 3.

Kadane: at each day, either extend the current run or start a new one. One pass, O(1) space. The best run here is 3 + 4 - 1 + 6 = 12.

Ask yourself	If yes
Does the answer for n depend on answers for smaller n?	Write that relationship first, in words
Does brute force solve the same sub-problem many times?	Add a cache (top-down) or fill a table (bottom-up)
Does each step only look back one or two steps?	Keep two variables instead of a whole table: O(1) space

PART 4 · DSA AND COMPLEXITY · PRACTICE

PRACTICE SET 3: DSA

P3

Every answer ends with its time and space cost. Saying those two out loud is half the score in a DSA round.

Q16. Order ids should be 1 to n with exactly one missing. Find it. DE EASY

Approach: The expected total is n(n+1)/2; subtract what is there. O(n) time, O(1) space.

```
def missing(ids):
    n = len(ids) + 1
    return n * (n + 1) // 2 - sum(ids)

print(missing([1, 2, 4, 5, 6]), missing([2, 3, 1, 5]))
```

OUTPUT
```
3 4
```

Q17. Find the k-th largest order amount without sorting the whole list. DE MEDIUM

Approach: Keep a min-heap of size k; its top is the answer. O(n log k) time, O(k) space.

```
import csv, heapq
amounts = [float(r["amount"])
           for r in csv.DictReader(open("orders.csv"))]
def kth_largest(xs, k):
    heap = xs[:k]
    heapq.heapify(heap)
    for x in xs[k:]:
        if x > heap[0]:
            heapq.heapreplace(heap, x)
    return heap[0]
print(kth_largest(amounts, 3), sorted(amounts, reverse=True)[2])
```

OUTPUT
```
1999.0 1999.0
```

Q18. Return the longest common prefix of these S3 paths. DE EASY

Approach: Compare characters column by column until one path differs. O(total characters) time, O(1) extra.

```
paths = ["s3://raw/orders/2025/04/01/part-0.csv",
         "s3://raw/orders/2025/04/02/part-0.csv",
         "s3://raw/orders/2025/03/31/part-1.csv"]
def common_prefix(strs):
    first, last = min(strs), max(strs)   # compare only these
    i = 0
    while i < len(first) and first[i] == last[i]:
        i += 1
    return first[:i]
print(common_prefix(paths))
```

OUTPUT
```
s3://raw/orders/2025/0
```

Q19. For each day, the product of every other day's growth factor, without division. DE MEDIUM

Approach: Prefix products from the left, suffix products from the right, multiplied. O(n) time, O(1) extra besides the output.

```
def product_except_self(xs):
    out = [1] * len(xs)
    left = 1
    for i in range(len(xs)):
        out[i] = left; left *= xs[i]
    right = 1
    for i in reversed(range(len(xs))):
        out[i] *= right; right *= xs[i]
    return out

print(product_except_self([2, 3, 4, 5]))
```

OUTPUT
```
[60, 40, 30, 24]
```

Q20. Find files in a folder that have exactly the same content, even under different names. DE MEDIUM

Approach: Group by a hash of the bytes (P3 with a different key). O(total bytes) time, O(number of files) space.

```
import hashlib, pathlib, shutil
from collections import defaultdict

shutil.copy("orders.csv", "backup.csv")    # plant a duplicate
groups = defaultdict(list)
for p in sorted(pathlib.Path(".").glob("*.csv")):
    digest = hashlib.sha256(p.read_bytes()).hexdigest()
    groups[digest].append(p.name)
print([names for names in groups.values() if len(names) > 1])
```

OUTPUT
```
[['backup.csv', 'orders.csv']]
```

PART 5 · CODING ROUND PATTERNS

THE PATTERN MAP

18

Most DE Python coding questions are one of these 18 shapes, or two of them stacked. Learn to hear the shape inside the question.

#	Pattern	The question sounds like	Core tool	Pg
P1	**Frequency count**	*"how many of each", "most common error"*	`Counter, dict.get(k, 0) + 1`	59
P2	**Top-K**	*"top 3 customers", "k largest files"*	`heapq.nlargest, size-k min-heap`	59
P3	**Group by key**	*"totals per order", "items per user"*	`defaultdict(list)`	60
P4	**Dedupe, keep latest**	*"remove duplicates", "latest record per id"*	`dict keyed by id + timestamp test`	60
P5	**Sliding window**	*"rolling 3-day sum", "moving average"*	`deque, running total`	61
P6	**Merge sorted streams**	*"merge two sorted logs", "combine feeds"*	`heapq.merge, two pointers`	61
P7	**Flatten nested JSON**	*"flatten this payload", "explode items"*	`recursion, json_normalize`	62
P8	**Sessionize events**	*"split into sessions", "30 min idle"*	`sort + gap test`	63
P9	**Consecutive streaks**	*"longest login streak", "consecutive days"*	`set + start-of-run test`	63
P10	**Parse logs**	*"count errors per job from this log"*	`re named groups + dict`	64
P11	**Merge intervals**	*"overlapping bookings", "busy periods"*	`sort by start, extend last`	65
P12	**Hash join**	*"join these two files without pandas"*	`dict on the small side`	65
P13	**Chunked processing**	*"the file is 50 GB", "batch inserts"*	`generator + islice`	66
P14	**Retry with backoff**	*"the API returns 429", "flaky warehouse"*	`exponential delay + jitter`	66
P15	**Paginated extraction**	*"pull all pages from this API"*	`cursor loop + generator`	67
P16	**Memoize / LRU cache**	*"avoid repeat lookups", "build an LRU"*	`lru_cache, OrderedDict`	67
P17	**Validate and quarantine**	*"reject bad rows", "data contract"*	`per-row rules, two outputs`	68
P18	**Diff two snapshots**	*"what changed since yesterday", "CDC"*	`set algebra on keys`	68

APPROACH

Hearing the pattern in 10 seconds

"each / per" means group by key. "top / most" means Counter or a heap. "latest / duplicate" means a dict keyed by id. "consecutive / streak / session" means sort, then compare neighbours. "big file / API / flaky" means generators, batches and retries.

INTERVIEW TIP

Stacked patterns are the HARD questions. "Top 3 users by session count" is P8 then P2. "Daily error counts from a 20 GB log" is P13 then P10 then P1. Solve the inner pattern as its own function, test it, then wrap the next one around it.

One skeleton covers half the list

THE ONE-PASS DICT SKELETON — ANY PYTHON

```
result = {}                                # or defaultdict(...) / Counter()
for record in stream:                      # one pass over the input: O(n)
    key = key_of(record)                   # what defines "the same thing"?
    result[key] = combine(result.get(key), record)   # count, sum, append, keep latest
answer = finish(result)                    # sort, take top k, format
```

REMEMBER

How to practise a pattern so it sticks. Read the worked question, close the page, and write the solution from memory on the store files. Then solve one new question with the same pattern on a different file: P9 on order dates instead of logins, P2 on products instead of customers. If it works on a file you have not seen it on, you own the pattern.

P1 FREQUENCY COUNT

DE EASY

Sounds like: *"how many of each status", "most common error message", "count events per page"*

Worked question: Count orders per status and per (customer, status), then the two most visited pages from the event stream.

Approach: **1.** pick the key, a tuple if it is two columns **2.** Counter over a generator **3.** most_common for the top

SOLUTION — PYTHON 3.12

```python
import csv, json
from collections import Counter

orders = list(csv.DictReader(open("orders.csv")))
by_status = Counter(r["status"] for r in orders)
pairs = Counter((r["customer_id"], r["status"]) for r in orders)
print(by_status)
print(by_status.most_common(1), pairs[("1", "delivered")])

with open("events.jsonl") as f:
    pages = Counter(json.loads(line)["props"]["page"] for line in f)
print(pages.most_common(2))

counts = {}                                  # by hand
for r in orders:
    counts[r["status"]] = counts.get(r["status"], 0) + 1
print(counts == dict(by_status))
```

OUTPUT

```
Counter({'delivered': 12, 'cancelled': 2, 'retu
  rned': 1})
[('delivered', 12)] 3
[('/cart', 3), ('/checkout', 2)]
True
```

INTERVIEW TIP

Be ready to write it without Counter: dict.get(key, 0) + 1 is the same O(n) pass. The last line proves the two agree, which is a nice thing to show live.

P2 TOP-K

DE MEDIUM

Sounds like: *"top 3 customers by spend", "10 largest files", "k most frequent words"*

Worked question: The top 3 customers by delivered spend, first with the library, then with a heap you write yourself.

Approach: **1.** aggregate first (P1/P3) **2.** keep only k items in a min-heap **3.** sort the k survivors at the end

SOLUTION — PYTHON 3.12

```python
import csv, heapq
from collections import defaultdict

spend = defaultdict(float)
for r in csv.DictReader(open("orders.csv")):
    if r["status"] == "delivered":
        spend[r["customer_id"]] += float(r["amount"])

print(heapq.nlargest(3, spend.items(), key=lambda kv: kv[1]))

def top_k(pairs, k):                 # size-k min-heap: O(n log k)
    heap = []
    for cid, amt in pairs:
        if len(heap) < k:
            heapq.heappush(heap, (amt, cid))
        elif amt > heap[0][0]:       # beats the smallest kept
            heapq.heapreplace(heap, (amt, cid))
    return sorted(heap, reverse=True)

print(top_k(spend.items(), 3))
```

OUTPUT

```
[('5', 5096.0), ('1', 5094.0), ('2', 4143.0)]
[(5096.0, '5'), (5094.0, '1'), (4143.0, '2')]
```

APPROACH

Sorting everything costs O(n log n). A min-heap of size k costs O(n log k) and only k items of memory, which is why it is the answer for "top 10 of a billion". The heap top is the smallest of the best k, so one comparison decides each new item.

Follow-ups interviewers ask

They ask	You answer
"Count words in a 30 GB log."	Stream lines (P13), Counter per batch, then add the batch Counters: Counter supports +. Or split the file across processes and merge.
"Top 10 with ties at the cut-off?"	nlargest drops ties at the boundary. Find the 10th value, then keep everything at or above it.
"The data does not fit in memory."	Keep a size-k heap while streaming (above): memory is O(k) no matter how big the input is.

P3 GROUP BY KEY

DE EASY

Sounds like: *"total per order", "list of items per user", "reconcile header vs lines"*

Worked question: Group order_items by order, total each basket, and check every total against orders.amount.

Approach: **1.** defaultdict(list) to group **2.** aggregate each group **3.** compare with the other source

SOLUTION — PYTHON 3.12

```python
import csv
from collections import defaultdict
from decimal import Decimal

lines = defaultdict(list)
for r in csv.DictReader(open("order_items.csv")):
    lines[r["order_id"]].append(r)

totals = {oid: sum(Decimal(r["qty"]) * Decimal(r["unit_price"])
                   for r in rows)
          for oid, rows in lines.items()}
orders = {r["order_id"]: Decimal(r["amount"])
          for r in csv.DictReader(open("orders.csv"))}

bad = [o for o in orders if totals.get(o) != orders[o]]
print(len(orders), "orders checked, mismatches:", bad)
biggest = max(lines, key=lambda o: len(lines[o]))
print("biggest basket:", biggest, len(lines[biggest]), "items")
```

OUTPUT

```
15 orders checked, mismatches: []
biggest basket: 114 3 items
```

DATA ENGINEERING NOTE

This is the header-vs-lines reconciliation every finance pipeline runs. Decimal compares by value, so 1198 equals 1198.00, while floats could disagree on the last digit.

P4 DEDUPE, KEEP LATEST

DE MEDIUM

Sounds like: *"remove duplicates", "latest version of each record", "the CDC feed has repeats"*

Worked question: A change feed repeats orders, and one older record arrives after a newer one. Keep the latest version per order.

Approach: **1.** the key is order_id **2.** the winner is the newest ts, not the last arrival **3.** one dict, one pass

SOLUTION — PYTHON 3.12

```python
feed = [
  {"order_id": 101, "status": "placed",    "ts": "2025-01-10 09:00"},
  {"order_id": 101, "status": "delivered", "ts": "2025-01-12 18:30"},
  {"order_id": 102, "status": "placed",    "ts": "2025-01-15 11:00"},
  {"order_id": 103, "status": "delivered", "ts": "2025-02-04 20:00"},
  {"order_id": 103, "status": "placed",    "ts": "2025-02-02 08:15"},
]                                      # last record arrived late

latest = {}
for rec in feed:
    cur = latest.get(rec["order_id"])
    if cur is None or rec["ts"] > cur["ts"]:
        latest[rec["order_id"]] = rec
for rec in latest.values():
    print(rec["order_id"], rec["status"], rec["ts"])

print(list(dict.fromkeys([3, 1, 3, 2, 1])))   # keep first, keep order
```

OUTPUT

```
101 delivered 2025-01-12 18:30
102 placed 2025-01-15 11:00
103 delivered 2025-02-04 20:00
[3, 1, 2]
```

TRAP

"Last one wins" by arrival order would mark order 103 as placed again. Compare timestamps. ISO strings compare correctly as text; other formats need parsing first.

Follow-ups interviewers ask

They ask	You answer
"Group by two columns."	Use a tuple key, groups[(customer_id, month)]. Same one pass.
"Keep the latest, but ties on ts?"	Add a tiebreaker to the comparison, such as a sequence number or file offset, so the winner is repeatable.
"Dedupe 500 million ids."	A set of 500 million ints is tens of GB. Partition ids by hash into files, dedupe each file, or push it to Spark or SQL.

P5 SLIDING WINDOW

DE DA MEDIUM

Sounds like: *"rolling 3-day revenue", "moving average of latency", "max sum of k consecutive"*

Worked question: A rolling 3-day revenue total over daily delivered revenue for 1 to 7 April (zeros already filled in).

Approach: **1.** add the new value **2.** subtract the one leaving **3.** O(1) per step instead of re-summing k values

SOLUTION — PYTHON 3.12

```python
from collections import deque

daily = [("04-01", 0), ("04-02", 1999), ("04-03", 0), ("04-04", 0),
         ("04-05", 846), ("04-06", 0), ("04-07", 0)]

window, total = deque(), 0
for day, amt in daily:
    window.append(amt)
    total += amt
    if len(window) > 3:            # drop the day leaving the window
        total -= window.popleft()
    print(day, f"{amt:>5}", f"3-day: {total:>5}")
```

OUTPUT

```
04-01     0 3-day:     0
04-02  1999 3-day:  1999
04-03     0 3-day:  1999
04-04     0 3-day:  1999
04-05   846 3-day:   846
04-06     0 3-day:   846
04-07     0 3-day:   846
```

TRAP

Re-summing the last k values every step is O(n x k). The running total is O(n). The input must have one row per day, zeros included, or "last 3 rows" stops meaning "last 3 days". Fill gaps first.

P6 MERGE SORTED STREAMS

DE MEDIUM

Sounds like: *"merge two sorted log files", "combine web and app events in time order"*

Worked question: Merge web and app event streams, each already sorted by time, into one ordered stream. Then write the merge by hand.

Approach: **1.** two pointers, one per input **2.** take the smaller head **3.** append whatever is left

SOLUTION — PYTHON 3.12

```python
import heapq

web = [("10:00", "u1 view"), ("10:20", "u1 checkout"),
       ("11:00", "u3 view")]
app = [("10:05", "u1 cart"), ("10:22", "u1 purchase")]
print([e for _, e in heapq.merge(web, app)])     # lazy, O(n)

def merge_two(a, b):
    i = j = 0
    out = []
    while i < len(a) and j < len(b):
        if a[i] <= b[j]:
            out.append(a[i]); i += 1
        else:
            out.append(b[j]); j += 1
    return out + a[i:] + b[j:]          # one side has leftovers

print(merge_two([1, 4, 9], [2, 3, 10, 11]))
```

OUTPUT

```
['u1 view', 'u1 cart', 'u1 checkout', 'u1 purch
  ase', 'u3 view']
[1, 2, 3, 4, 9, 10, 11]
```

APPROACH

Concatenating and sorting costs O(n log n) and needs everything in memory. Merging pre-sorted inputs is O(n) and streams: heapq.merge takes any number of sorted iterables, even files read line by line.

Follow-ups interviewers ask

They ask	You answer
"Moving average, not sum."	Divide the running total by len(window). Same O(1) update.
"Max of the last k values."	A running total does not work for max. Use a deque of indices kept in decreasing order (monotonic queue): still O(n).
"Merge 200 sorted files."	heapq.merge on 200 open file iterators: memory holds one line per file, not the files.

P7 FLATTEN NESTED JSON

DE HARD

Sounds like: *"flatten this API payload", "turn nested JSON into table rows", "explode the items array"*

Worked question: Flatten customer records with a nested address and a tags list into flat columns, then explode an order's items into one row each.

Approach: **1.** recurse into dicts and lists **2.** build dotted keys **3.** decide: lists become columns, or rows?

RECURSIVE FLATTEN — PYTHON 3.12

```
import json

def flatten(obj, parent="", sep="."):
    items = {}
    if isinstance(obj, dict) and obj:
        for k, v in obj.items():
            key = f"{parent}{sep}{k}" if parent else k
            items.update(flatten(v, key, sep))
    elif isinstance(obj, list) and obj:
        for i, v in enumerate(obj):
            key = f"{parent}{sep}{i}" if parent else str(i)
            items.update(flatten(v, key, sep))
    else:
        items[parent] = obj        # leaf, or empty dict/list
    return items

customers = json.load(open("customers.json"))
print(flatten(customers[1]))
print(flatten(customers[5]))
```

OUTPUT

```
{'customer_id': 2, 'name': 'Diya', 'signup_date':
  '2025-01-12', 'referred_by': 1, 'address.city':
  'Mumbai', 'address.pincode': '400001', 'tags.0':
   'referral'}
{'customer_id': 6, 'name': 'Sana', 'signup_date':
  '2025-03-15', 'referred_by': None, 'address.city
  ': None, 'address.pincode': None, 'tags': []}
```

REMEMBER

Diya's tags list becomes a tags.0 column. Sana has no city, so address.city is None, and her empty tags list survives as one column holding [], instead of disappearing.

EXPLODE A LIST INTO ROWS — PYTHON 3.12

```
order = {"order_id": 114,
         "customer": {"id": 1, "city": "Bengaluru"},
         "items": [{"sku": 4, "qty": 1}, {"sku": 1, "qty": 1},
                   {"sku": 5, "qty": 1}]}

def explode(record, list_key):
    parent = {k: v for k, v in record.items() if k != list_key}
    base = flatten(parent)
    for child in record.get(list_key) or [{}]:   # keep empty
        kids = {f"{list_key}.{k}": v for k, v in child.items()}
        yield {**base, **kids}

for row in explode(order, "items"):
    print(row)
```

OUTPUT

```
{'order_id': 114, 'customer.id': 1, 'customer.city'
  : 'Bengaluru', 'items.sku': 4, 'items.qty': 1}
{'order_id': 114, 'customer.id': 1, 'customer.city'
  : 'Bengaluru', 'items.sku': 1, 'items.qty': 1}
{'order_id': 114, 'customer.id': 1, 'customer.city'
  : 'Bengaluru', 'items.sku': 5, 'items.qty': 1}
```

TRAP

Flattening a list of records into columns (items.0.sku, items.1.sku…) gives a different number of columns per row, which no table can hold. Lists of records become rows (explode); lists of simple tags can become columns or a joined string.

THE PANDAS SHORTCUT — PYTHON 3.12

```
import json
import pandas as pd

events = [json.loads(l) for l in open("events.jsonl")]
df = pd.json_normalize(events)
print(df.columns.tolist())
print(df.head(3).to_string(index=False))
```

OUTPUT

```
['user_id', 'ts', 'event', 'props.page', 'props.device']
 user_id                  ts    event props.page props.device
       1 2025-04-01T10:00:00     view       /p/4       mobile
       1 2025-04-01T10:05:00     cart      /cart       mobile
       1 2025-04-01T10:20:00 checkout  /checkout       mobile
```

`pd.json_normalize` does the dict part for you and `record_path` does the explode. In the interview, write the recursive version first: that is what they are testing.

P8 SESSIONIZE EVENTS

DE DA HARD

Sounds like: *"split clickstream into sessions", "a session ends after 30 minutes idle", "average session length"*

Worked question: Group each user's events into sessions, where a gap of more than 30 minutes starts a new session.

Approach: **1.** group by user **2.** sort by time **3.** new session when the gap exceeds 30 minutes

SOLUTION — PYTHON 3.12

```python
import json
from collections import defaultdict
from datetime import datetime, timedelta

GAP = timedelta(minutes=30)
by_user = defaultdict(list)
for line in open("events.jsonl"):
    e = json.loads(line)
    ts = datetime.fromisoformat(e["ts"])
    by_user[e["user_id"]].append((ts, e["event"]))

for user, events in sorted(by_user.items()):
    events.sort()                          # never trust input order
    sessions, prev = [], None
    for ts, ev in events:
        if prev is None or ts - prev > GAP:
            sessions.append([])            # start a new session
        sessions[-1].append(ev)
        prev = ts
    print(user, sessions)
```

OUTPUT

```
1 [['view', 'cart', 'checkout', 'purchase'], ['
  view']]
2 [['view', 'view'], ['cart']]
3 [['view', 'cart', 'checkout']]
4 [['view']]
```

REMEMBER

User 1 comes back at 14:00 and user 2 waits 38 minutes before adding to cart, so both get a second session. Same logic as the SQL version (LAG + running sum), written as a loop.

P9 CONSECUTIVE STREAKS

DE DA MEDIUM

Sounds like: *"longest login streak", "most consecutive days with sales", "longest consecutive sequence"*

Worked question: Each user's longest run of consecutive login days.

Approach: **1.** put dates in a set **2.** only count from a day whose previous day is missing **3.** walk forward while days exist

SOLUTION — PYTHON 3.12

```python
import csv
from collections import defaultdict
from datetime import date, timedelta

ONE = timedelta(days=1)
days = defaultdict(set)
for r in csv.DictReader(open("logins.csv")):
    days[int(r["user_id"])].add(date.fromisoformat(r["login_date"]))

for user, ds in sorted(days.items()):
    best = (0, None)
    for d in ds:
        if d - ONE not in ds:              # d starts a streak
            n = 1
            while d + n * ONE in ds:
                n += 1
            best = max(best, (n, d))
    print(user, "longest:", best[0], "days from", best[1])
```

OUTPUT

```
1 longest: 3 days from 2025-04-01
2 longest: 2 days from 2025-04-01
3 longest: 4 days from 2025-04-03
```

APPROACH

Checking "is yesterday missing?" means each streak is walked once, so the whole thing is O(n). This is the Python twin of the SQL "date minus row number" trick, and the classic "longest consecutive sequence" question.

Follow-ups interviewers ask

They ask	You answer
"Average session length."	Keep first and last timestamp per session and average the differences. A one-event session has length 0: say whether to exclude it.
"Streaks with gaps of one day allowed."	Change the test: a day starts a streak only if neither of the two days before it exists.
"Events arrive out of order in a stream."	Sort within a time window before sessionizing, and accept late events up to a cut-off (a watermark in Spark).

P10 PARSE LOGS

DE HARD

Sounds like: *"from this log, report errors per job", "which step is slowest", "parse these lines into records"*

Worked question: Parse app.log into records, skip anything malformed, then report per job: lines, errors, slowest step and the most common error.

Approach: **1.** write one regex with named groups **2.** match line by line, keep skipped lines **3.** aggregate with dicts and Counter

STEP 1: LINES TO RECORDS — PYTHON 3.12

```python
import re

LINE = re.compile(
    r"(?P<ts>\d{4}-\d{2}-\d{2} \d{2}:\d{2}:\d{2})\s+"
    r"(?P<level>INFO|WARN|ERROR)\s+"
    r"job=(?P<job>\w+)\s+step=(?P<step>\w+)"
    r"(?:\s+rows=(?P<rows>\d+))?\s+duration_ms=(?P<ms>\d+)"
    r'(?:\s+msg="(?P<msg>[^"]*)")?'
)

parsed, skipped = [], []
with open("app.log") as f:
    for n, line in enumerate(f, start=1):
        m = LINE.match(line)
        if m:
            parsed.append(m.groupdict())
        else:
            skipped.append((n, line.strip()))

print(len(parsed), "parsed; skipped:", skipped)
print(parsed[4])
```

OUTPUT

```
13 parsed; skipped: [(7, '--- scheduler restarte
  d ---')]
{'ts': '2025-04-01 02:05:02', 'level': 'ERROR',
  'job': 'customers_dim', 'step': 'load', 'rows'
  : None, 'ms': '5003', 'msg': 'timeout connecti
  ng to warehouse'}
```

REMEMBER

Named groups (`(?P<job>...)`) give you a dict per line through `groupdict()`. Optional parts such as rows= and msg= sit inside `(?:...)?`, so they come back as None when missing.

The "scheduler restarted" line does not match and is reported, not silently dropped.

STEP 2: AGGREGATE — PYTHON 3.12

```python
from collections import Counter, defaultdict
# parsed comes from step 1

report = defaultdict(lambda: {"lines": 0, "errors": 0,
                              "slowest": ("", 0)})
for p in parsed:
    r = report[p["job"]]
    r["lines"] += 1
    r["errors"] += p["level"] == "ERROR"
    ms = int(p["ms"])
    if ms > r["slowest"][1]:
        r["slowest"] = (p["step"], ms)

print(f"{'job':<14}{'lines':>6}{'errs':>5}  slowest")
for job, r in report.items():
    step, ms = r["slowest"]
    print(f"{job:<14}{r['lines']:>6}{r['errors']:>5}  {step} {ms}ms")

errors = Counter(p["msg"] for p in parsed if p["level"] == "ERROR")
print(errors.most_common(1))
```

OUTPUT

```
job            lines errs  slowest
orders_daily       6    0  load 1530ms
customers_dim      7    3  load 5003ms
[('timeout connecting to warehouse', 3)]
```

TRAP

`re.match` anchors at the start of the line; `re.search` finds a match anywhere. Use match for structured logs, so a stray substring cannot pass as a record. Compile the pattern once outside the loop.

DATA ENGINEERING NOTE

On a 20 GB log, this same code still works because it reads one line at a time. Wrap the line loop in P13 batches and you can fan it out across processes (page 39).

Regex piece	Matches
`\d{4}-\d{2}-\d{2}`	A date like 2025-04-01
`\s+`	One or more spaces, so INFO and ERROR line up either way
`(?P<name>...)`	A named group, read back with m.group("name") or groupdict()
`(?:...)?`	An optional, non-capturing chunk
`[^"]*`	Anything up to the next double quote

P11 MERGE INTERVALS

DE MEDIUM

Sounds like: *"merge overlapping bookings", "find busy periods", "total time covered by sessions"*

Worked question: Merge each room's bookings into continuous occupied periods, then solve the classic numeric version.

Approach: **1.** sort by start **2.** if the next starts before the last ends, extend **3.** otherwise start a new interval

SOLUTION — PYTHON 3.12

```python
from collections import defaultdict

bookings = [(10, "05-01", "05-04"), (10, "05-03", "05-06"),
            (10, "05-06", "05-08"), (20, "05-02", "05-05"),
            (20, "05-04", "05-07")]

def merge(intervals):
    merged = []
    for start, end in sorted(intervals):
        if merged and start <= merged[-1][1]:     # overlaps last
            merged[-1][1] = max(merged[-1][1], end)
        else:
            merged.append([start, end])
    return merged

rooms = defaultdict(list)
for room, s, e in bookings:
    rooms[room].append((s, e))
for room, spans in rooms.items():
    print(room, merge(spans))
print(merge([(1, 3), (2, 6), (8, 10), (15, 18)]))
```

OUTPUT

```
10 [['05-01', '05-08']]
20 [['05-02', '05-07']]
[[1, 6], [8, 10], [15, 18]]
```

TRAP

Extend with `max(last_end, end)`, not just `end`: a short booking inside a long one would otherwise shrink the period. Here <= also merges back-to-back stays (one ends 05-06, the next starts 05-06). Ask whether touching counts as overlap.

P12 HASH JOIN

DE MEDIUM

Sounds like: *"join these two files without pandas", "enrich orders with customer data"*

Worked question: Join orders to customers in plain Python, count orphan orders, and list customers who never ordered.

Approach: **1.** load the small side into a dict **2.** stream the big side and look up each row **3.** a set of keys gives the anti join

SOLUTION — PYTHON 3.12

```python
import csv, json

customers = {c["customer_id"]: c                 # build side: small
             for c in json.load(open("customers.json"))}

joined, orphans = [], 0
for o in csv.DictReader(open("orders.csv")):   # probe: streamed
    c = customers.get(int(o["customer_id"]))
    if c is None:
        orphans += 1
        continue
    city = (c["address"] or {}).get("city") or "Unknown"
    joined.append((o["order_id"], c["name"], city, o["amount"]))

print(len(joined), "joined,", orphans, "orphans")
for row in joined[8:11]:
    print(row)
seen = {int(r["customer_id"])
        for r in csv.DictReader(open("orders.csv"))}
print("anti join:",
      [c["name"] for i, c in customers.items() if i not in seen])
```

OUTPUT

```
15 joined, 0 orphans
('109', 'Kabir', 'Delhi', '747.00')
('110', 'Sana', 'Unknown', '1998.00')
('111', 'Diya', 'Mumbai', '1999.00')
anti join: ['Zoya']
```

APPROACH

A nested loop compares every order with every customer: O(n x m). A dict on the smaller side makes it O(n + m). This is exactly a hash join, the same algorithm Spark and every database use.

P13 CHUNKED PROCESSING

DE MEDIUM

Sounds like: *"the file is 50 GB and memory is 8 GB", "insert in batches of 1,000", "process an S3 object stream"*

Worked question: Process orders.csv four rows at a time (pretend it is 50 GB) and keep a running delivered total.

Approach: **1.** stream, never read() the whole file **2.** cut the stream into fixed-size batches **3.** carry state between batches

SOLUTION — PYTHON 3.12

```python
import csv
from itertools import islice

def read_in_chunks(path, size):
    with open(path, newline="") as f:
        reader = csv.DictReader(f)
        while chunk := list(islice(reader, size)):
            yield chunk                    # only `size` rows in RAM

running = 0.0
for i, chunk in enumerate(read_in_chunks("orders.csv", 4), 1):
    part = sum(float(r["amount"]) for r in chunk
               if r["status"] == "delivered")
    running += part
    print(f"chunk {i}: {len(chunk)} rows  +{part:>8,.2f}"
          f"  running {running:>9,.2f}")
```

OUTPUT

```
chunk 1: 4 rows  +3,493.00  running  3,493.00
chunk 2: 4 rows  +5,894.00  running  9,387.00
chunk 3: 4 rows  +5,590.00  running 14,977.00
chunk 4: 3 rows  +3,995.00  running 18,972.00
```

DATA ENGINEERING NOTE

The same shape loads a warehouse in batches: build 1,000 rows, `executemany` them, commit, repeat. pandas has it built in: `pd.read_csv(path, chunksize=100_000)` returns one DataFrame per chunk.

P14 RETRY WITH BACKOFF

DE MEDIUM

Sounds like: *"the API returns 429 sometimes", "the warehouse times out at night", "make this call resilient"*

Worked question: Retry a rate-limited API with exponential backoff and full jitter, and give up after a fixed number of tries.

Approach: **1.** catch only retryable errors **2.** wait base x 2^n, capped **3.** add random jitter, stop after N

SOLUTION — PYTHON 3.12

```python
import random, time

random.seed(7)                  # fixed seed so this page is repeatable

def backoff(base=1.0, cap=30.0, tries=6):
    for n in range(tries):
        yield round(random.uniform(0, min(cap, base * 2 ** n)), 2)
print("planned waits:", list(backoff()))

class RateLimited(Exception):
    pass

replies = iter([RateLimited("429"), RateLimited("429"), "200 OK"])
def call_api():
    r = next(replies)
    if isinstance(r, Exception):
        raise r
    return r

def with_retry(fn, tries=5, base=0.01):
    for n in range(tries):
        try:
            return fn()
        except RateLimited as e:
            wait = random.uniform(0, base * 2 ** n)
            print(f"try {n + 1}: {e}, sleep {wait:.4f}s")
            time.sleep(wait)
    raise RuntimeError("gave up after retries")

print(with_retry(call_api))
```

OUTPUT

```
planned waits: [0.32, 0.3, 2.6, 0.58, 8.57, 10.
  97]
try 1: 429, sleep 0.0006s
try 2: 429, sleep 0.0101s
200 OK
```

APPROACH

Doubling the wait gives the server room to recover. Jitter (a random wait between 0 and the cap) stops 100 workers from retrying at the same instant and knocking the server over again. Always cap the wait and the number of tries, then fail loudly.

P15 PAGINATED EXTRACTION

DE MEDIUM

Sounds like: *"pull every page from this API", "the endpoint returns 100 rows and a next link"*

Worked question: Extract every record from a cursor-paginated API (simulated, so it runs here) as one stream of rows.

Approach: **1.** call, yield the rows, read the next cursor **2.** stop when there is no cursor **3.** expose it as a generator

SOLUTION — PYTHON 3.12

```python
SERVER = list(range(101, 116))        # 15 orders on the "server"

def fake_api(cursor=None, limit=4):
    start = int(cursor or 0)
    page = SERVER[start:start + limit]
    nxt = start + limit
    return {"data": page,
            "next_cursor": str(nxt) if nxt < len(SERVER) else None}

def extract_all(limit=4):
    cursor, n = None, 0
    while True:
        resp = fake_api(cursor, limit)
        n += 1
        print(f"  page {n}: {len(resp['data'])} rows,"
              f" next={resp['next_cursor']}")
        yield from resp["data"]
        cursor = resp["next_cursor"]
        if cursor is None:
            break

ids = list(extract_all())
print(len(ids), ids[:3], "...", ids[-1])
```

OUTPUT

```
  page 1: 4 rows, next=4
  page 2: 4 rows, next=8
  page 3: 4 rows, next=12
  page 4: 3 rows, next=None
15 [101, 102, 103] ... 115
```

THE REAL THING — REQUESTS · NOT RUN

```python
import requests

s = requests.Session()
s.headers["Authorization"] = f"Bearer {token}"

def pages(url, params=None):
    while url:
        r = s.get(url, params=params, timeout=30)
        r.raise_for_status()   # 4xx/5xx raise
        body = r.json()
        yield from body["data"]
        url, params = body.get("next"), None
```

P16 MEMOIZE, LRU CACHE

DE MEDIUM

Sounds like: *"avoid repeat API lookups", "cache dimension lookups", "design an LRU cache"*

Worked question: Cache pincode lookups so repeats cost nothing, then build an LRU cache from scratch.

Approach: **1.** same input, same output? then cache it **2.** bound the cache size **3.** evict the least recently used

SOLUTION — PYTHON 3.12

```python
from collections import OrderedDict
from functools import lru_cache

calls = 0
@lru_cache(maxsize=128)
def city_for_pincode(pin):            # pretend: a slow API call
    global calls
    calls += 1
    return {"560001": "Bengaluru", "400001": "Mumbai"}.get(pin, "?")

for pin in ["560001", "400001", "560001", "560001", "999999"]:
    city_for_pincode(pin)
print("real calls:", calls, city_for_pincode.cache_info())

class LRU:                            # build it yourself
    def __init__(self, cap):
        self.cap, self.d = cap, OrderedDict()
    def get(self, k):
        if k not in self.d:
            return None
        self.d.move_to_end(k)         # mark as recently used
        return self.d[k]
    def put(self, k, v):
        self.d[k] = v
        self.d.move_to_end(k)
        if len(self.d) > self.cap:
            self.d.popitem(last=False)   # evict least recent

c = LRU(2)
c.put("a", 1); c.put("b", 2); c.get("a"); c.put("c", 3)
print(list(c.d))                      # "b" was evicted
```

OUTPUT

```
real calls: 3 CacheInfo(hits=2, misses=3, maxsiz
  e=128, currsize=3)
['a', 'c']
```

TRAP

Cache only pure lookups. Caching something that changes (an exchange rate, a status) returns stale data for the life of the process. Unbounded caches (maxsize=None) are a slow memory leak in a long-running job.

P17 VALIDATE AND QUARANTINE

DE MEDIUM

Sounds like: *"reject bad rows but load the rest", "enforce a data contract", "where do bad records go?"*

Worked question: Check every row of orders_dirty.csv against the rules, load the good ones, and park the bad ones with reasons.

Approach: **1.** one function returns the cleaned row and a list of errors **2.** collect every error, not just the first **3.** two outputs: good and quarantine

SOLUTION — PYTHON 3.12

```python
import csv
from datetime import date

ALLOWED = {"delivered", "cancelled", "returned"}

def validate(row):
    errs = []
    if not row["customer_id"].strip():
        errs.append("customer_id missing")
    try:
        date.fromisoformat(row["order_date"])
    except ValueError:
        errs.append(f"bad date {row['order_date']}")
    status = row["status"].strip().lower()
    if status not in ALLOWED:
        errs.append(f"unknown status {status!r}")
    try:
        if float(row["amount"]) <= 0:
            errs.append(f"amount {row['amount']} <= 0")
    except ValueError:
        errs.append(f"amount not numeric {row['amount']!r}")
    return {**row, "status": status}, errs

good, quarantine = [], []
for row in csv.DictReader(open("orders_dirty.csv")):
    clean, errs = validate(row)
    (quarantine if errs else good).append((clean, errs))

print("good:", [(r["order_id"], r["status"]) for r, _ in good])
for r, errs in quarantine:
    print("bad ", r["order_id"], errs)
```

OUTPUT

```
good: [('121', 'delivered'), ('122', 'delivered')
  ]
bad  116 ['amount -50 <= 0']
bad  117 ['customer_id missing']
bad  118 ['bad date 2025-02-30']
bad  119 ["unknown status 'shipped'"]
bad  120 ["amount not numeric 'abc'"]
```

DATA ENGINEERING NOTE

Order 122 arrives as "DELIVERED " and passes after cleaning. Normalise before you validate, or you reject good data. The quarantine goes to its own table with the reasons, so someone can fix the source and replay it.

P18 DIFF TWO SNAPSHOTS

DE MEDIUM

Sounds like: *"what changed since yesterday", "build a CDC feed from daily dumps", "feed an SCD Type 2 load"*

Worked question: Compare two daily customer snapshots and emit inserts, deletes and field-level updates.

Approach: **1.** key both snapshots by id **2.** set algebra on the keys **3.** compare fields for ids in both

SOLUTION — PYTHON 3.12

```python
import csv

def load(path):
    return {r["customer_id"]: r for r in csv.DictReader(open(path))}

old, new = load("snapshot_day1.csv"), load("snapshot_day2.csv")

inserts = sorted(new.keys() - old.keys())
deletes = sorted(old.keys() - new.keys())
updates = []
for k in sorted(old.keys() & new.keys(), key=int):
    changed = {c: (old[k][c], new[k][c])
               for c in new[k] if old[k][c] != new[k][c]}
    if changed:
        updates.append((k, changed))

print("inserts:", inserts, " deletes:", deletes)
for k, ch in updates:
    print("update ", k, ch)
```

OUTPUT

```
inserts: ['9']  deletes: ['8']
update  4 {'city': ('Bengaluru', 'Pune')}
update  6 {'city': ('', 'Chennai')}
```

APPROACH

new - old are inserts, old - new are deletes, the intersection gets a field-by-field compare. Sana's city went from empty to Chennai and Meera moved to Pune: those are exactly the rows an SCD Type 2 load needs.

PART 6 · PYTHON FOR DATA ENGINEERING · CHAPTER 30

PANDAS ESSENTIALS

30

If you know SQL, you already know what pandas does. The work is learning which method maps to which clause.

READ, FILTER, GROUP, AGGREGATE — PYTHON 3.12

```
import pandas as pd

orders = pd.read_csv("orders.csv", parse_dates=["order_date"])
print(orders.shape)
print(orders.dtypes.to_string())

monthly = (orders
    .query("status == 'delivered'")              # WHERE
    .assign(month=lambda d: d.order_date.dt.to_period("M"))
    .groupby("month", as_index=False)             # GROUP BY
    .agg(orders=("order_id", "count"),
         revenue=("amount", "sum"),
         avg=("amount", "mean"))
    .round(1))
print(monthly.to_string(index=False))
```

OUTPUT

```
(15, 5)
order_id               int64
customer_id            int64
order_date     datetime64[us]
status                   str
amount               float64
  month  orders  revenue    avg
2025-01       2   1994.0  997.0
2025-02       2   2847.0 1423.5
2025-03       4   7291.0 1822.8
2025-04       4   6840.0 1710.0
```

REMEMBER

Method chaining reads top to bottom like a SQL query. `parse_dates` turns order_date into a real datetime column; without it, every date is a string and `.dt` fails.

Named aggregation (`revenue=('amount', 'sum')`) gives clean column names in one step.

MERGE AND PIVOT — PYTHON 3.12

```
import pandas as pd

orders = pd.read_csv("orders.csv", parse_dates=["order_date"])
items = pd.read_csv("order_items.csv")
products = pd.read_csv("products.csv")

lines = (items
    .merge(products, on="product_id", how="left",
           validate="many_to_one")               # JOIN + a check
    .merge(orders[["order_id", "order_date", "status"]],
           on="order_id")
    .query("status == 'delivered'")
    .assign(revenue=lambda d: d.qty * d.unit_price,
            month=lambda d: d.order_date.dt.month))

print(lines.pivot_table(index="category", columns="month",
                        values="revenue", aggfunc="sum",
                        fill_value=0))
```

OUTPUT

```
month          1     2     3     4
category
Electronics  799  1499  5096  4396
Home         399  1199  1199  1997
Stationery   796   149   996   447
```

DATA ENGINEERING NOTE

`validate='many_to_one'` makes pandas raise if products has a duplicate product_id, instead of silently doubling your revenue (the fan-out trap from the SQL notes). These numbers match the SQL pivot on page 18 of the SQL End to End Notes.

SQL	pandas	Note
`WHERE status = 'delivered'`	`df.query("status == 'delivered'")` or `df[df.status == "delivered"]`	Boolean mask under the hood
`GROUP BY ... SUM()`	`df.groupby(k).agg(total=("amount", "sum"))`	NaN keys dropped by default (page 70)
`JOIN ... ON`	`a.merge(b, on=k, how="left")`	how = inner / left / right / outer
`SUM() OVER (PARTITION BY k)`	`df.groupby(k)["amount"].transform("sum")`	transform keeps every row
`LAG(x) OVER (...)`	`df.groupby(k)["x"].shift(1)`	Sort first
`ORDER BY ... LIMIT n`	`df.sort_values(c).head(n)` / `df.nlargest(n, c)`	

PART 6 · PYTHON FOR DATA ENGINEERING · CHAPTER 31

PANDAS PITFALLS AND SPEED

31

Four mistakes that return wrong numbers without an error, and the one habit that makes pandas fast.

TRAP 1: GROUPBY DROPS NAN KEYS — PYTHON 3.12

```
import json
import pandas as pd

cust = pd.json_normalize(json.load(open("customers.json")))
orders = pd.read_csv("orders.csv")
df = orders.merge(cust[["customer_id", "address.city"]],
                  on="customer_id")

by_city = df.groupby("address.city")["amount"].sum()
print(len(by_city), "cities, total", by_city.sum())
print(df["amount"].sum(), "<- real total")
print(df.groupby("address.city", dropna=False)["amount"]
        .sum().tail(2))
```

OUTPUT

```
4 cities, total 21270.0
23268.0 <- real total
address.city
Pune    5096.0
NaN     1998.0
Name: amount, dtype: float64
```

TRAP

Sana has no city, so her order sits under a NaN key, and groupby drops NaN keys by default. The city totals no longer add up to the real total. Pass `dropna=False`, or fill the key first with `.fillna('Unknown')`.

TRAP 2 AND 3: CHAINED ASSIGNMENT, SILENT FAN-OUT — PYTHON 3.12

```
import pandas as pd

df = pd.read_csv("orders.csv")
returned = df.status == "returned"

df[returned]["amount"] = 0       # chained: edits a copy
print(df.loc[returned, "amount"].tolist())

df.loc[returned, "amount"] = 0   # one .loc call: works
print(df.loc[returned, "amount"].tolist())

dup = pd.DataFrame({"customer_id": [1, 1],
                    "tier": ["gold", "new"]})
try:
    df.merge(dup, on="customer_id", validate="many_to_one")
except pd.errors.MergeError as e:
    print("MergeError:", e)
```

OUTPUT

```
[798.0]
[0.0]
MergeError: Merge keys are not unique in right dat
  aset; not a many-to-one merge

Duplicates in right:
  customer_id
          1 ...
```

REMEMBER

pandas 3 uses Copy-on-Write: `df[mask][col] = x` always edits a temporary copy and never your DataFrame. Select and assign in one `.loc[rows, col]`.

A merge on a key that is duplicated on the right multiplies rows. `validate=` turns that into an error you can see.

SPEED: VECTORIZE, AND USE THE RIGHT DTYPE — PYTHON 3.12

```
import time
import numpy as np
import pandas as pd

n = 200_000
rng = np.random.default_rng(0)
df = pd.DataFrame({"qty": rng.integers(1, 5, n), "price": 799.0})

def ms(fn):
    start = time.perf_counter(); fn()
    return (time.perf_counter() - start) * 1000

row_fn = lambda r: r.qty * r.price
print(f"vectorized {ms(lambda: df.qty * df.price):8.1f} ms")
print(f"apply      {ms(lambda: df.apply(row_fn, axis=1)):8.1f} ms")
loop = lambda: [row_fn(r) for _, r in df.iterrows()]
print(f"iterrows   {ms(loop):8.1f} ms")

s = pd.Series(["delivered", "cancelled", "returned"] * 100_000)
as_text = s.memory_usage(deep=True) // 1024
as_cat = s.astype("category").memory_usage(deep=True) // 1024
print(as_text, "KB as text,", as_cat, "KB as category")
```

OUTPUT

```
vectorized      2.1 ms
apply        3157.4 ms
iterrows     8050.6 ms
4882 KB as text, 293 KB as category
```

APPROACH

Vectorized operations run in compiled code over whole columns. apply and iterrows call Python once per row. Your timings will differ from these, but the gap is always orders of magnitude. If you write a loop over rows in pandas, stop and look for the column operation.

PART 6 · PYTHON FOR DATA ENGINEERING · CHAPTER 32

PYTHON AND DATABASES

32

sqlite3 ships with Python, so every example here runs anywhere. The same calls work with psycopg (PostgreSQL), mysql-connector and the Snowflake connector.

BULK LOAD, THEN QUERY WITH PARAMETERS — PYTHON 3.12

```
import csv, sqlite3

con = sqlite3.connect(":memory:")
con.execute("""CREATE TABLE orders (order_id INTEGER PRIMARY KEY,
    customer_id INT, order_date TEXT, status TEXT, amount REAL)""")
rows = [tuple(r.values()) for r in csv.DictReader(open("orders.csv"))]
con.executemany("INSERT INTO orders VALUES (?, ?, ?, ?, ?)", rows)
con.commit()

q = "SELECT COUNT(*), SUM(amount) FROM orders WHERE status = ?"
print(con.execute(q, ("delivered",)).fetchone())

user_input = "x' OR '1'='1"
unsafe = f"SELECT COUNT(*) FROM orders WHERE status = '{user_input}'"
print("f-string:", con.execute(unsafe).fetchone()[0], "rows")
safe = "SELECT COUNT(*) FROM orders WHERE status = ?"
print("param:   ", con.execute(safe, (user_input,)).fetchone()[0], "rows")
```

OUTPUT

```
(12, 18972.0)
f-string: 15 rows
param:    0 rows
```

TRAP

Building SQL with f-strings lets the input rewrite your query. Here it turned a filter into "every row". Placeholders (? in sqlite3, %s in psycopg) send the value separately, so it is always treated as data.

TRANSACTIONS AND UPSERTS — PYTHON 3.12

```
with con:                               # commit on success
    con.execute("INSERT INTO orders VALUES "
                "(116, 8, '2025-05-02', 'delivered', 499)")

try:
    with con:                           # rollback on any error
        con.execute("UPDATE orders SET status = 'returned' "
                    "WHERE order_id = 101")
        con.execute("INSERT INTO orders VALUES "
                    "(101, 1, '2025-01-10', 'dup', 0)")    # PK clash
except sqlite3.IntegrityError as e:
    print("rolled back:", e)
q101 = "SELECT status FROM orders WHERE order_id = 101"
print(con.execute(q101).fetchone())

con.execute("""INSERT INTO orders
    VALUES (115, 5, '2025-04-28', 'returned', 1598)
    ON CONFLICT(order_id) DO UPDATE SET status = excluded.status""")
print(con.execute("SELECT order_id, status FROM orders "
                  "WHERE order_id IN (115, 116)").fetchall())
```

OUTPUT

```
rolled back: UNIQUE constraint failed: order
  s.order_id
('delivered',)
[(115, 'returned'), (116, 'delivered')]
```

REMEMBER

`with con:` wraps the block in a transaction: commit if it finishes, rollback if it raises. The UPDATE to order 101 was undone along with the failed insert.

`ON CONFLICT ... DO UPDATE` is the upsert in SQLite and PostgreSQL; MySQL uses ON DUPLICATE KEY UPDATE.

PANDAS IN AND OUT OF SQL — PYTHON 3.12

```
import pandas as pd

pd.read_csv("products.csv").to_sql("products", con, index=False)
q = ("SELECT category, COUNT(*) AS n, MAX(price) AS top "
     "FROM products GROUP BY category")
print(pd.read_sql(q, con))
```

OUTPUT

```
      category  n   top
0  Electronics  4  1999
1         Home  2  1199
2   Stationery  2   249
```

For large results use `cursor.fetchmany(10_000)` in a loop, or `pd.read_sql(..., chunksize=)`, so the result never has to fit in memory at once.

PART 6 · PYTHON FOR DATA ENGINEERING · CHAPTER 33

BUILD AN ETL PIPELINE

33

One file, three pure-ish functions and a main(). Run it twice for the same day and the warehouse does not change: that is the property interviewers probe.

ORDERS_DAILY.PY — PYTHON 3.12

```
import argparse, csv, logging, sqlite3, sys

logging.basicConfig(level=logging.INFO, stream=sys.stdout,
                    format="%(levelname)s %(name)s: %(message)s")
log = logging.getLogger("orders_daily")

def extract(path, day):                       # E: stream one day
    with open(path, newline="") as f:
        for r in csv.DictReader(f):
            if r["order_date"] == day:
                yield r

def transform(rows):                          # T: pure, testable
    for r in rows:
        yield (int(r["order_id"]), int(r["customer_id"]), r["order_date"],
               r["status"].strip().lower(), round(float(r["amount"]), 2))

def load(con, day, rows):                     # L: idempotent
    with con:                                 # one transaction
        con.execute("DELETE FROM fact_orders WHERE order_date = ?", (day,))
        cur = con.executemany("INSERT INTO fact_orders VALUES (?,?,?,?,?)", rows)
    return cur.rowcount

def main(argv=None):
    p = argparse.ArgumentParser(description="Load one day of orders")
    p.add_argument("--date", required=True)
    p.add_argument("--src", default="orders.csv")
    args = p.parse_args(argv)
    con = sqlite3.connect("warehouse.db")
    con.execute("""CREATE TABLE IF NOT EXISTS fact_orders (order_id INT PRIMARY KEY,
                   customer_id INT, order_date TEXT, status TEXT, amount REAL)""")
    n = load(con, args.date, transform(extract(args.src, args.date)))
    total = con.execute("SELECT COUNT(*) FROM fact_orders").fetchone()[0]
    log.info("date=%s loaded=%d table_rows=%d", args.date, n, total)
    return 0

if __name__ == "__main__":
    main(["--date", "2025-04-20"])
    main(["--date", "2025-04-20"])            # rerun the same day
    main(["--date", "2025-04-28"])
    try:
        main([])                              # missing --date
    except SystemExit as e:
        log.error("bad arguments, exit code %s", e.code)
```

OUTPUT

```
INFO orders_daily: date=2025-04-20 loaded=1 table_rows=1
INFO orders_daily: date=2025-04-20 loaded=1 table_rows=1
INFO orders_daily: date=2025-04-28 loaded=1 table_rows=2
ERROR orders_daily: bad arguments, exit code 2
```

Decision in the code	Why it matters in production
DELETE the day, then INSERT, in one transaction	Reruns and backfills give the same table. A crash halfway rolls back, so there is never a half-loaded day.
extract and transform are generators	Memory stays flat whether the day has 15 rows or 15 million.
logging, not print, and a pure transform	Log viewers can filter levels and names. The pure transform is unit tested on the next page.
argparse with --date	The scheduler passes the date. The same file runs today, for a backfill, or by hand.
bad input exits with code 2	A non-zero exit code is how the scheduler knows the task failed.

INTERVIEW TIP

"How would you make this pipeline idempotent?" Name the mechanism, not the adjective: overwrite a partition in a transaction, or MERGE on a key. Then mention what breaks it: appending without a key, using now() instead of the run date, or side effects such as emails inside the load step.

PART 6 · PYTHON FOR DATA ENGINEERING · CHAPTER 34

TESTING AND DATA QUALITY

34

Unit tests check your code on known inputs. Data quality checks check the data on every run. A pipeline needs both.

TRANSFORM.PY — PYTHON 3.12

```python
STATUS = {"delivered": "delivered", "dlvd": "delivered",
          "cancelled": "cancelled",
          "canceled": "cancelled", "returned": "returned"}

def normalize_status(raw):
    key = (raw or "").strip().lower()
    return STATUS.get(key, "unknown")

def to_amount(text):
    value = float(str(text).replace(",", ""))
    if value <= 0:
        raise ValueError(f"amount must be > 0: {value}")
    return round(value, 2)
```

TEST_TRANSFORM.PY — PYTEST

```python
import pytest
from transform import normalize_status, to_amount

@pytest.mark.parametrize("raw, expected", [
    (" DLVD ", "delivered"),
    ("Canceled", "cancelled"),
    (None, "unknown"),
    ("shipped", "unknown"),
])
def test_normalize_status(raw, expected):
    assert normalize_status(raw) == expected

def test_amount_parses_commas():
    assert to_amount("1,198.50") == 1198.5

def test_amount_rejects_negative():
    with pytest.raises(ValueError, match="> 0"):
        to_amount("-50")
```

$ PYTEST -V

```
test_normalize_status[ DLVD -delivered] PASSED
test_normalize_status[Canceled-cancelled] PASSED
test_normalize_status[None-unknown] PASSED
test_normalize_status[shipped-unknown] PASSED
test_amount_parses_commas PASSED
test_amount_rejects_negative PASSED
6 passed
```

REMEMBER

`@pytest.mark.parametrize` runs one test per input row, so one function covers every messy status you have seen in production. `pytest.raises` checks the bad path, which is half the point of testing a transform.

DATA QUALITY CHECKS THAT RUN EVERY LOAD — PYTHON 3.12

```python
import json
import pandas as pd

def run_checks(orders, customers):
    known = set(customers.customer_id)
    checks = {
        "order_id unique": orders.order_id.is_unique,
        "no null customer_id": orders.customer_id.notna().all(),
        "status allowed": orders.status.isin(
            ["delivered", "cancelled", "returned"]).all(),
        "amount > 0": (pd.to_numeric(orders.amount,
                       errors="coerce") > 0).all(),
        "customer exists": orders.customer_id.isin(known).all(),
    }
    return [name for name, ok in checks.items() if not ok]

customers = pd.json_normalize(json.load(open("customers.json")))
for path in ["orders.csv", "orders_dirty.csv"]:
    df = pd.read_csv(path)
    failed = run_checks(df, customers)
    print(path, "->", "PASS" if not failed else f"FAIL {failed}")
```

OUTPUT

```
orders.csv -> PASS
orders_dirty.csv -> FAIL ['no null customer_id',
   'status allowed', 'amount > 0', 'customer exi
  sts']
```

DATA ENGINEERING NOTE

Run checks after transform and before load, and stop the load when a blocking check fails. Great Expectations, Soda, pandera and dbt tests are this same idea with reporting on top. Knowing the plain version is what interviews test.

	Unit test	Integration test	Data quality check
Checks	One function on fixed inputs	Steps together on a small real file and a test database	The real data, every run
Runs	On every commit, in CI	Before a deploy	Inside the pipeline, after transform
Catches	Logic bugs: wrong mapping, bad rounding	Wiring bugs: wrong path, schema mismatch, bad SQL	Source changes: new status, nulls, duplicates, late files
Example here	test_normalize_status	run main() on `orders.csv` into a temp sqlite file	run_checks() on orders_dirty.csv

PART 6 · PYTHON FOR DATA ENGINEERING · CHAPTER 35

PANDAS TO PYSPARK, AND AIRFLOW

35

When the data no longer fits on one machine, the same ideas move to PySpark. When it has to run every night, Airflow runs it.

EACH CUSTOMER'S BIGGEST DELIVERED ORDER — PYSPARK 4 · LOCAL

```
from pyspark.sql import SparkSession, Window, functions as F

spark = (SparkSession.builder.master("local[2]")
         .appName("orders").config("spark.ui.enabled", "false")
         .getOrCreate())
spark.sparkContext.setLogLevel("ERROR")

orders = spark.read.csv("orders.csv", header=True, inferSchema=True)
w = Window.partitionBy("customer_id").orderBy(F.col("amount").desc())

(orders
    .filter(F.col("status") == "delivered")
    .withColumn("rnk", F.dense_rank().over(w))
    .filter("rnk = 1")
    .select("customer_id", "order_id", "amount")
    .orderBy("customer_id")
    .show())
spark.stop()
```

OUTPUT

```
+-----------+--------+------+
|customer_id|order_id|amount|
+-----------+--------+------+
|          1|     114|2397.0|
|          2|     111|1999.0|
|          3|     109| 747.0|
|          4|     106|1048.0|
|          5|     108|3498.0|
|          6|     110|1998.0|
|          7|     112| 846.0|
+-----------+--------+------+
```

REMEMBER

This ran on a real local Spark session while the PDF was built. Same window logic as P1 in the SQL notes: partition by customer, rank by amount, keep rank 1.

Nothing executes until `.show()`: Spark builds a plan from the transformations and runs it on the action, like a generator.

Task	pandas	PySpark
Filter	`df[df.status == 'delivered']`	`df.filter(F.col('status') == 'delivered')`
New column	`df.assign(rev=df.qty * df.price)`	`df.withColumn('rev', F.col('qty') * F.col('price'))`
Group and sum	`df.groupby('c').agg(t=('amt', 'sum'))`	`df.groupBy('c').agg(F.sum('amt').alias('t'))`
Join	`a.merge(b, on='id', how='left')`	`a.join(b, 'id', 'left')`
Window rank	`df.groupby('c')['amt'].rank(ascending=False)`	`F.dense_rank().over(Window.partitionBy('c')...)`
When it runs	Immediately, in memory	Lazily, on an action (show, count, write)
Scale	One machine, data must fit in RAM	A cluster, data split into partitions

THE SAME PIPELINE AS AN AIRFLOW DAG — AIRFLOW 3 · NOT RUN

```
from airflow.sdk import dag, task
import pendulum

@dag(schedule="@daily", catchup=False, tags=["orders"],
     start_date=pendulum.datetime(2025, 4, 1, tz="Asia/Kolkata"))
def orders_daily():

    @task(retries=3)
    def extract(ds=None) -> str:     # ds = logical date
        return f"s3://raw/orders/{ds}.csv"

    @task
    def transform(path: str) -> str:
        return path.replace("raw", "clean")

    @task
    def load(path: str) -> int:
        ...                                # MERGE into the warehouse

    load(transform(extract()))             # sets the task order

orders_daily()
```

DATA ENGINEERING NOTE

Each @task becomes a node in the DAG and Airflow handles schedule, retries and logs. Return values pass between tasks through XCom, so pass paths and ids, not whole datasets.

Airflow 3 imports from `airflow.sdk`; on Airflow 2 the same decorators come from `airflow.decorators`.

INTERVIEW TIP

"Why use the run date instead of today?" So a rerun of 1 April next week still loads 1 April. That is the same idempotency rule as page 72.

PART 7 · PRACTICE, PREP FLOW AND REVISION

INTERVIEW SET: PYTHON CODING

PY

Mixed questions in the style of DE coding rounds. Name the pattern before you write anything; it is printed in each approach line.

Q39. From events.jsonl, return each user's latest event and the device it came from. DE EASY

Approach: Dedupe, keep latest (P4): one dict keyed by user, replace when the timestamp is newer.

```
import json

latest = {}
for line in open("events.jsonl"):
    e = json.loads(line)
    cur = latest.get(e["user_id"])
    if cur is None or e["ts"] > cur["ts"]:
        latest[e["user_id"]] = e
for u, e in sorted(latest.items()):
    print(u, e["event"], e["props"]["device"], e["ts"][11:16])
```

OUTPUT

```
1 view desktop 14:00
2 cart desktop 10:40
3 checkout mobile 11:15
4 view tablet 12:00
```

Q40. For each customer, the longest gap in days between two consecutive orders. DE MEDIUM

Approach: Group by customer (P3), sort dates, compare neighbours with pairwise.

```
import csv
from collections import defaultdict
from datetime import date
from itertools import pairwise

dates = defaultdict(list)
for r in csv.DictReader(open("orders.csv")):
    d = date.fromisoformat(r["order_date"])
    dates[r["customer_id"]].append(d)

gaps = {c: max((b - a).days for a, b in pairwise(sorted(ds)))
        for c, ds in dates.items() if len(ds) > 1}
print(dict(sorted(gaps.items(), key=lambda kv: -kv[1])))
```

OUTPUT

```
{'5': 47, '1': 46, '4': 45, '2': 43, '3': 38}
```

Q41. Find every pair of orders whose amounts add up to exactly 2847.00. DE MEDIUM

Approach: Two-sum with a dict: for each amount, look up the complement among amounts seen so far. One pass, O(n).

```
import csv
from collections import defaultdict
from decimal import Decimal

TARGET = Decimal("2847.00")
seen = defaultdict(list)                # amount -> order ids
pairs = []
for r in csv.DictReader(open("orders.csv")):
    amt = Decimal(r["amount"])
    for other in seen[TARGET - amt]:
        pairs.append((other, r["order_id"]))
    seen[amt].append(r["order_id"])
print(pairs)
```

OUTPUT

```
[('103', '105'), ('105', '113')]
```

Q42. Unflatten {"address.city": "Pune", "address.pincode": "411001", "name": "Rohan"} back into nested JSON. DE HARD

Approach: The reverse of P7: split each key on dots and walk down, creating dicts with setdefault.

```
def unflatten(flat, sep="."):
    root = {}
    for key, value in flat.items():
        *parents, leaf = key.split(sep)
        node = root
        for p in parents:
            node = node.setdefault(p, {})
        node[leaf] = value
    return root

print(unflatten({"address.city": "Pune",
                 "address.pincode": "411001", "name": "Rohan"}))
```

OUTPUT

```
{'address': {'city': 'Pune', 'pincode': '411001'
  }, 'name': 'Rohan'}
```

PART 7 · PRACTICE, PREP FLOW AND REVISION

INTERVIEW SET: PANDAS

PD

The pandas round usually mirrors the SQL round. These five come up in some form in most DA and DE loops.

Q43. The top customer by delivered spend in each month. DA DE MEDIUM

Approach: Aggregate to (month, customer), then idxmax within each month to pick the row.

```
import pandas as pd

o = pd.read_csv("orders.csv", parse_dates=["order_date"])
spend = (o[o.status == "delivered"]
         .groupby([o.order_date.dt.to_period("M"), "customer_id"])
         ["amount"].sum().reset_index())
top = spend.loc[spend.groupby("order_date")["amount"].idxmax()]
print(top.to_string(index=False))
```

OUTPUT

```
order_date  customer_id  amount
   2025-01            1  1198.0
   2025-02            1  1499.0
   2025-03            5  3498.0
   2025-04            1  2397.0
```

Q44. Month-over-month growth in delivered revenue, as a percentage. DA EASY

Approach: Resample to months, then pct_change compares each month with the one before.

```
import pandas as pd

o = pd.read_csv("orders.csv", parse_dates=["order_date"])
rev = (o[o.status == "delivered"]
       .set_index("order_date")["amount"].resample("MS").sum())
print(pd.DataFrame({"revenue": rev,
                    "mom_pct": (rev.pct_change() * 100).round(1)}))
```

OUTPUT

```
            revenue  mom_pct
order_date
2025-01-01   1994.0      NaN
2025-02-01   2847.0     42.8
2025-03-01   7291.0    156.1
2025-04-01   6840.0     -6.2
```

Q45. Daily delivered revenue for 1 to 7 April with the missing days as 0, plus a 3-day rolling sum. DA DE MEDIUM

Approach: Aggregate per day, reindex onto a full date range, fill zeros, then rolling(3).

```
import pandas as pd

o = pd.read_csv("orders.csv", parse_dates=["order_date"])
delivered = o[o.status == "delivered"]
daily = delivered.groupby("order_date")["amount"].sum()
days = pd.date_range("2025-04-01", "2025-04-07", freq="D")
daily = daily.reindex(days, fill_value=0)
print(pd.DataFrame({"revenue": daily,
                    "rev_3d": daily.rolling(3, 1).sum()}))
```

OUTPUT

```
            revenue  rev_3d
2025-04-01      0.0     0.0
2025-04-02   1999.0  1999.0
2025-04-03      0.0  1999.0
2025-04-04      0.0  1999.0
2025-04-05    846.0   846.0
2025-04-06      0.0   846.0
2025-04-07      0.0   846.0
```

Q46. Customers whose every order was delivered. DA EASY

Approach: groupby + all() on a boolean column. A "did all of X" question.

```
import pandas as pd

o = pd.read_csv("orders.csv")
flag = o.assign(d=o.status.eq("delivered"))
ok = flag.groupby("customer_id")["d"].all()
print(ok[ok].index.tolist())
```

OUTPUT

```
[2, 5, 6, 7]
```

Q47. For each customer: first order date, number of orders, and days until their second order. DA DE MEDIUM

Approach: Sort, number each customer's orders with cumcount, then line up order 0 and order 1 by customer.

```
import pandas as pd

o = pd.read_csv("orders.csv", parse_dates=["order_date"])
o = o.sort_values(["customer_id", "order_date"])
o["n"] = o.groupby("customer_id").cumcount()      # 0, 1, 2...
first = o[o.n == 0].set_index("customer_id").order_date
second = o[o.n == 1].set_index("customer_id").order_date
print(pd.DataFrame({"first": first.dt.date,
                    "orders": o.groupby("customer_id").size(),
                    "days_to_2nd": (second - first).dt.days}))
```

OUTPUT

```
                first  orders  days_to_2nd
customer_id
1          2025-01-10       4         23.0
2          2025-01-15       3         34.0
3          2025-02-10       2         38.0
4          2025-03-01       2         45.0
5          2025-03-12       2         47.0
6          2025-03-22       1          NaN
7          2025-04-05       1          NaN
```

PART 7 · PRACTICE, PREP FLOW AND REVISION

CONCEPT QUESTIONS, SHORT ANSWERS

Q&A

The no-keyboard part of the round. Answer in two sentences, then give one example from the store data.

List vs tuple?
Lists are mutable, tuples are not. Tuples can be dict keys and signal "fixed record".

is vs ==?
is checks identity (same object), == checks value. Use is only for None, True, False (page 26).

Why is a mutable default argument a bug?
Defaults are created once at def time, so every call shares the same list (page 26).

Shallow vs deep copy?
Shallow copies the outer container only; nested objects are still shared. deepcopy copies everything.

Why are dict and set lookups O(1)?
Keys are hashed to a slot, so lookups do not scan. That is why keys must be hashable.

Generator vs list?
A generator yields one item at a time and can be used once. A list holds everything and can be reused (page 35).

Iterator vs iterable?
An iterable can give you an iterator (iter()). An iterator gives values with next() until StopIteration.

What does yield from do?
Passes every value from a sub-generator through, which keeps recursive generators short.

What is a decorator?
A function that takes a function and returns a wrapped one. Used for timing, retries, logging, caching (page 38).

Why functools.wraps?
It copies the name and docstring onto the wrapper, so logs and tools see the real function.

What is a context manager?
An object with __enter__/__exit__ used with "with". Cleanup runs even when the block raises.

***args and **kwargs?**
Extra positional arguments as a tuple and extra keyword arguments as a dict (page 31).

What is the GIL?
A lock that lets one thread run Python bytecode at a time. Threads still help for I/O (page 39).

Threads vs processes vs asyncio?
Threads for blocking I/O, processes for CPU work, asyncio for very many concurrent I/O calls.

Why Decimal for money?
Floats store binary approximations, so 0.1 + 0.2 is not 0.3. Decimal is exact in base 10 (page 27).

What is a dataclass?
A class whose __init__, __repr__ and __eq__ are generated from typed fields. frozen=True makes it read-only.

EAFP vs LBYL?
Try and catch the error vs check first. Python leans EAFP; it avoids race conditions between the check and the use.

How do you read a 50 GB file?
Stream it: iterate lines or use chunks (P13), never read() it whole. Or move to Spark.

JSON vs JSON Lines?
JSON is one document you load whole. JSON Lines is one record per line, so you can stream and split it.

Why parameterized SQL?
Values go separately from the query text, which prevents SQL injection and quoting bugs (page 71).

apply vs vectorized in pandas?
apply calls Python per row; vectorized runs compiled code on whole columns, far faster (page 70).

loc vs iloc?
loc selects by label and boolean mask, iloc by integer position.

merge vs concat?
merge joins on keys like SQL JOIN. concat stacks frames like UNION ALL.

Why does groupby lose rows?
NaN keys are dropped by default. Use dropna=False or fill them (page 70).

Parquet vs CSV?
Parquet is columnar, typed and compressed, so it is smaller and reads only needed columns. CSV is plain text with no types.

What makes a pipeline idempotent?
Rerunning gives the same result: overwrite a partition or MERGE on a key, never blind append (page 72).

logging vs print?
logging has levels, names and formats, and can be routed and filtered. print cannot.

What does if __name__ == "__main__" do?
Runs code only when the file is executed, not when imported. Needed for multiprocessing.

Unit test vs data quality check?
Unit tests check code on fixed inputs. DQ checks check real data on every run (page 73).

pandas vs PySpark?
pandas runs eagerly on one machine. PySpark runs lazily across a cluster when data outgrows one machine (page 74).

What does Big-O measure?
How the work grows as the input grows, not seconds. Drop constants, keep the biggest term (page 41).

What counts as space complexity?
Extra memory beyond the input: new collections, and the call stack for recursion (page 42).

BFS vs DFS?
BFS uses a queue and goes level by level (shortest steps). DFS uses a stack or recursion and goes deep first (page 53).

When is greedy wrong?
When a locally best step blocks a better total, like coins [1, 3, 4] for 6. Then use DP (page 56).

PART 7 · PRACTICE, PREP FLOW AND REVISION

THE COMPLEXITY CHEAT SHEET

O(N)

Every cost you will be asked to state, on one page. If you can fill this table from memory, the complexity questions are free marks.

Data structures

Structure	Access	Search	Insert	Delete	Space	Best for
list (array)	O(1)	O(n)	O(1) end, O(n) front	O(1) end, O(n) front	O(n)	Ordered data, indexing
dict / set	O(1)*	O(1)*	O(1)*	O(1)*	O(n)	Lookups, counting, dedupe
deque	O(1) ends	O(n)	O(1) both ends	O(1) both ends	O(n)	Queues, windows, BFS
heap (heapq)	O(1) min	O(n)	O(log n)	O(log n) min	O(n)	Top-k, scheduling, merge k
sorted list + bisect	O(1)	O(log n)	O(n)	O(n)	O(n)	Static sorted lookups
balanced BST / B-tree	O(log n)	O(log n)	O(log n)	O(log n)	O(n)	Database indexes
linked list	O(n)	O(n)	O(1) at a node	O(1) at a node	O(n)	Rare in Python

* average case. Worst case O(n) with heavy hash collisions, which you can mention but rarely need to worry about.

Algorithms

Algorithm	Time	Space	Page
Binary search	O(log n)	O(1)	50
Sorting (Timsort, merge sort)	O(n log n)	O(n)	51
Counting sort	O(n + k)	O(k)	51
Two pointers, sliding window	O(n)	O(1) to O(k)	45, 47
Prefix sums	O(n) build, O(1) query	O(n)	43, 46
BFS / DFS on V nodes, E edges	O(V + E)	O(V)	53, 54
Topological sort (Kahn)	O(V + E)	O(V)	54
Top-k with a heap	O(n log k)	O(k)	55, 59
Merge k sorted streams	O(n log k)	O(k)	61
Subsets / permutations	O(2^n) / O(n!)	O(n) depth	52
DP over one dimension	O(n x choices)	O(n), often O(1)	56
Hash join of n and m rows	O(n + m)	O(smaller side)	65

INTERVIEW TIP

Say it like this: "This is O(n) time because we read each order once, and O(k) space for the k customers in the dict. Sorting the output adds O(k log k)." Name the cost, then say where it comes from.

APPROACH

When they ask "can you do better?" check, in order: a hash map instead of a nested loop, sorting first, two pointers instead of a set (saves space), a heap instead of a full sort, and a prefix sum instead of re-summing.

Mistake when stating complexity	Say this instead
"It is O(n)" for a loop that calls x in some_list	O(n x m): the in on a list is its own loop (page 44)
Forgetting the sort you added	O(n log n) overall, even if the rest is O(n)
Calling recursion O(1) space	O(depth) for the call stack
O(n) for building a string with += in a loop	Up to O(n²) copying; join is O(n)
Ignoring the output size	Listing all subsets is at least O(2^n): the answer itself is that big

PART 7 · PRACTICE, PREP FLOW AND REVISION

THE PREPARATION FLOW

10W

A week-by-week path from your first line of Python to interview-ready, built for about two hours a day. Already know the basics? Start at week 3.

Week	Read	Practise	Done when
1	Pages 5 to 14: running Python, variables, numbers, strings, conditions, loops	Type every lesson yourself; change a value and predict the output before running	You write a loop with an if inside without looking anything up
2	Pages 15 to 24: lists, tuples, sets, dicts, functions, modules, errors, files	Rebuild the sales report (page 23) from a blank file; practice set 0	You turn a small task into a few clear functions
3	Pages 25 to 34: approach, foundations and their traps	Retype every snippet on the dataset; break each one on purpose	You explain every trap in Part 2 without looking
4	Pages 35 to 44: advanced Python, Big-O, space, built-in costs	Write chunked(), retry and a timing decorator from memory; 10 easy problems	You state time and space for any snippet you see
5	Pages 45 to 49: arrays, hashing, windows, stacks, linked lists	15 easy-medium problems, 25 minutes each	You solve two-sum, subarray-sum-k and longest-unique cold
6	Pages 50 to 57: search, sort, recursion, trees, graphs, heaps, DP	15 medium problems; redo every failure from week 5	You write binary search and BFS cold
7	Pages 58 to 63: patterns P1 to P9	Each worked question cold, then a variant on another file	You name the pattern of a new question in 10 seconds
8	Pages 64 to 68: patterns P10 to P18	Then timed: 2 data questions in 45 minutes	You finish a HARD data question in under 25 minutes
9	Pages 69 to 74: pandas, databases, ETL, tests, PySpark	Rebuild the ETL pipeline from a blank file; write its tests	You explain idempotency and scaling with your own code
10	Pages 75 to 80: sets, concepts, cheat sheet, revision	Three mock interviews out loud: DSA, data round, mixed	Two mocks in a row where you talk the whole time

A two-hour day

Minutes	Do this
0 to 20	Redo one problem from 3 days ago, cold
20 to 80	Two new problems, 25-minute timer each, talking out loud
80 to 100	Read the solution for anything you missed; write the pattern and the trap in one line
100 to 120	Read one page of this book and run its code

The one-problem loop

- Read the question, then spend 2 minutes on steps 1 to 3 of the approach (page 25) before any code.
- Code with a visible 25-minute timer, saying what you are doing.
- State time and space before you run it.
- Rate it: clean, slow, or failed. Failed and slow go back on the list for 3 days later, then 7.
- Keep a tracker: problem, pattern, rating, date, the one thing you missed.

REMEMBER

You are ready when you can take an unseen medium problem, name its pattern, say the approach and its cost in two minutes, and finish working code in 25, talking the whole time. Not when you have done a certain number of problems.

DATA ENGINEERING NOTE

The rest of the loop. This book covers the Python and DSA side. Product-company DE loops also test SQL, data modelling, system design and behaviour. Pair this flow with the SQL End to End Notes and practise one design question a week from week 5.

A tracker that keeps you honest (example rows)

Date	Problem	Pattern	Rating	What I missed	Redo on
Day 15	Subarray sum equals k	Prefix sum + hash map	Failed	Did not seed the map with {0: 1}	Day 18, Day 25
Day 16	Merge intervals	Sort, then extend	Slow	Used end instead of max(end)	Day 19
Day 16	Top k customers	Size-k heap	Clean		

PART 7 · PRACTICE, PREP FLOW AND REVISION

THE ONE-PAGE REVISION SHEET

80

Read this the night before the interview. Everything on it is covered in detail somewhere in the previous 79 pages.

PATTERNS TO WRITE COLD — CHEAT SHEET

```
# count / group / dedupe            (P1, P3, P4)
counts[k] = counts.get(k, 0) + 1
groups.setdefault(k, []).append(row)
if k not in best or row["ts"] > best[k]["ts"]:
    best[k] = row

# top k                             (P2)
heapq.nlargest(k, items, key=lambda x: x[1])

# window                            (P5)
win.append(x); total += x
if len(win) > k: total -= win.popleft()

# streak start                      (P9)
if d - ONE_DAY not in days: ...

# merge intervals                   (P11)
if out and s <= out[-1][1]:
    out[-1][1] = max(out[-1][1], e)

# stream a big file in batches      (P13)
while batch := list(islice(rows, n)): ...

# retry                             (P14)
time.sleep(random.uniform(0, min(cap, base * 2 ** n)))
```

TRAP

The nine traps

- Mutable default argument shared across calls.
- Shallow copy of nested data leaks edits.
- float for money. Use Decimal from a string.
- Deleting from a dict while iterating it.
- zip silently drops extra items (strict=True).
- Generators are single use.
- itertools.groupby needs sorted input.
- f-string SQL. Always use placeholders.
- pandas groupby drops NaN keys.

APPROACH

The approach, short form

1 pin the I/O, 2 edge cases, 3 brute force and its cost, 4 pick the structure, 5 small functions, 6 test and state Big-O.

Pick the structure

Need	Reach for	Cost
Look something up by key	`dict`	O(1) average
Is x in the collection?	`set`	O(1) average
Add and remove at both ends	`collections.deque`	O(1)
Smallest or largest k	`heapq`	O(n log k)
Huge input, one pass	generator	O(1) memory

The last 48 hours

When	Do this	Skip this
2 days before	Redo your 10 "failed" tracker problems cold. Read pages 4, 78 and 80.	New topics. You will not learn graphs well in two days.
1 day before	One timed mock out loud. Re-read the traps. Check your setup: editor, camera, network.	Grinding problems late into the night.
In the room	Say the I/O, edge cases and brute force before coding. State Big-O at the end. Test one edge case by hand.	Typing in silence, or going quiet when stuck: say what you are trying.

FROM DURGESH

One last thing. Coding rounds rarely fail on a missing colon. They fail on code that runs and returns the wrong answer, written in silence. Say your input and output first, test the edge cases out loud, and state the cost. Everything else in this book is detail.

More notes, roadmaps and interview prep at **prepnplaced.com**.